

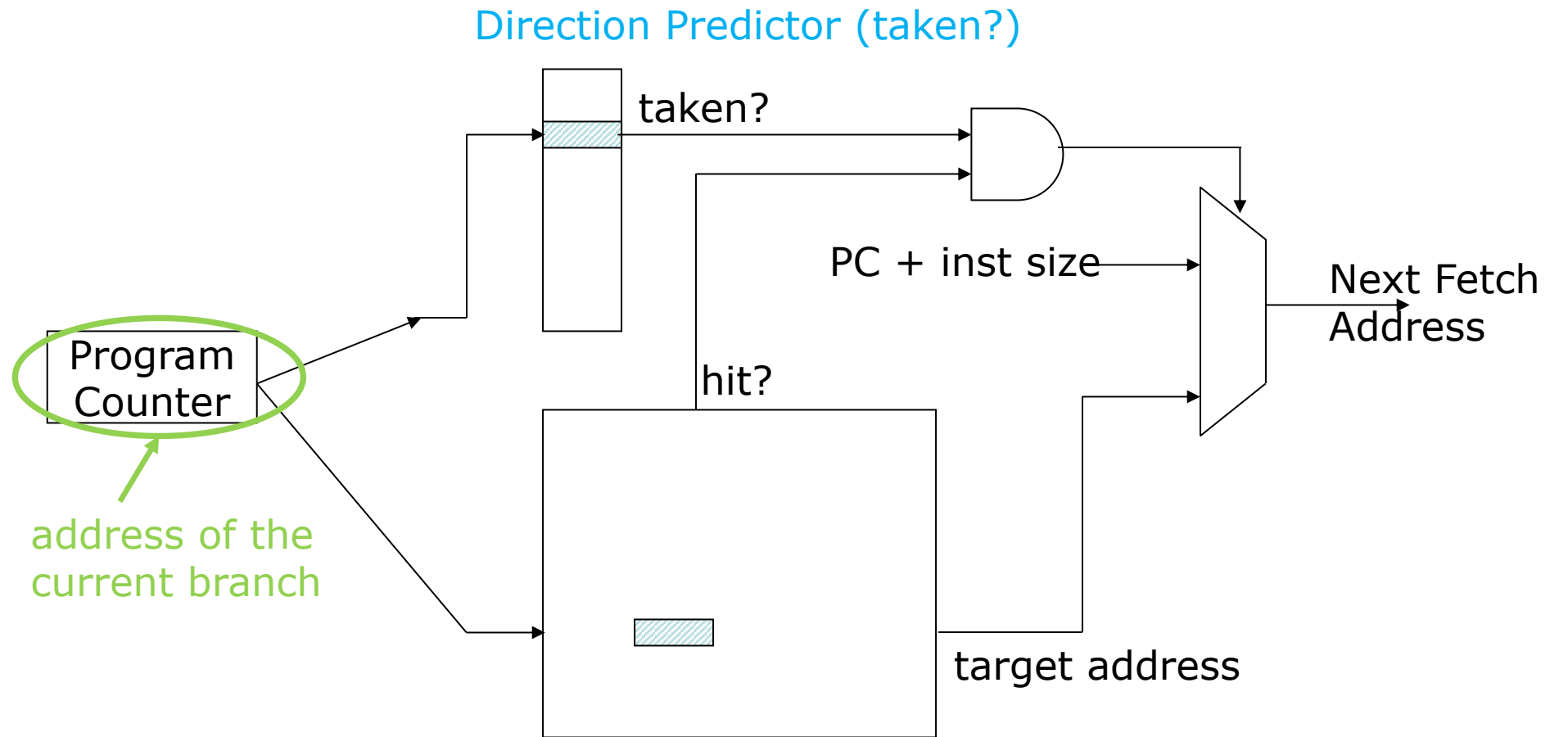
Instruction-Level Parallelism
Dynamic Parallelism

Dynamic?

Dynamic vs Static

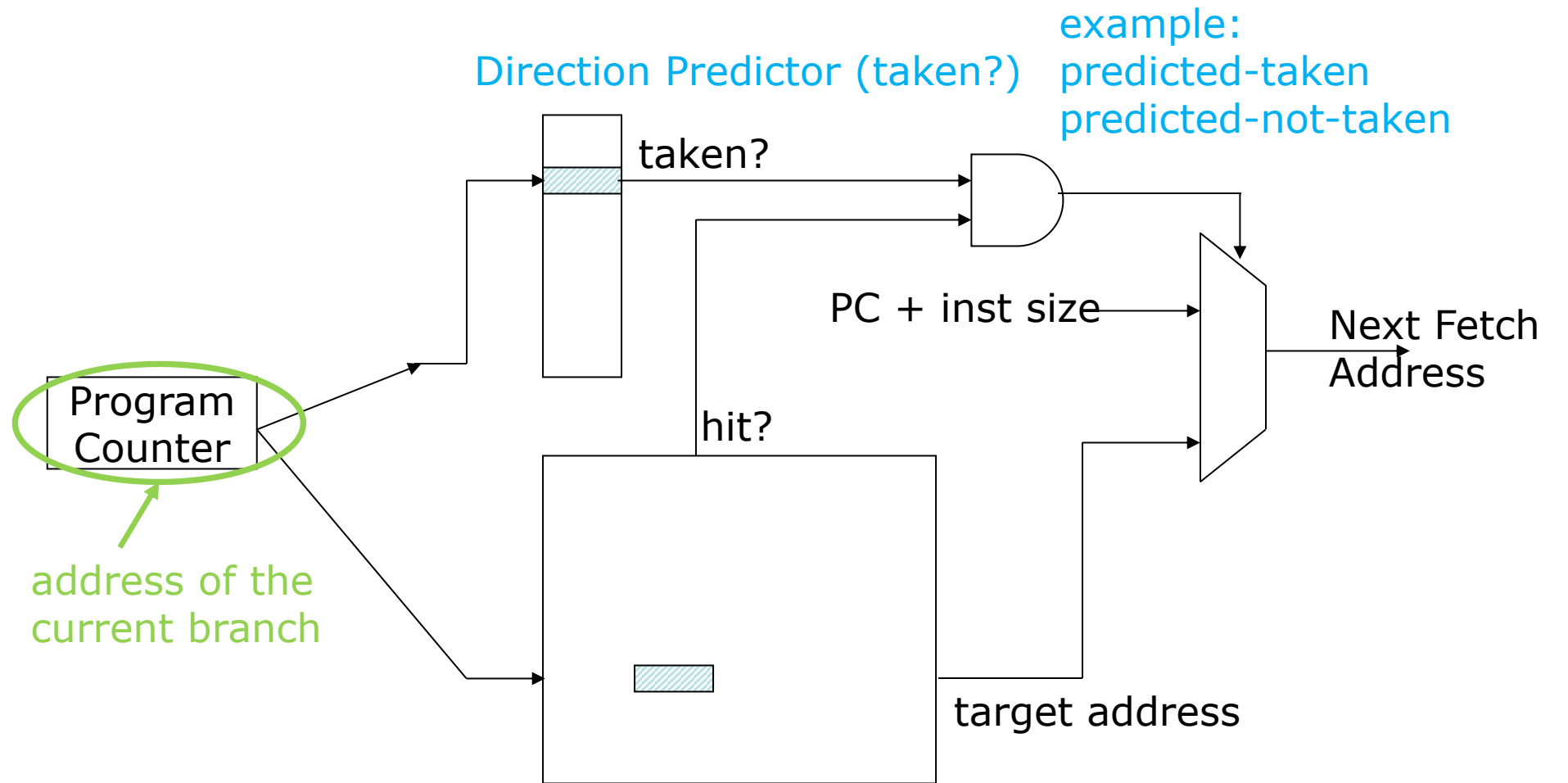
Static

Static Branch Prediction



Cache of Target Addresses (BTB: Branch Target Buffer)

Static Branch Prediction



Cache of Target Addresses (BTB: Branch Target Buffer)

Static Branch Prediction

favor branches biased toward taken/untaken

example:

predicted-taken

predicted-not-taken

Direction Predictor (taken?)

taken?

PC + inst size

Next Fetch Address

hit?

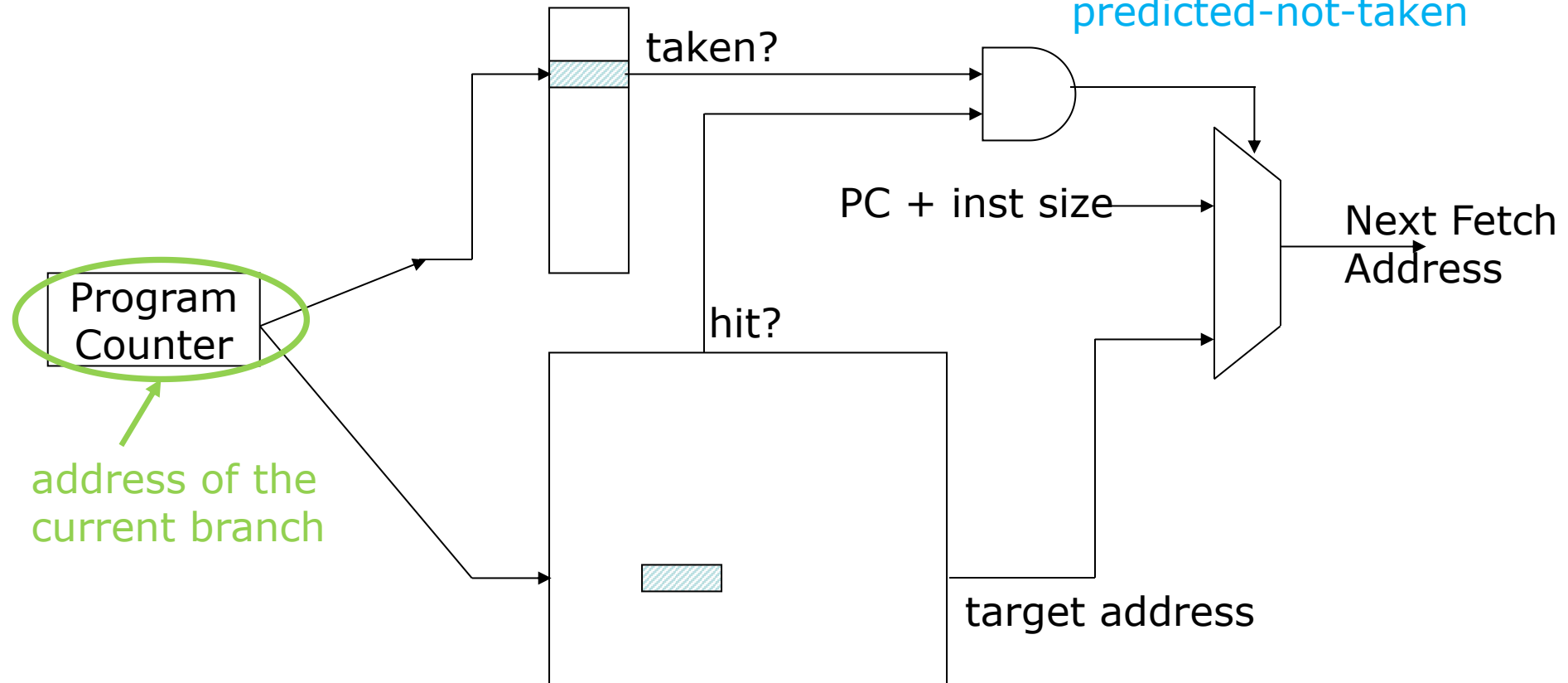
target address

Program Counter

address of the current branch

Cache of Target Addresses (BTB: Branch Target Buffer)

https://safari.ethz.ch/digitaltechnik/spring2021/lib/exe/fetch.php?media=onur-digitaldesign_comparch-2021-lecture17b-branch-prediction-i-afterlecture.pdf



Static Branch Prediction

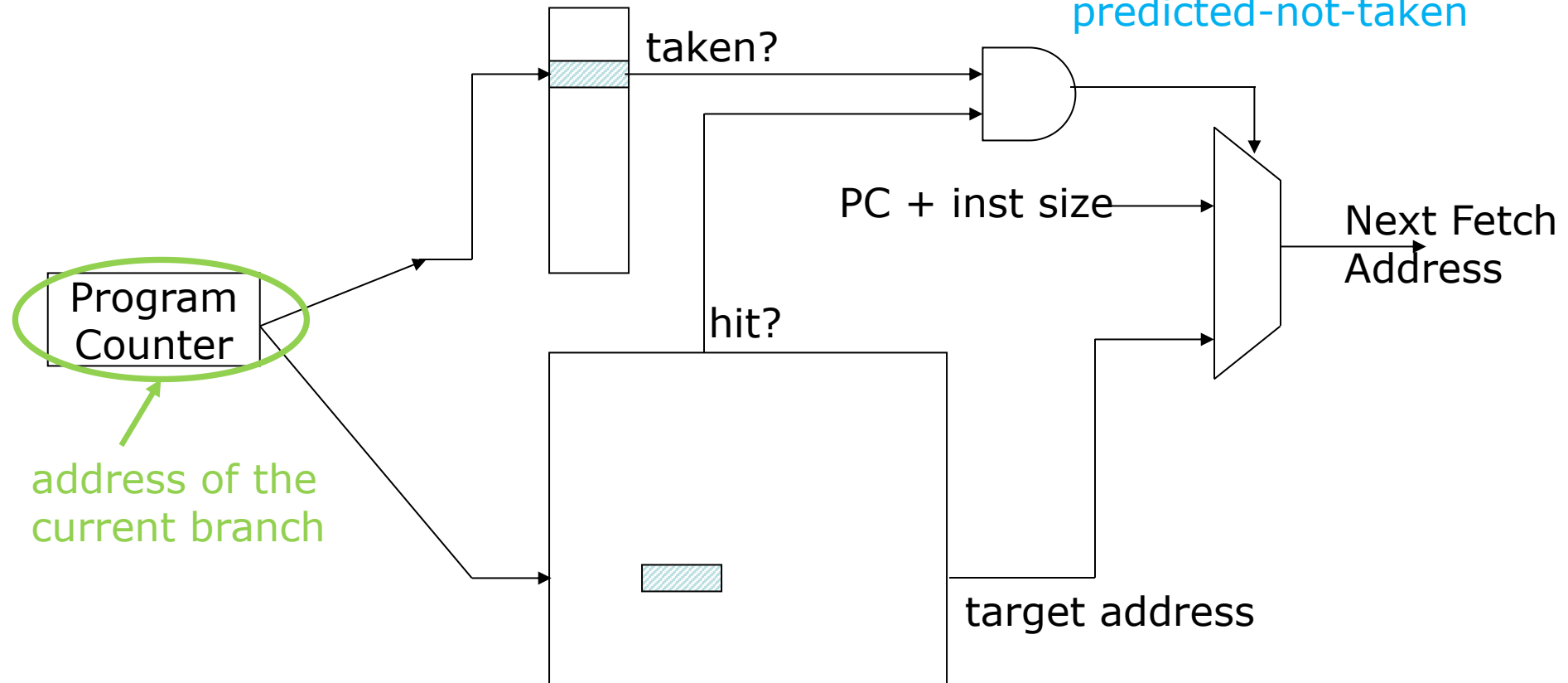
how to handle branches dynamically switching from taken/untaken?

example:

predicted-taken

predicted-not-taken

Direction Predictor (taken?)



Cache of Target Addresses (BTB: Branch Target Buffer)

Dynamic Branch Prediction

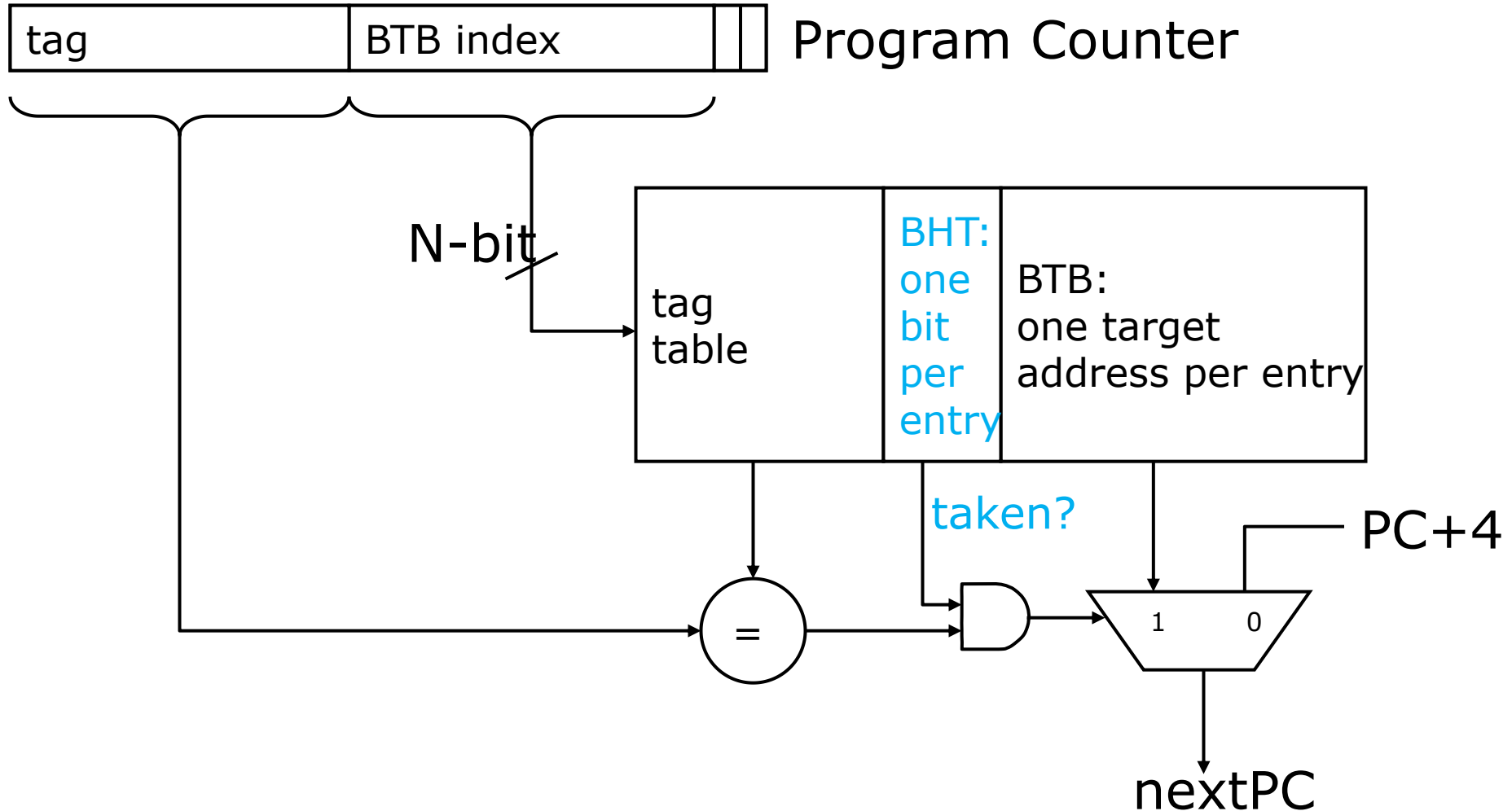
- Follow branch history of taken branch
- Use branch prediction buffer (or BHT: branch history table) indexed by lower portion of branch address and marked with a bit to track taken/untaken status

Dynamic Branch Prediction

- 1-bit predictor / last-time predictor
- Use branch prediction buffer (or BHT: branch history table) indexed by lower portion of branch address and marked with a bit to track taken/untaken status
- bit 1 for taken
- bit 0 for untaken

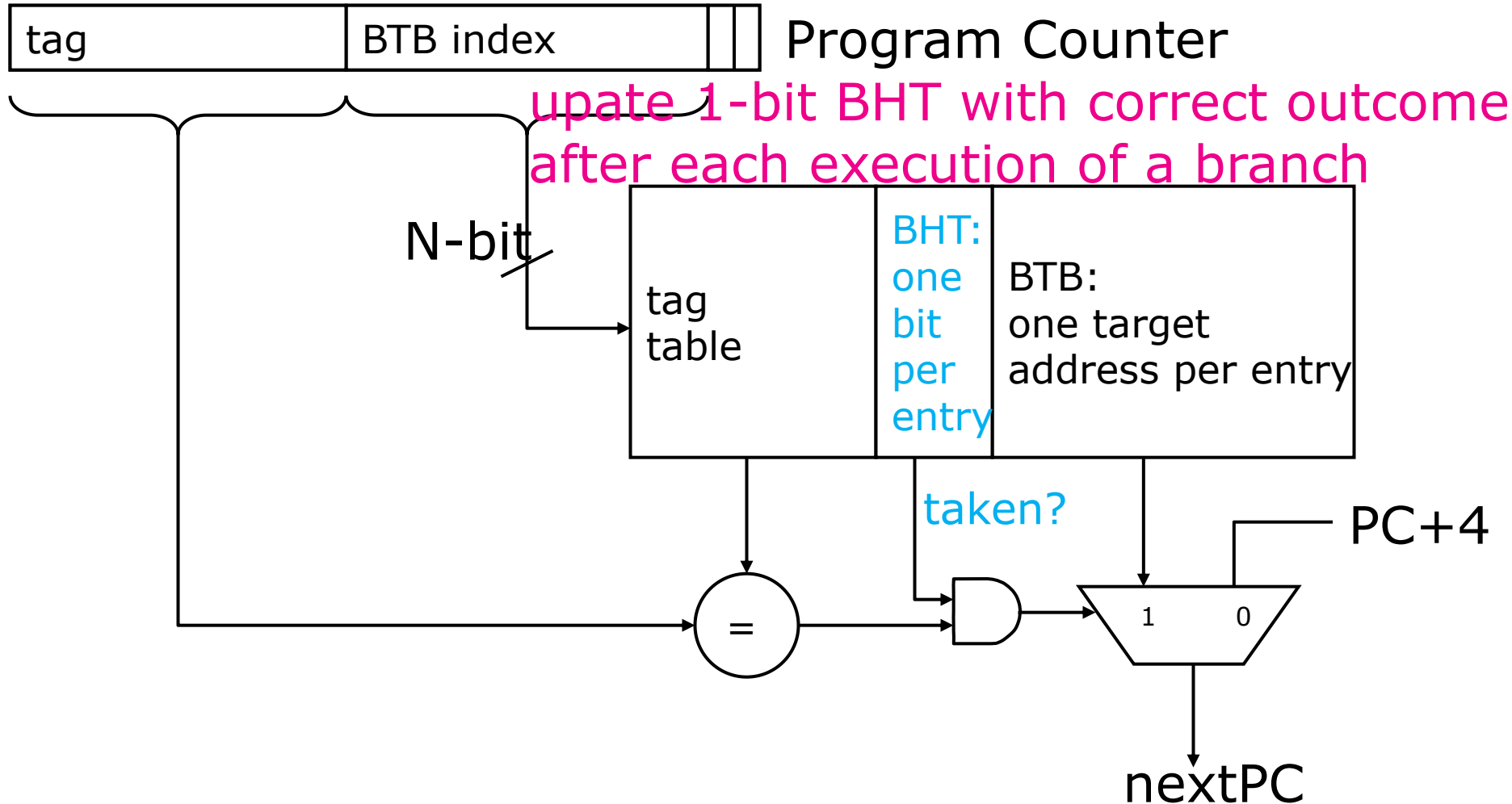
1-bit Predictor

BTB: Branch Target Buffer (Cache of Target Addresses)

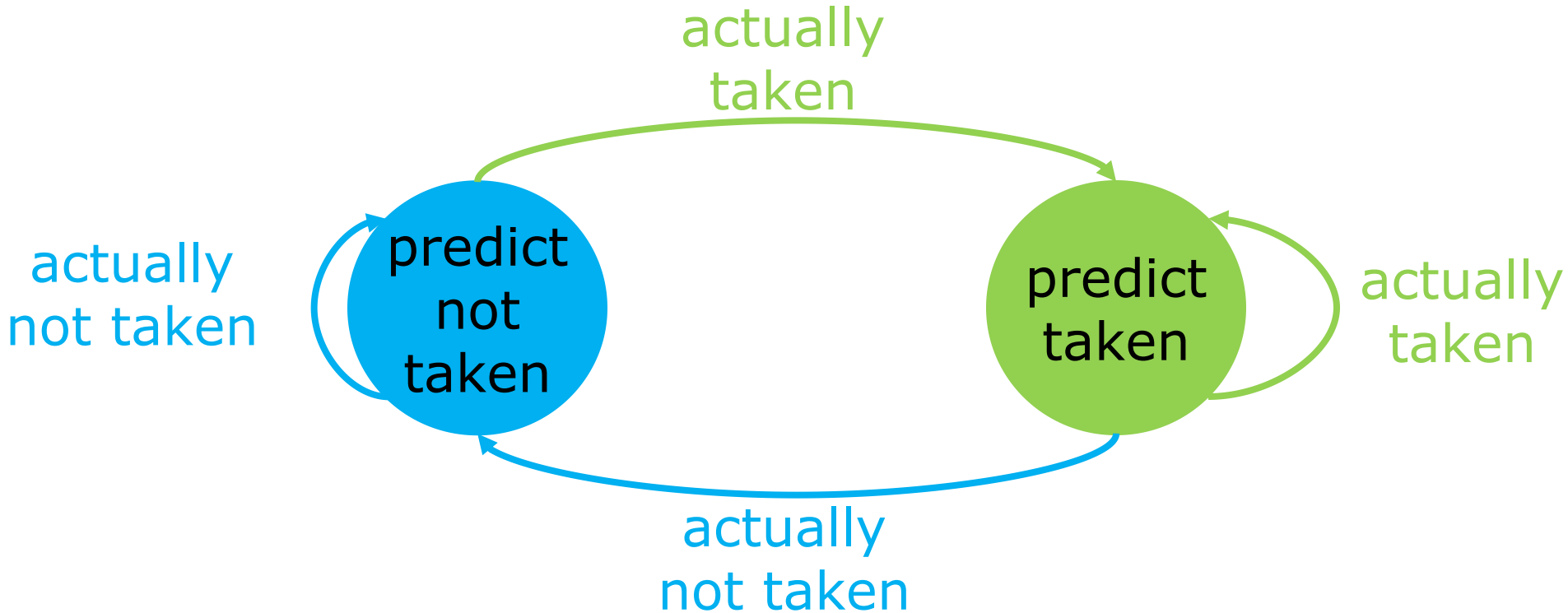


1-bit Predictor

BTB: Branch Target Buffer (Cache of Target Addresses)



1-bit Predictor



finite state machine

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit prediction?

1-bit Predictor

- **Example 1:**

a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;

- prediction accuracy using 1-bit prediction?

[illegible]

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit prediction?

trace	T	T	T	T	T	T	T	T	T	N
prior	0	1								
update	1									
accuracy										

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit prediction?

trace	T	T	T	T	T	T	T	T	T	N
prior	0	1								
update	1	1								
accuracy										

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit prediction?

trace	T	T	T	T	T	T	T	T	T	N
prior	0	1	1							
update	1	1	1							
accuracy										

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit prediction?

trace	T	T	T	T	T	T	T	T	T	N
prior	0	1	1	1	1	1	1	1	1	1
update	1	1	1	1	1	1	1	1	1	
accuracy										

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit prediction?

trace	T	T	T	T	T	T	T	T	T	N
prior	0	1	1	1	1	1	1	1	1	1
update	1	1	1	1	1	1	1	1	1	
accuracy										

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit
prediction: 80%

trace	T	T	T	T	T	T	T	T	T	N
prior	0	1	1	1	1	1	1	1	1	1
update	1	1	1	1	1	1	1	1	1	0
accuracy										

1-bit Predictor

- **Example 1:**
a loop branch that is taken nine times in a row and then is not taken once;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit prediction: 80%

[illegible]

1-bit Predictor

• Example 2:

a loop branch that is taken and then
untaken, repeating 10 iterations;
assume predictor initialized as untaken;

- prediction accuracy using 1-bit prediction?

[illegible]

1-bit Predictor

- **Example 2:**
a loop branch that is taken and then
untaken, repeating 10 iterations;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit
prediction?

trace	T	N	T	N	T	N	T	N	T	N
prior	0	1								
update	1									
accuracy	😬	😬								

1-bit Predictor

- **Example 2:**
a loop branch that is taken and then
untaken, repeating 10 iterations;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit
prediction?

trace	T	N	T	N	T	N	T	N	T	N
prior	0	1								
update	1	0								
accuracy	😬	😬								

1-bit Predictor

- **Example 2:**
a loop branch that is taken and then
untaken, repeating 10 iterations;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit
prediction?

trace	T	N	T	N	T	N	T	N	T	N
prior	0	1	0							
update	1	0								
accuracy	😬	😬	😬							

1-bit Predictor

- **Example 2:**
a loop branch that is taken and then
untaken, repeating 10 iterations;
assume predictor initialized as untaken;
- prediction accuracy using 1-bit
prediction?

trace	T	N	T	N	T	N	T	N	T	N
prior	0	1	0							
update	1	0	1							
accuracy	😬	😬	😬							

1-bit Predictor

- **Example 2:**

a loop branch that is taken and then
untaken, repeating 10 iterations;
assume predictor initialized as untaken;

- prediction accuracy using 1-bit prediction: 0%

trace	T	N	T	N	T	N	T	N	T	N
prior	0	1	0	1	0	1	0	1	0	1
update	1	0	1	0	1	0	1	0	1	0
accuracy										

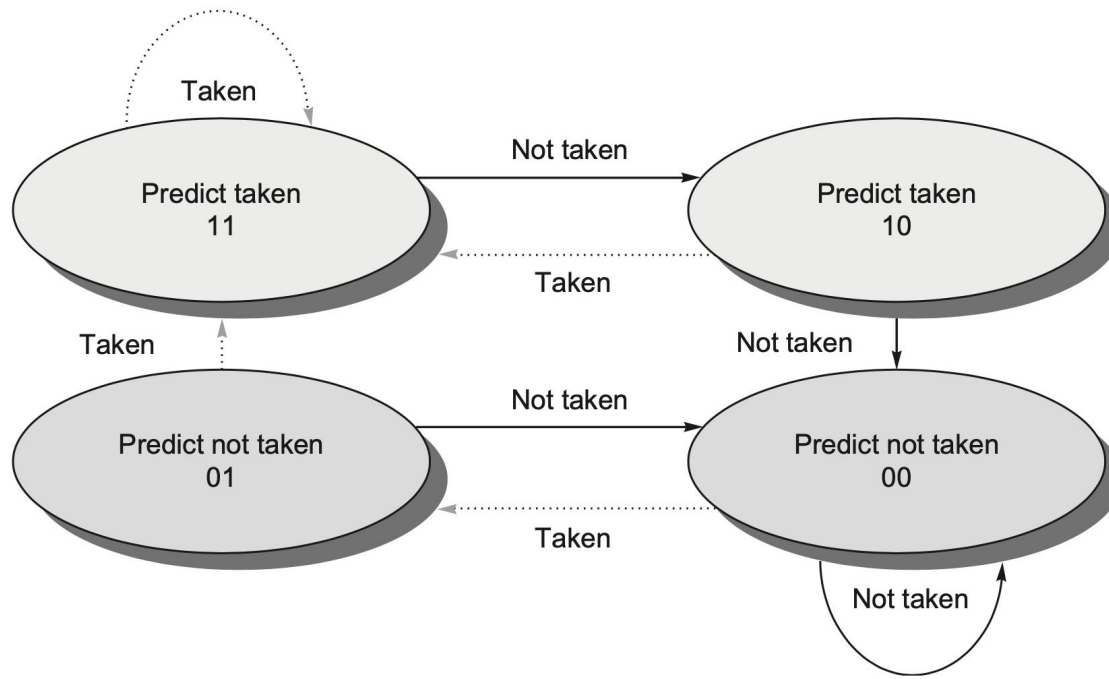
1-bit Predictor

- **Limitation:**
predictor changes too quickly upon each single misprediction
- even though the branch may be mostly taken or mostly not taken

trace	T	N	T	N	T	N	T	N	T	N
prior	0	1	0	1	0	1	0	1	0	1
update	1	0	1	0	1	0	1	0	1	0
accuracy										

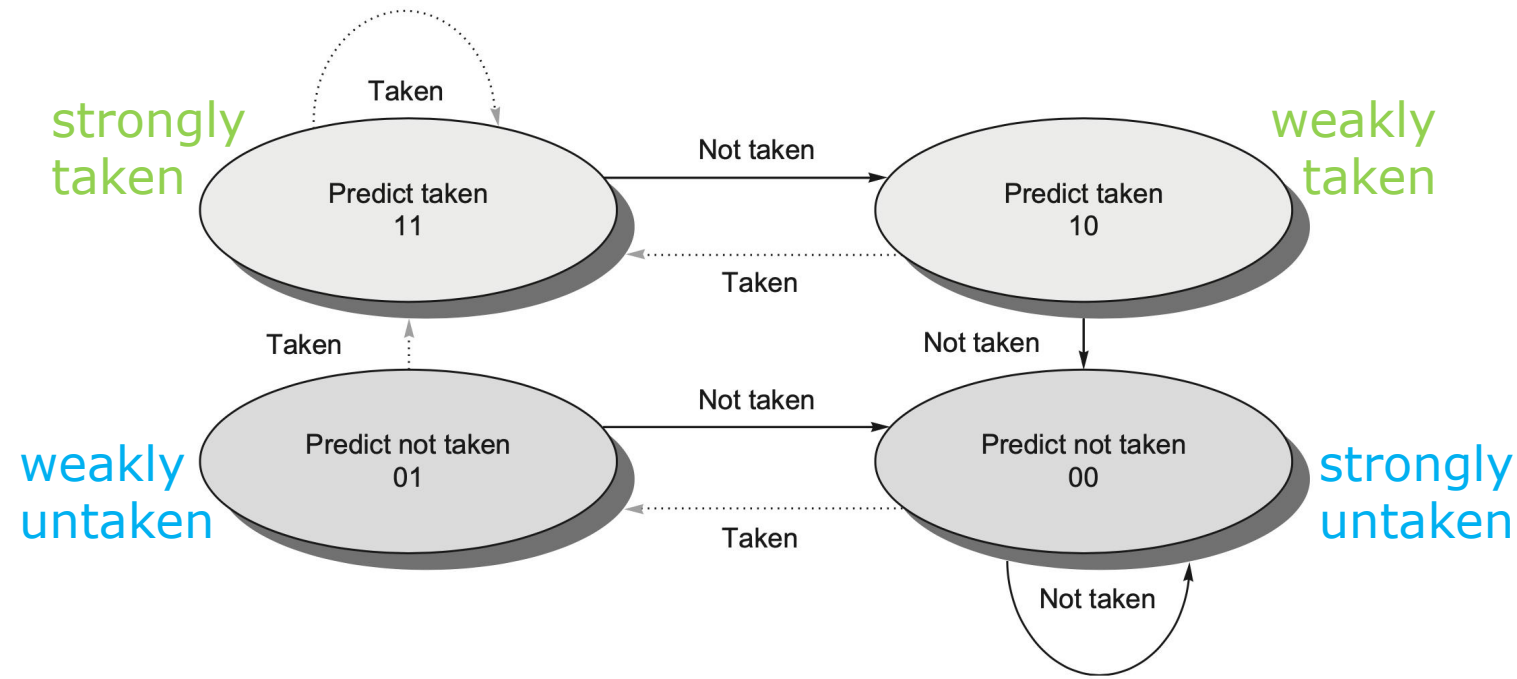
2-bit Predictor

- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice
- finite state machine



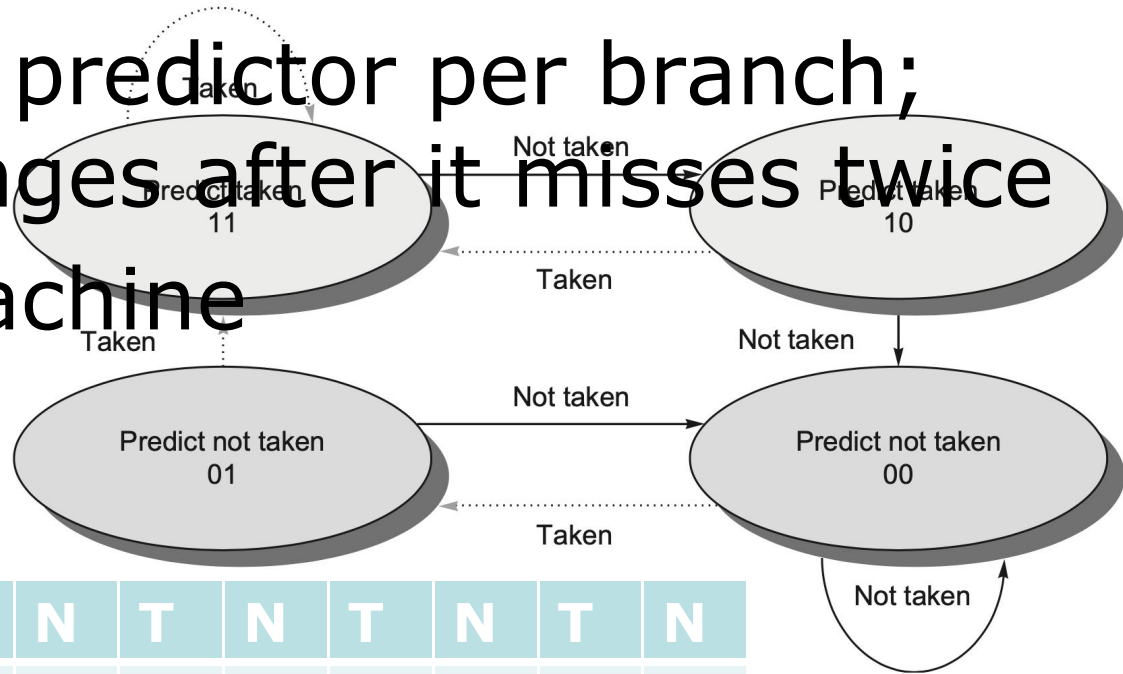
2-bit Predictor

- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice
- finite state machine



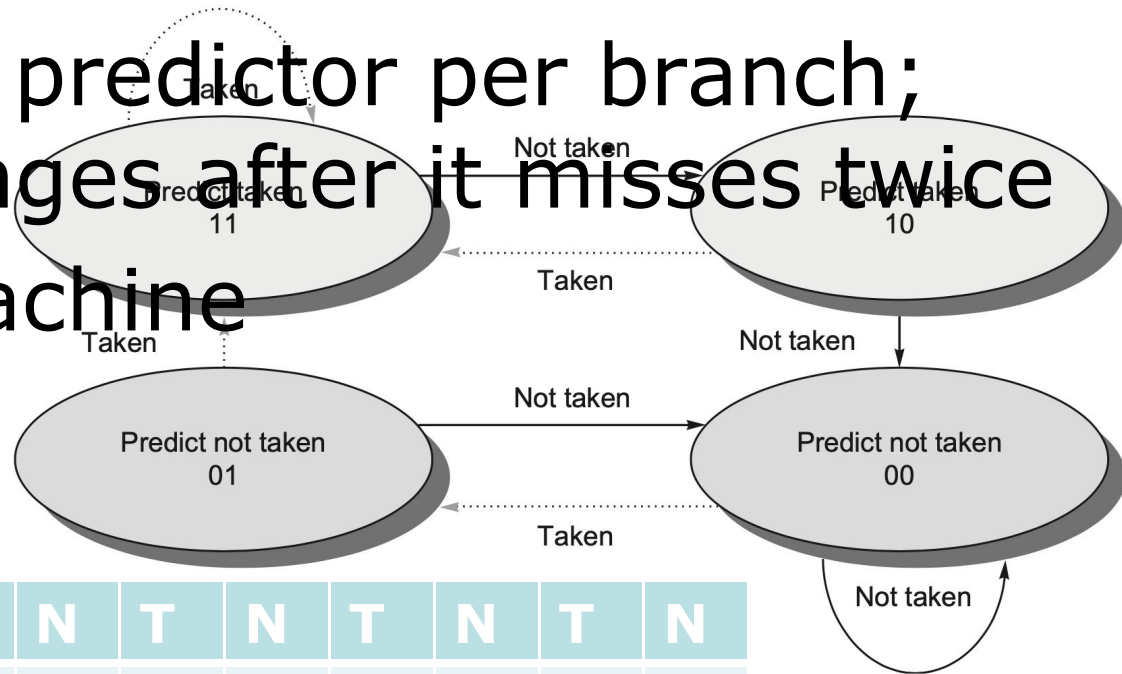
2-bit Predictor

- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice
- finite state machine

[illegible]

2-bit Predictor

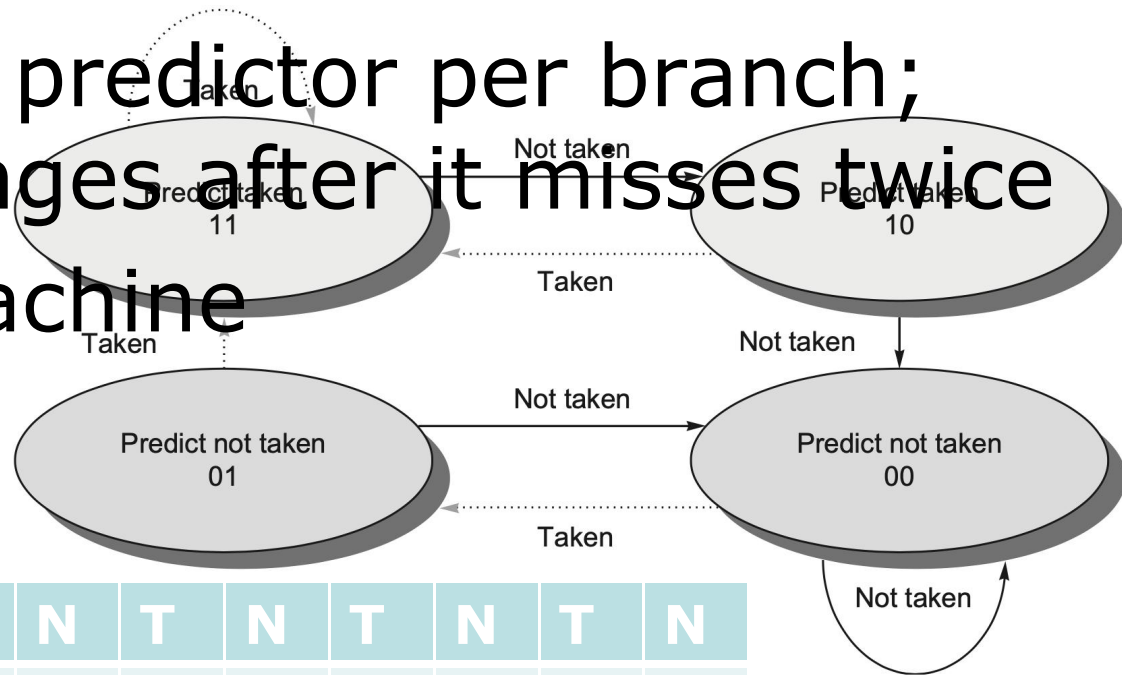
- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice
- finite state machine



trace	T	N	T	N	T	N	T	N	T	N
prior	10	11								
update	11									
accuracy	😊	😬								

2-bit Predictor

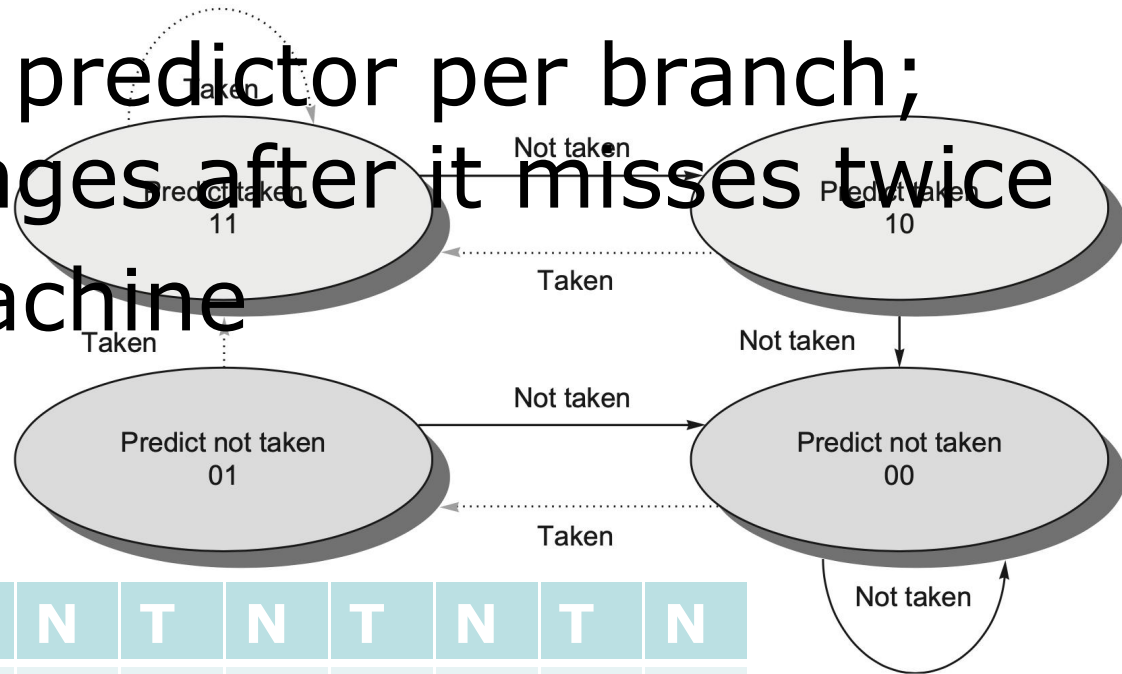
- Idea:
assign a 2-bit predictor per branch;
predictor changes **after it misses twice**
- finite state machine



trace	T	N	T	N	T	N	T	N	T	N
prior	10	11	10							
update	11	10								
accuracy	😊	😬								

2-bit Predictor

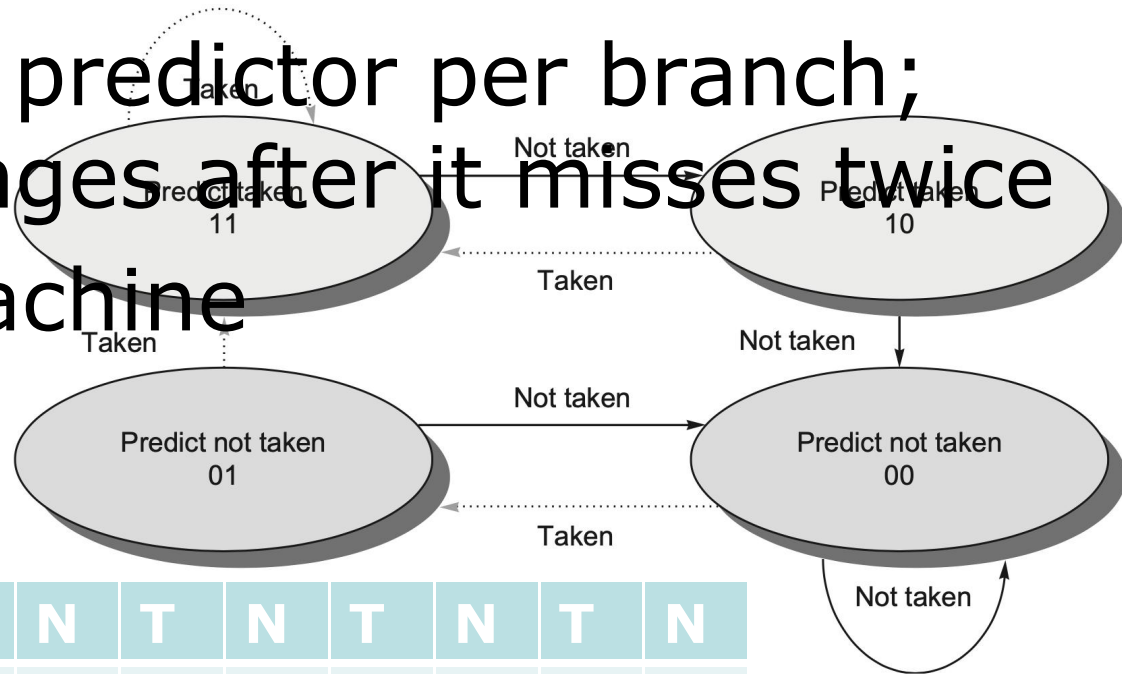
- Idea:
assign a 2-bit predictor per branch;
predictor changes **after it misses twice**
- finite state machine



trace	T	N	T	N	T	N	T	N	T	N
prior	10	11	10							
update	11	10								
accuracy	😊	😬	😊							

2-bit Predictor

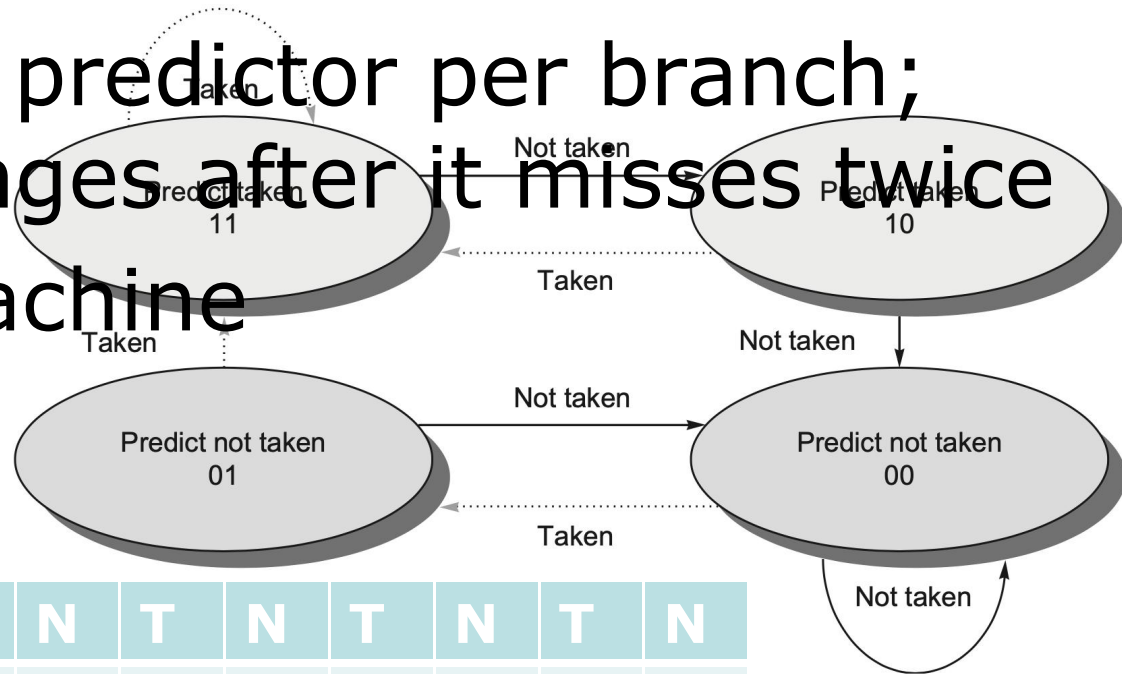
- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice
- finite state machine



trace	T	N	T	N	T	N	T	N	T	N
prior	10	11	10	11						
update	11	10	11							
accuracy	😊	😬	😊							

2-bit Predictor

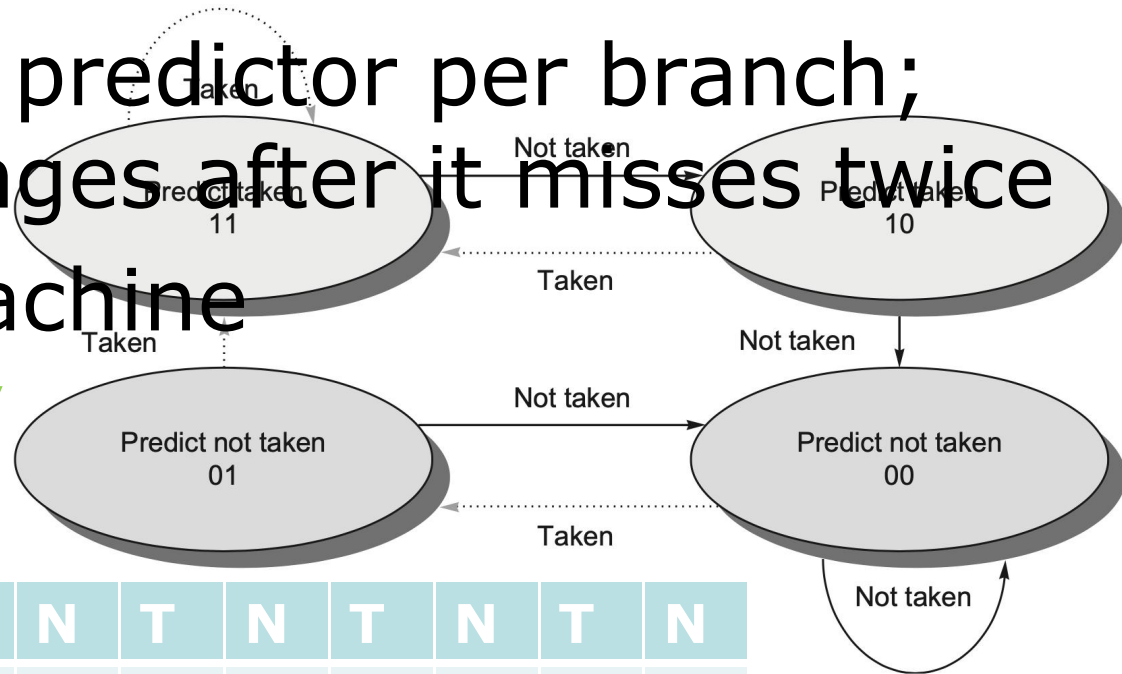
- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice
- finite state machine



trace	T	N	T	N	T	N	T	N	T	N
prior	10	11	10	11	10	11	10	11	10	11
update	11	10	11	10	11	10	11	10	11	10
accuracy	😊	😬	😊	😬	😊	😬	😊	😬	😊	😬

2-bit Predictor

- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice
- finite state machine
50% accuracy



trace	T	N	T	N	T	N	T	N	T	N
prior	10	11	10	11	10	11	10	11	10	11
update	11	10	11	10	11	10	11	10	11	10
accuracy	😊	😬	😊	😬	😊	😬	😊	😬	😊	😬

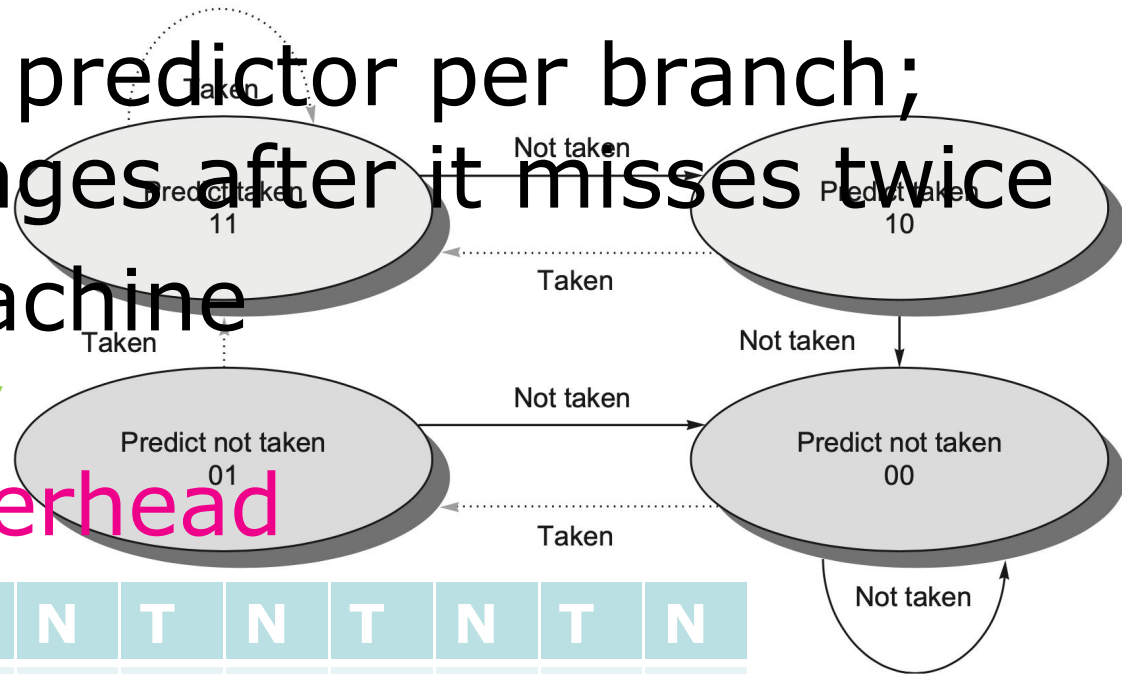
2-bit Predictor

- Idea:
assign a 2-bit predictor per branch;
predictor changes after it misses twice

- finite state machine

50% accuracy

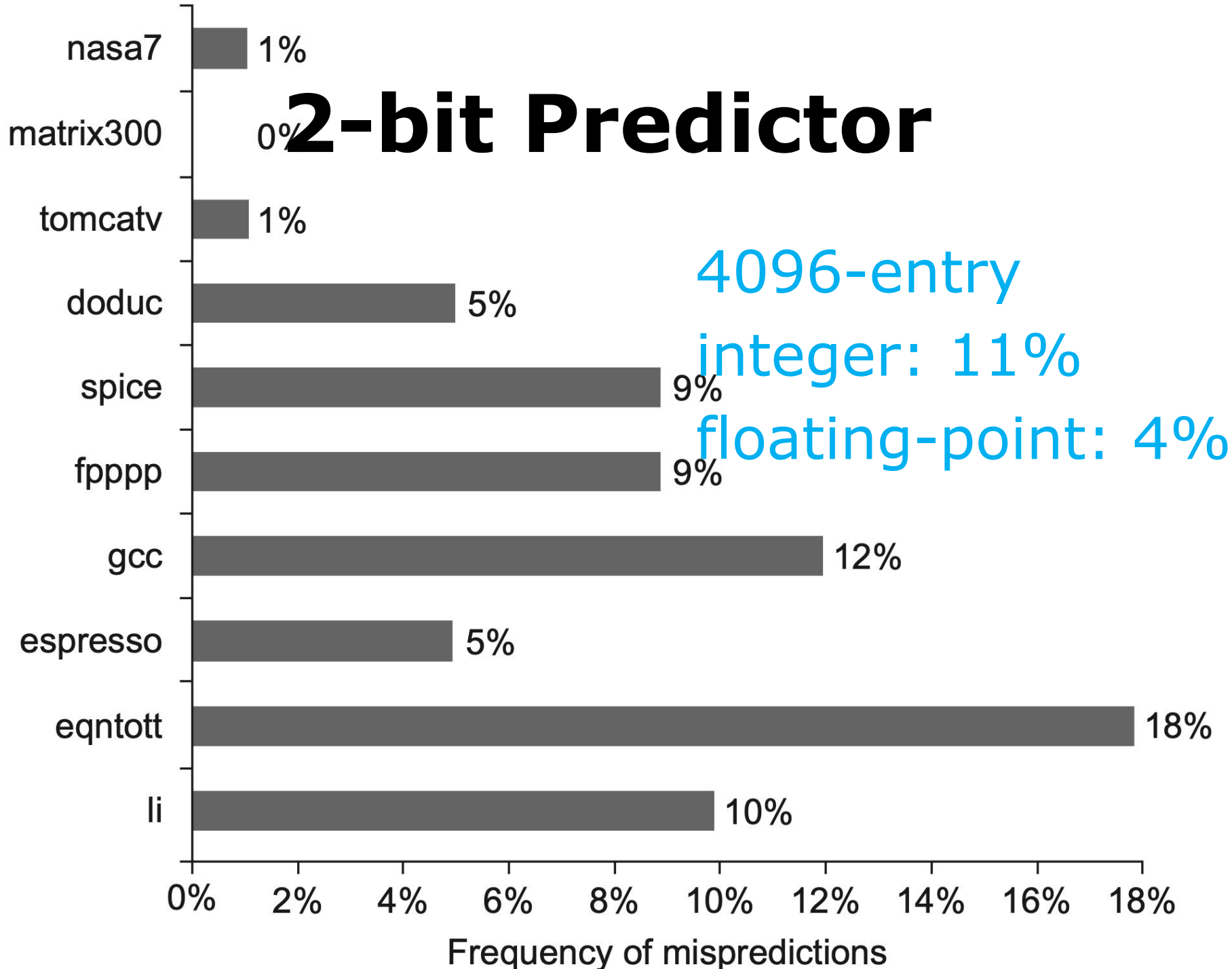
2x storage overhead



trace	T	N	T	N	T	N	T	N	T	N
prior	10	11	10	11	10	11	10	11	10	11
update	11	10	11	10	11	10	11	10	11	10
accuracy	😊	😬	😊	😬	😊	😬	😊	😬	😊	😬

2-bit Predictor

SPEC89 benchmarks



N-bit Predictor

- Idea:
assign an N-bit predictor per branch;
range from 0 to $2^N - 1$;

N-bit Predictor

- Idea:
assign an N-bit predictor per branch;
range from 0 to $2^N - 1$;
- Counter threshold:
 $(2^N - 1) / 2$
- greater/ $\geq$ counter predicts taken branch
- smaller counter predicts untaken branch

only the most recent so far

generalized to local history

Local Predictor

- A branch's outcome can be correlated with past outcomes of the same branch (other than the outcome of the branch "last-time" it was executed)
- Example
for (i=1, i<4, i++) { }

Local Predictor

- A branch's outcome can be correlated with past outcomes of the same branch (other than the outcome of the branch "last-time" it was executed)
- Example
for (i=1, i<4, i++) { }
3 consecutive taken + 1 untaken: 1110

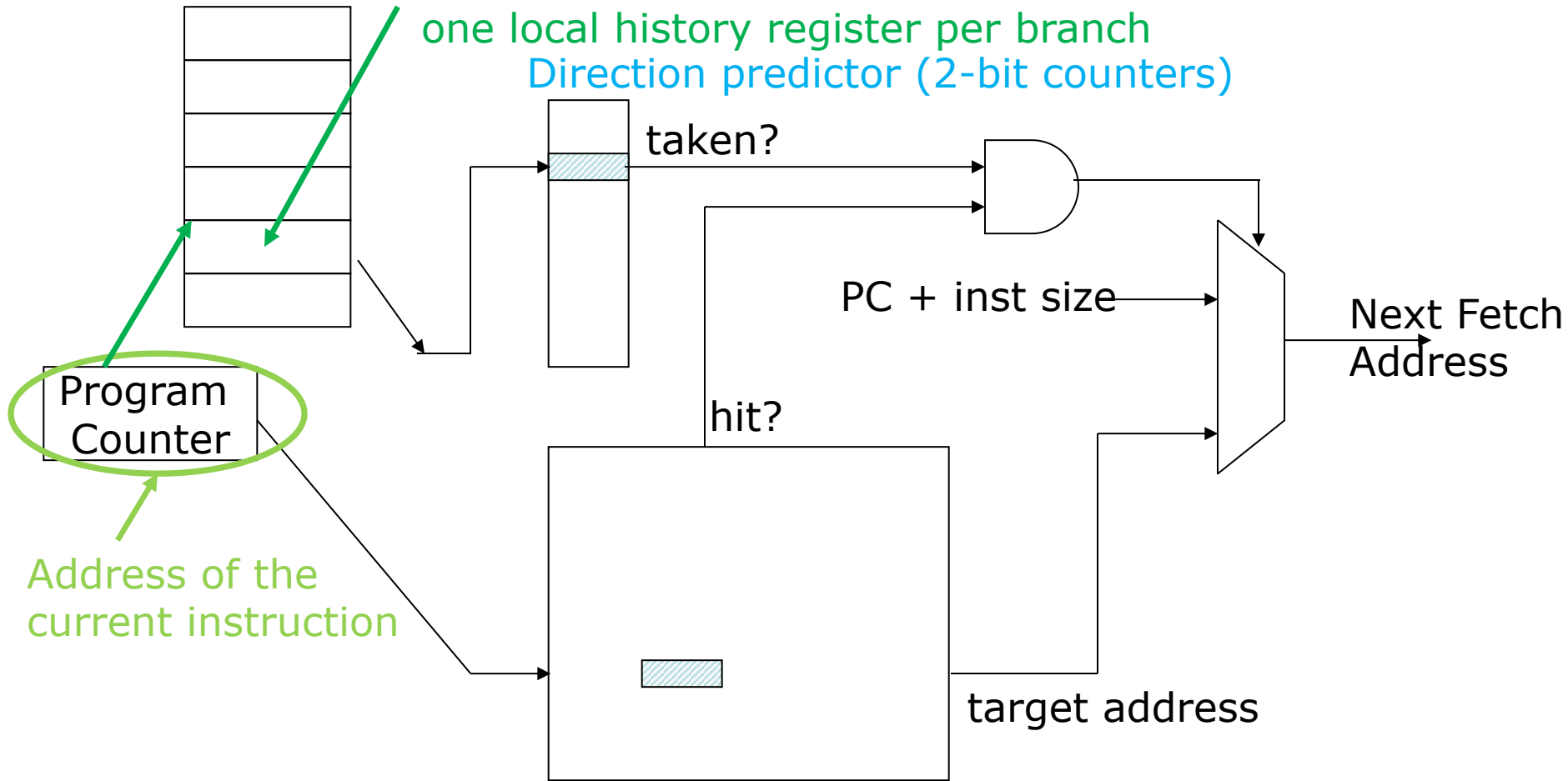
Local Predictor

- A branch's outcome can be correlated with past outcomes of the same branch (other than the outcome of the branch "last-time" it was executed)
- Example
for (i=1, i<4, i++) { }
3 consecutive taken + 1 untaken: 1110
3-bit local history register: 111→!taken

Local Predictor

which directions earlier instances of *this branch* went
one local history register per branch

Direction predictor (2-bit counters)



Cache of Target Addresses (BTB: Branch Target Buffer)

https://safari.ethz.ch/digitaltechnik/spring2021/lib/exe/fetch.php?media=onur-digitaldesign_comparch-2021-lecture17b-branch-prediction-i-afterlecture.pdf

Local Predictor

- **Example:**
 - 1-bit history initialized as 0;
 - 2-bit counter/predictor per history initialized as SN;
 - given a $(NNT)^*$ loop, how many mispredictions after 100 iterations?
- # of mispredictions: ?

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a $(NNT)^*$ loop, how many
misprecisions after 100 iterations?

- # of mispredictions: ?

[illegible]

Local Predictor

- **Example:**

1-bit history initialized as 0;

2-bit counter/predictor per history
initialized as SN;

given a (NNT)* loop, how many misprecisions after 100 iterations?

- # of mispredictions: ?

[illegible]

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN								
pred	N									
accuracy	😊					😊				

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN							
pred	N	N								
accuracy	😊	😊								

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN							
pred	N	N	N							
accuracy	😊	😊								

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN						
pred	N	N	N							
accuracy	😊	😊	😬							

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN						
pred	N	N	N	N						
accuracy	😊	😊	😬							

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN					
pred	N	N	N	N						
accuracy	😊	😊	😬	😊						

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN					
pred	N	N	N	N	N					
accuracy	😊	😊	😬	😊						

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN	0, SN,SN				
pred	N	N	N	N	N					
accuracy	😊	😊	😬	😊	😊					

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN	0, SN,SN				
pred	N	N	N	N	N	N				
accuracy	😊	😊	😬	😊	😊					

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN	0, SN,SN	1, WN,SN			
pred	N	N	N	N	N	N				
accuracy	😊	😊	😬	😊	😊	😬				

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: ?

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN	0, SN,SN	1, WN,SN			
pred	N	N	N	N	N	N				
accuracy	😊	😊	😬	😊	😊	😬				

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: 1 per iteration

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN	0, SN,SN	1, WN,SN			
pred	N	N	N	N	N	N				
accuracy	😊	😊	😬	😊	😊	😬				

Local Predictor

- **Example:**

1-bit history initialized as 0;
2-bit counter/predictor per history
initialized as SN;
given a (NNT)* loop, how many
misprecitions after 100 iterations?

- # of mispredictions: 100

trace	N	N	T	N	N	T	N	N	T	N
state	0, SN,SN	0, SN,SN	0, SN,SN	1, WN,SN	0, WN,SN	0, SN,SN	1, WN,SN			
pred	N	N	N	N	N	N				
accuracy	😊	😊	😬	😊	😊	😬				

only a single branch so far

An aerial view of the Beijing 2008 Olympic Opening Ceremony in the Bird's Nest stadium. The stadium is filled with a large crowd of people, many of whom are holding flags. In the background, the Olympic cauldron is visible, and the Olympic rings logo is displayed on the right side of the image. The text "no man is an island" is overlaid in the center-left.

no man is an island

Faster
Higher
Stronger
Together



Correlate Branch Prediction

- Correlating/Two-level Predictor

use the behavior of other branches to make a prediction;

make prediction based on the outcome of the branch the last time the same global branch history was encountered

Correlate Branch Prediction

- Example: eqntott benchmark

```
if (aa==2)
```

```
    aa=0;
```

```
if (bb==2)
```

```
    bb=0;
```

```
if (aa!=bb) {...
```

Correlate Branch Prediction

- Example: eqntott benchmark

```
if (aa==2)
    aa=0;
if (bb==2)
    bb=0;
if (aa!=bb) {...
```

x1 for aa

x2 for bb

```
addi    x3, x1, -2
bnez     x3, L1
add      x1, x0, x0
L1:      addi    x3, x2, -2
bnez     x3, L2
add      x2, x0, x0
L2:      sub     x3, x1, x2
beqz     x3, L3
```

Correlate Branch Prediction

- Example: eqntott benchmark

```
if (aa==2)
    aa=0;
if (bb==2)
    bb=0;
if (aa!=bb) {...
```

```
        addi    x3, x1, -2
b1 bnez    x3, L1
        add     x1, x0, x0
L1:      addi    x3, x2, -2
b2 bnez    x3, L2
        add     x2, x0, x0
L2:      sub     x3, x1, x2
b3 beqz    x3, L3
```

Correlate Branch Prediction

- Example: eqntott benchmark

```
if (aa==2)
    aa=0;
if (bb==2)
    bb=0;
if (aa!=bb) {...
```

if b1&b2 taken

```
        addi    x3, x1, -2
b1 bnez    x3, L1 😊
        add     x1, x0, x0
L1:      addi    x3, x2, -2
b2 bnez    x3, L2 😊
        add     x2, x0, x0
L2:      sub     x3, x1, x2
b3 beqz    x3, L3
```


Correlate Branch Prediction

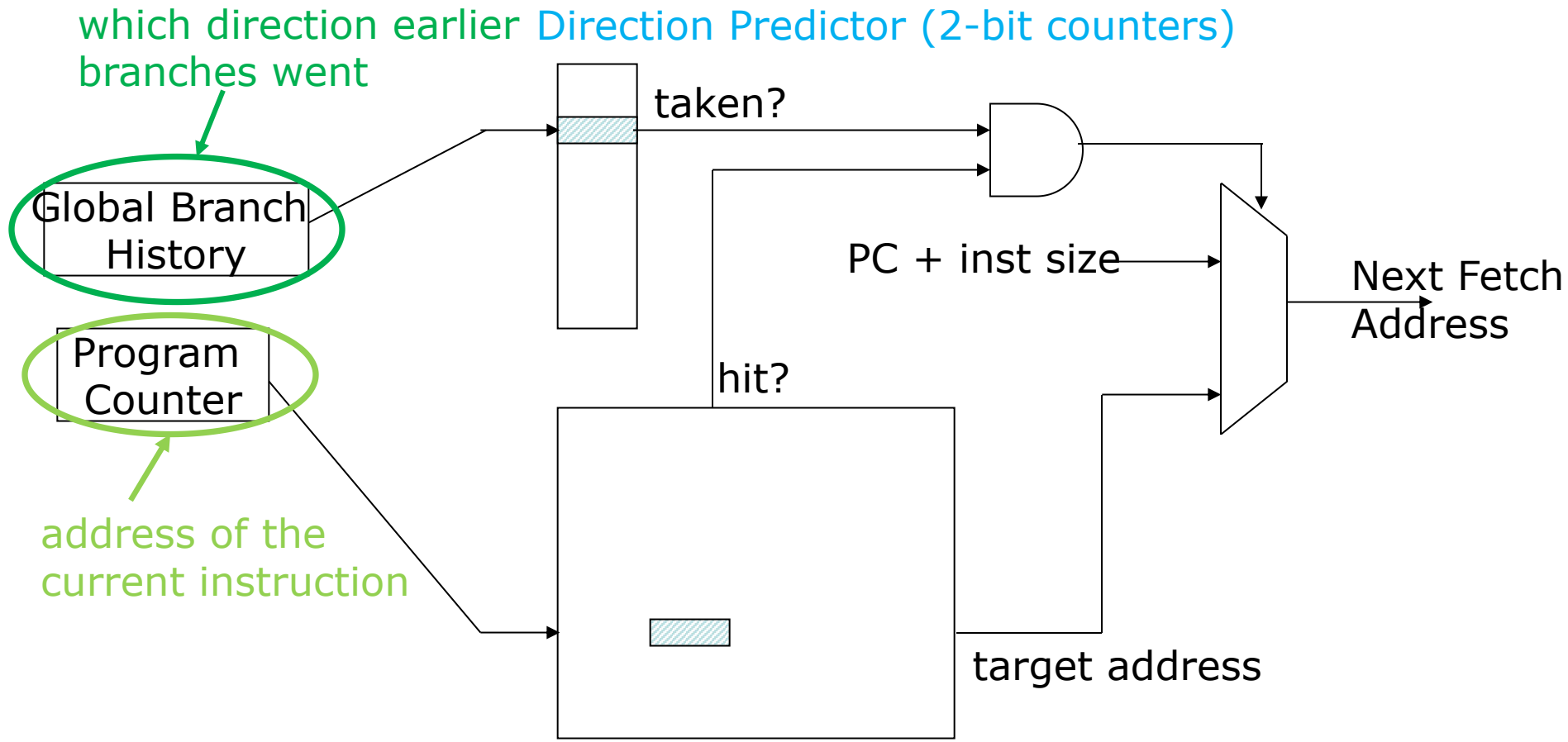
- Example: eqntott benchmark

```
if (aa==2)
    aa=0;
if (bb==2)
    bb=0;
if (aa!=bb) {...
```

if b1&b2 taken
then b3 untaken

```
        addi    x3, x1, -2
b1 bnez    x3, L1 😊
        add     x1, x0, x0
L1:      addi    x3, x2, -2
b2 bnez    x3, L2 😊
        add     x2, x0, x0
L2:      sub     x3, x1, x2
b3 beqz    x3, L3 😬
```

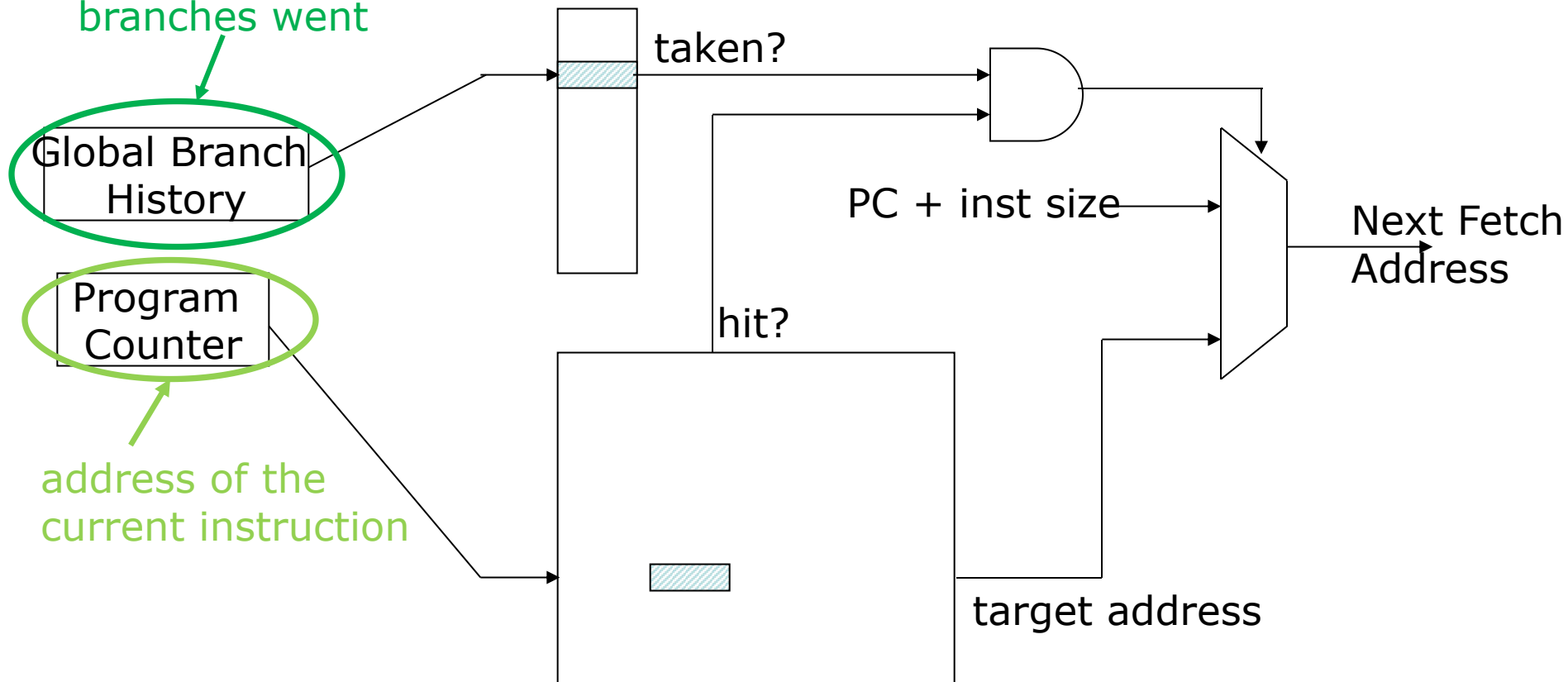
Two-Level Predictor



Cache of Target Addresses (BTB: Branch Target Buffer)

Two-Level Predictor

use a left-shift register to track global branch history
which direction earlier branches went
Direction Predictor (2-bit counters)



Cache of Target Addresses (BTB: Branch Target Buffer)

Two-Level Predictor

- Example: eqntott benchmark

predict b3 using
b1 & b2

state	global	predictor
prior	00	?...
b1 taken	01	?...
b2 taken	11	untaken

if b1&b2 taken
then b3 untaken

```
        addi    x3, x1, -2
b1 bnez    x3, L1 😊
        add     x1, x0, x0
L1:      addi    x3, x2, -2
b2 bnez    x3, L2 😊
        add     x2, x0, x0
L2:      sub     x3, x1, x2
b3 beqz    x3, L3 😬
```

Two-Level Predictor

- (m, n) predictor
use the behavior of the last m branches to choose from 2^m branch predictors, each predictor is an n -bit counter
- storage cost per branch given an individual pattern history table (PHT)
 $2^m \times n$ bits

Two-Level Predictor

- (m, n) predictor

use the behavior of the last m branches to choose from 2^m branch predictors, each predictor is an n -bit counter

- storage cost per branch given an individual pattern history table (PHT)

$2^m \times n$ bits

- lower b -bit of branch address for indices

$2^m \times n \times 2^b$ bits

Two-Level Predictor

- (m, n) predictor
use the behavior of the last m branches
to choose from 2^m branch predictors,
each predictor is an n -bit counter
- storage cost of all branch predictors
given a unified PHT
 $2^m \times n$ bits

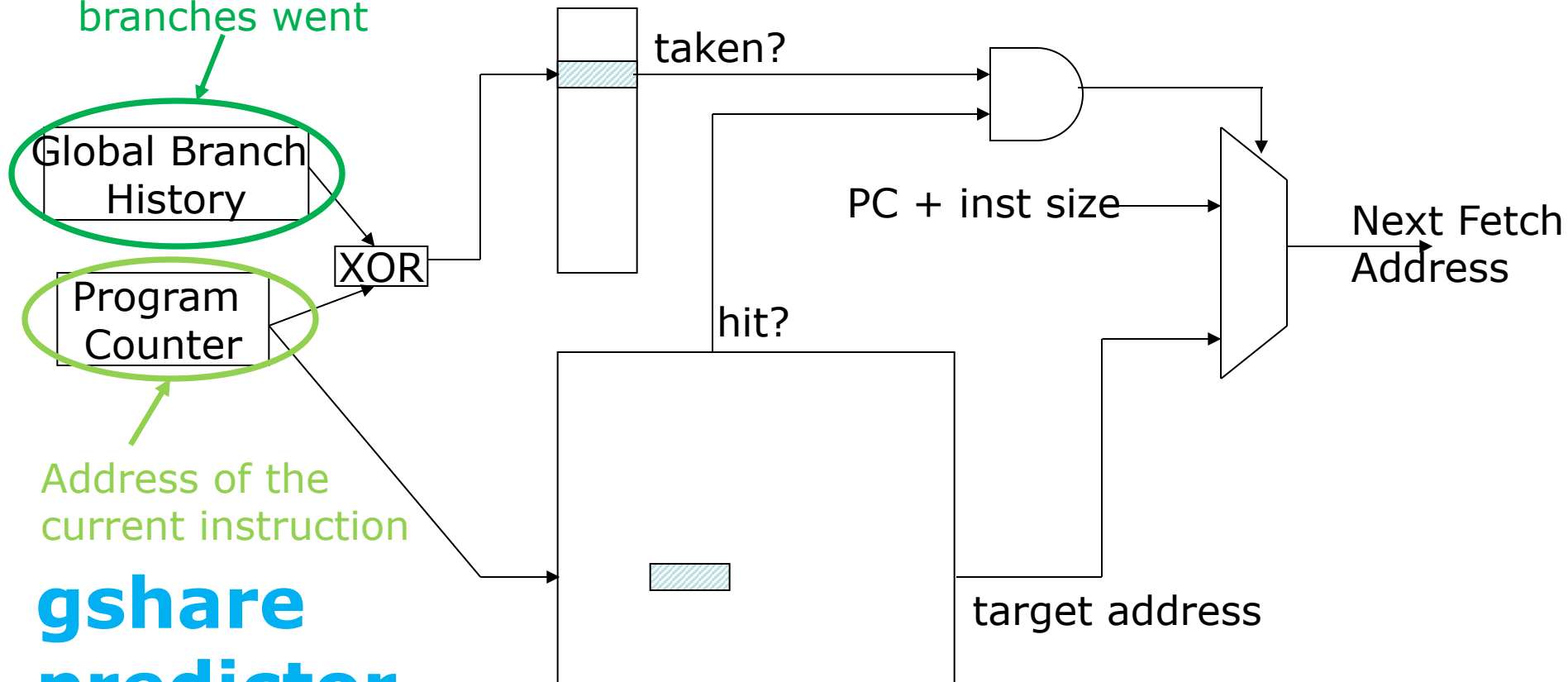
Two-Level Predictor

- (m, n) predictor
 - use the behavior of the last m branches to choose from 2^m branch predictors, each predictor is an n-bit counter
- storage cost of all branch predictors given a unified PHT
 - $2^m \times n$ bits
 - predictor becomes less branch specific

Hybrid/Alloyed Predictor

combine local branch info and global branch history

which direction earlier Direction predictor (2-bit counters)
branches went

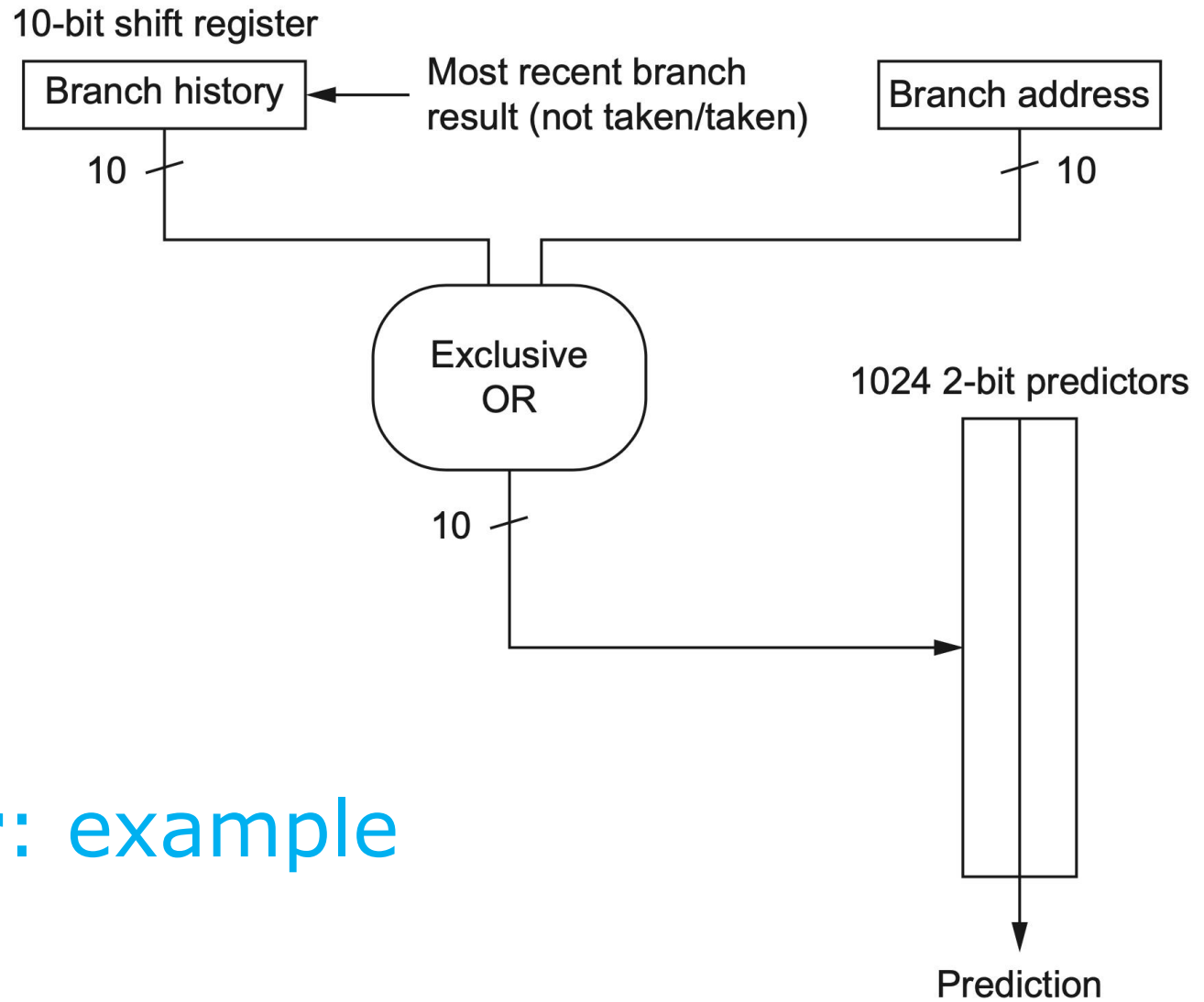


gshare
predictor

Cache of Target Addresses (BTB: Branch Target Buffer)

Hybrid/Alloyed Predictor

combine local branch info and global branch history



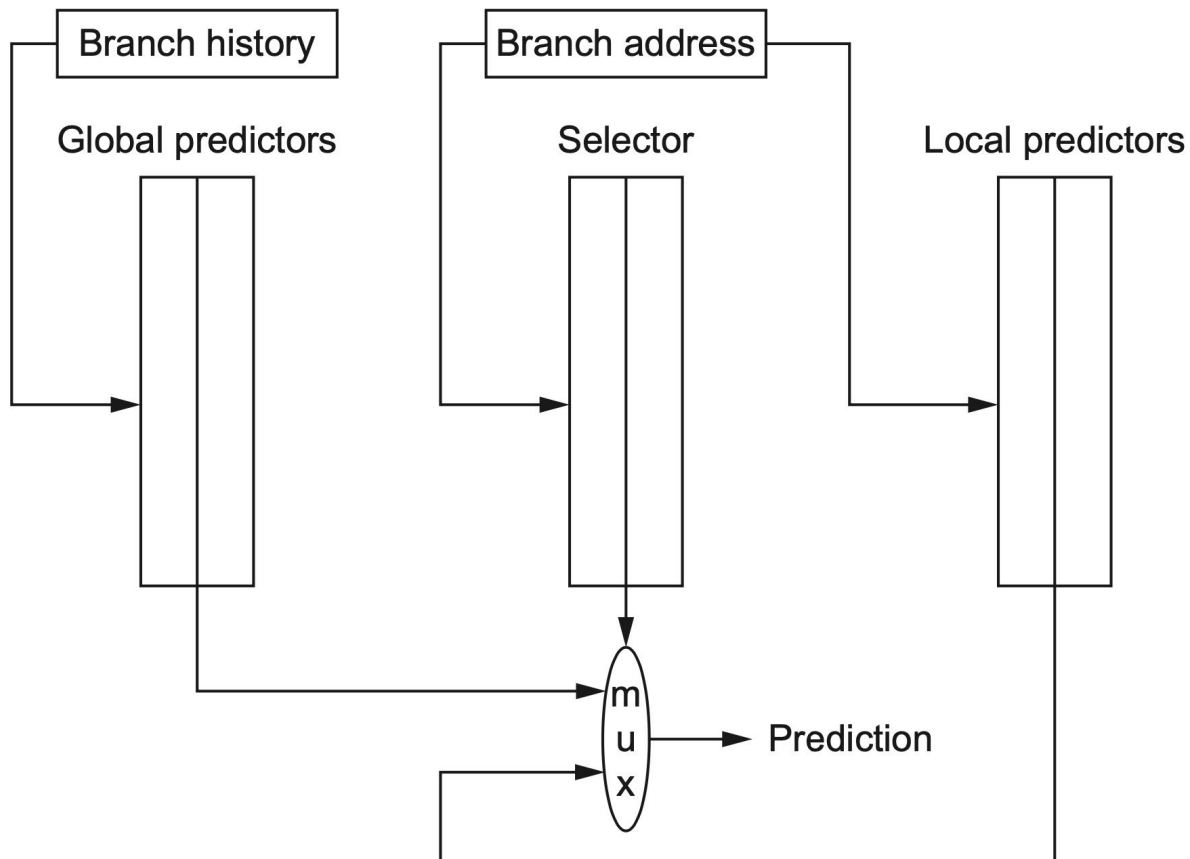
gshare
predictor: example

only a single predictor so far

no predictor fits all branches

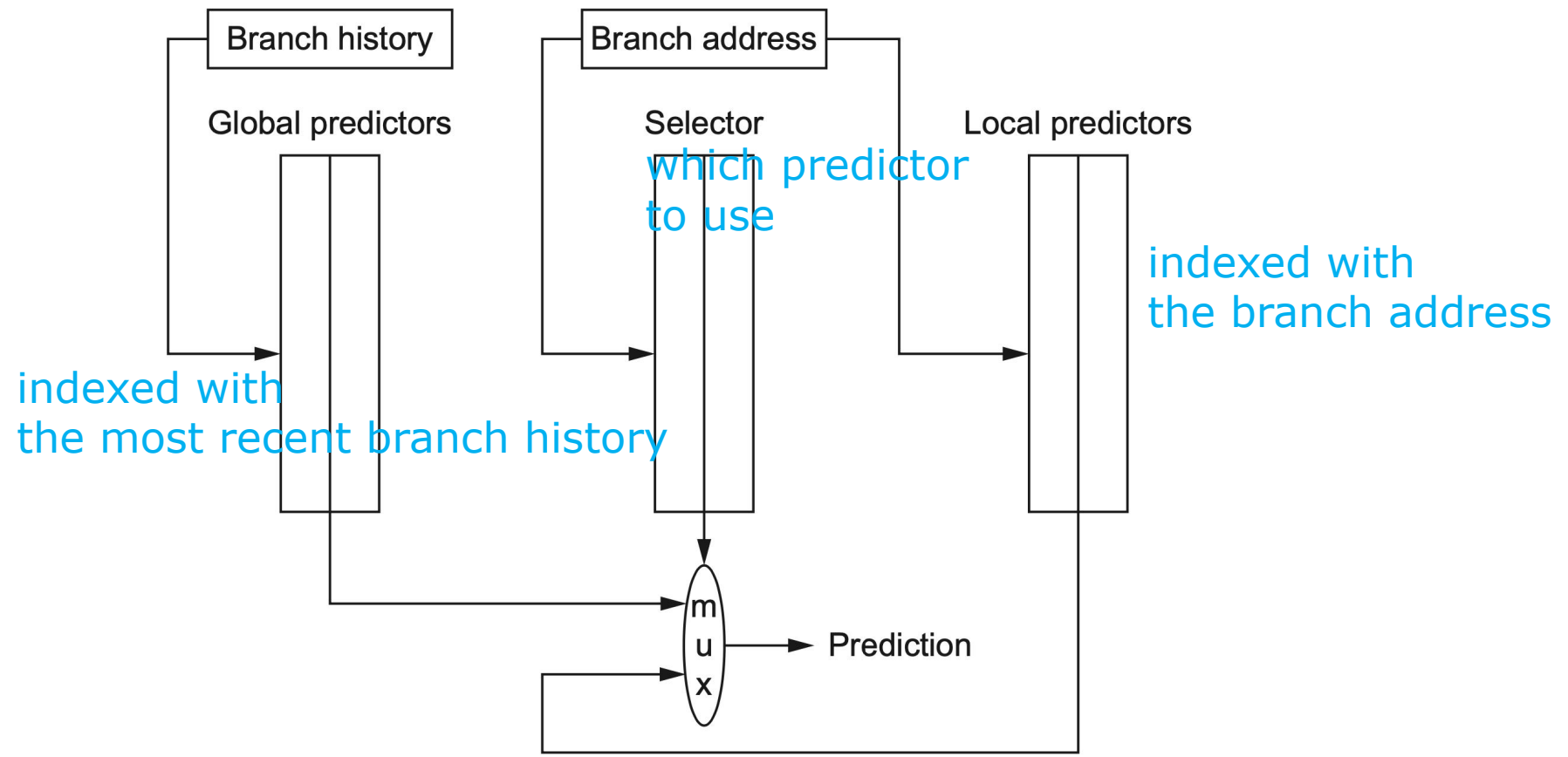
Hybrid Predictor

- Use more than one type of predictor
- Select the best prediction



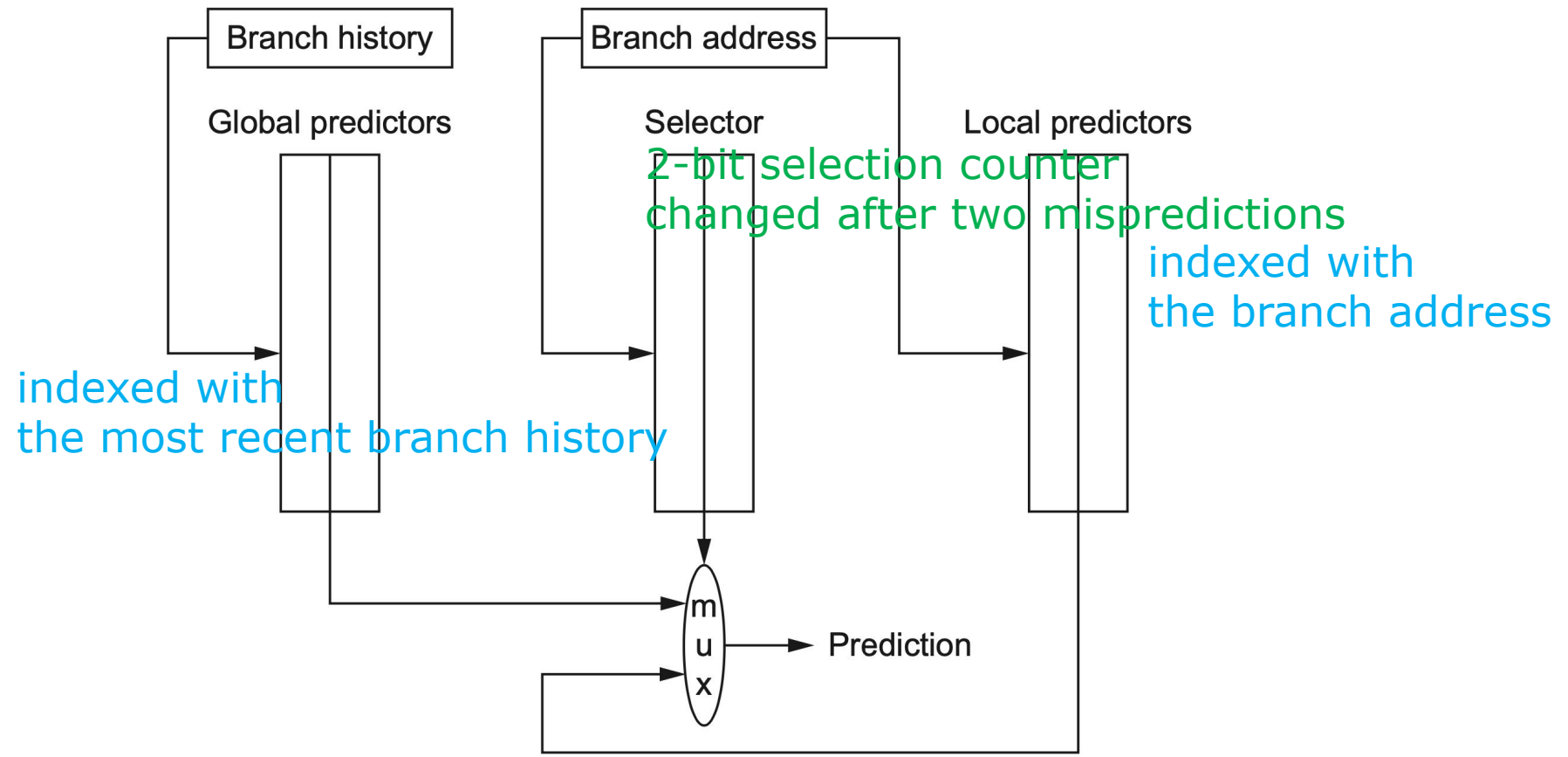
Tournament Predictor

- Global predictor + Local predictor
- Choose either with a selector



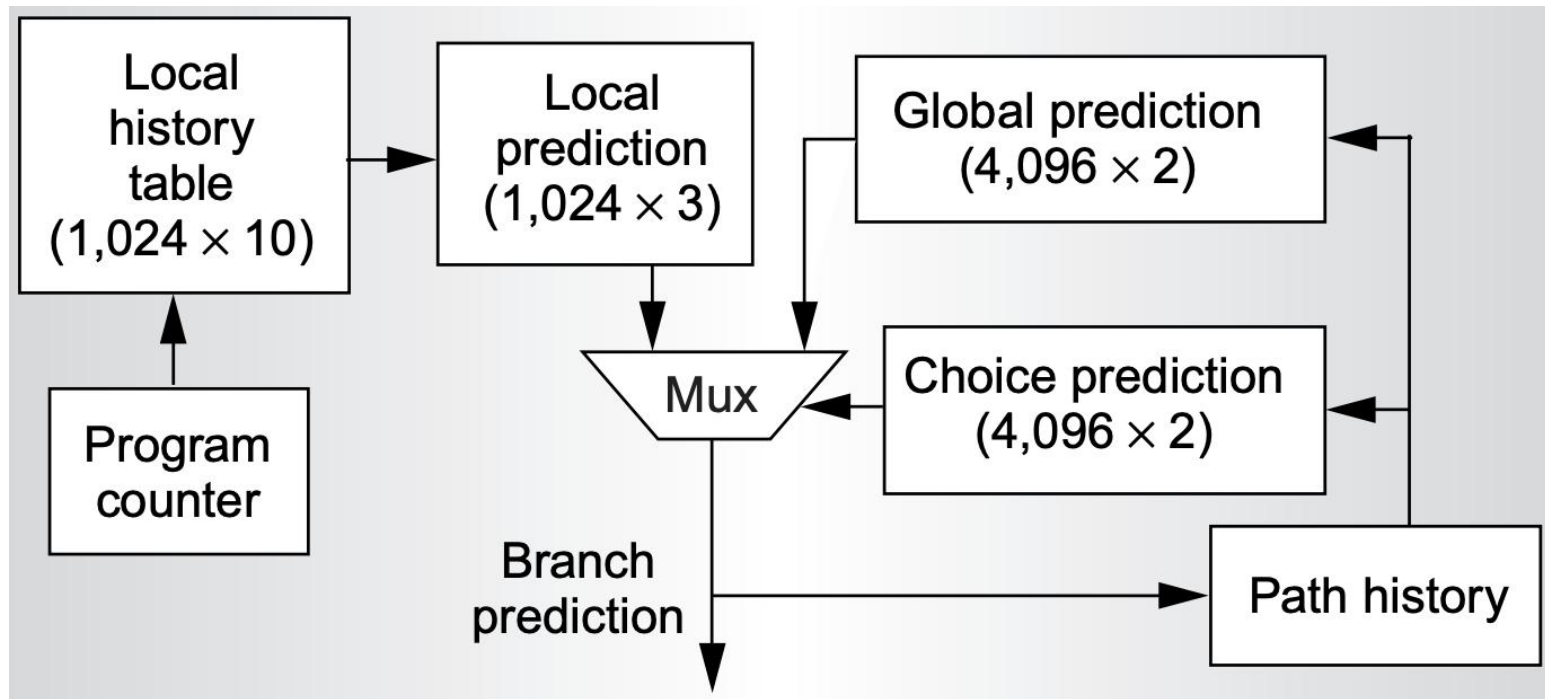
Tournament Predictor

- Global predictor + Local predictor
- Choose either with a selector



Tournament Predictor

- Global predictor + Local predictor
- Choose either with a selector



Alpha 21264 Tournament Predictor

Tagged Hybrid Predictor

- Use a series of global predictors indexed with different length histories
- Use tags in global history tables
- Choose a prediction from a global history table only if its tag matches the hash of branch address and global branch history
- Select the predictor with the longest history where the tags also match

Tagged Hybrid Predictor

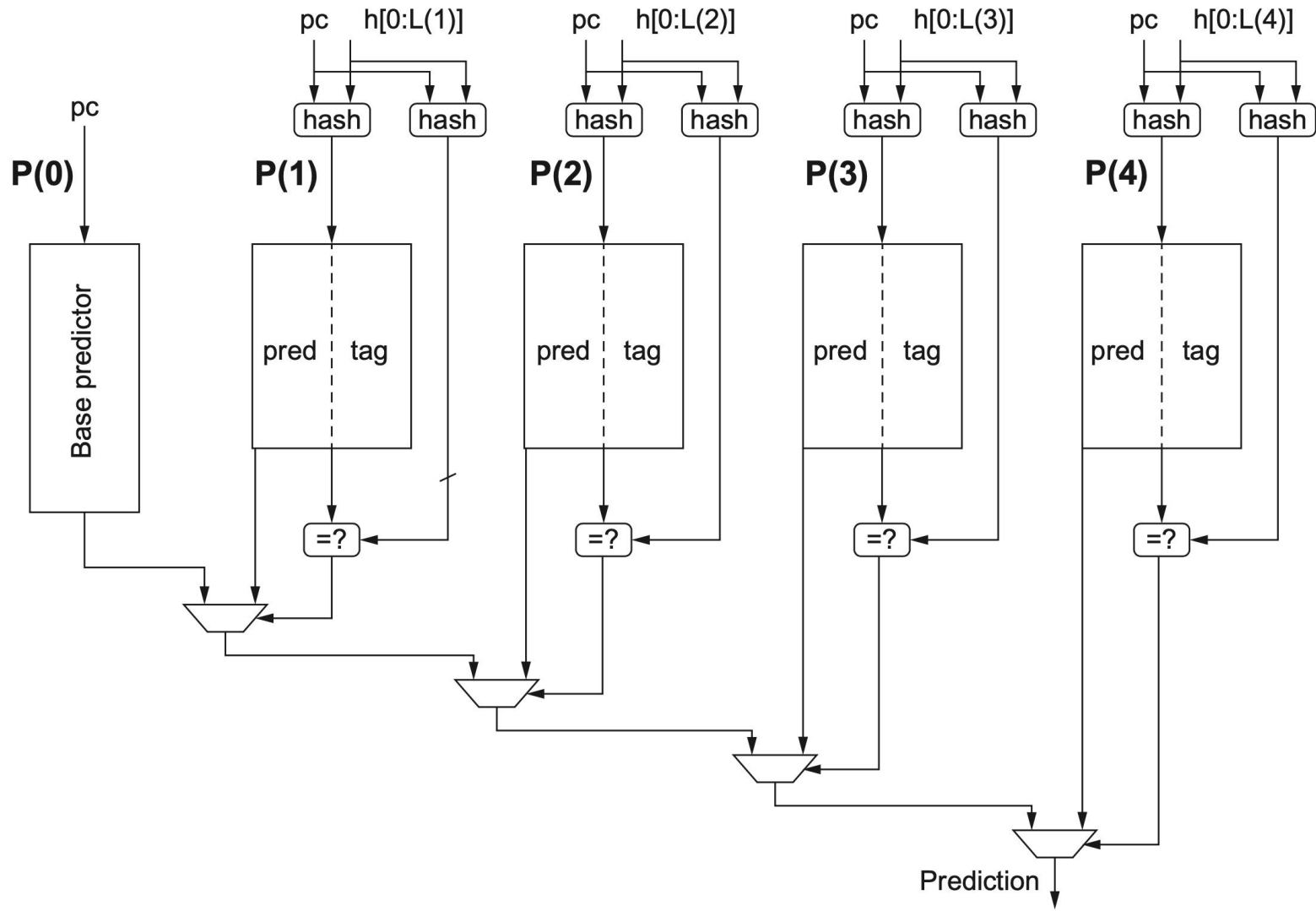
- Use a series of global predictors indexed with different length histories
- Branch history length forms a geometric series

TAgger Hybrid Predictor

- Use a series of global predictors indexed with different length histories
- Branch history length forms a GEometric series: 0, 2, 4, 8, 16...

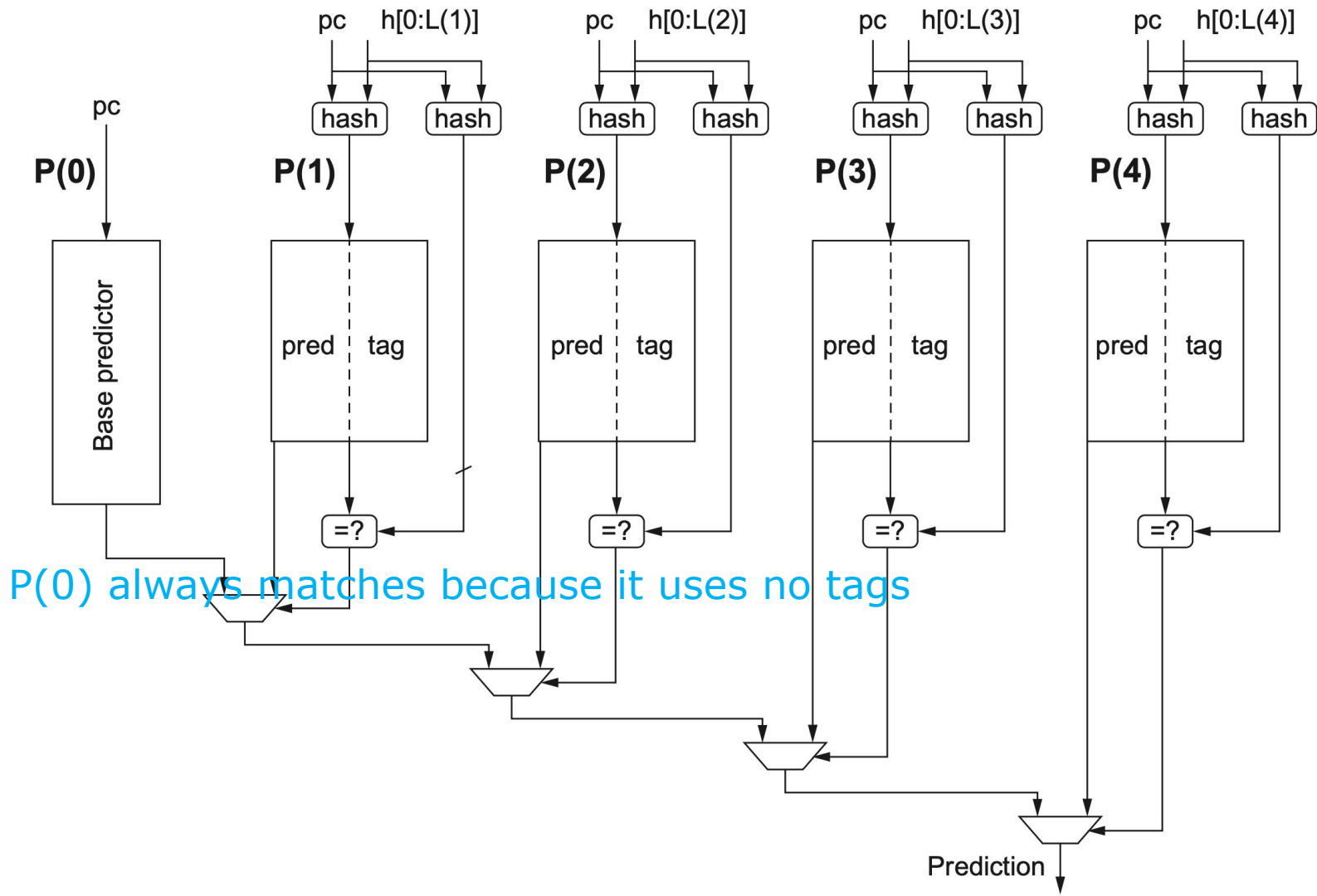
TAGE

$P(i)$ is accessed using hash of PC & history of the most recent i branches



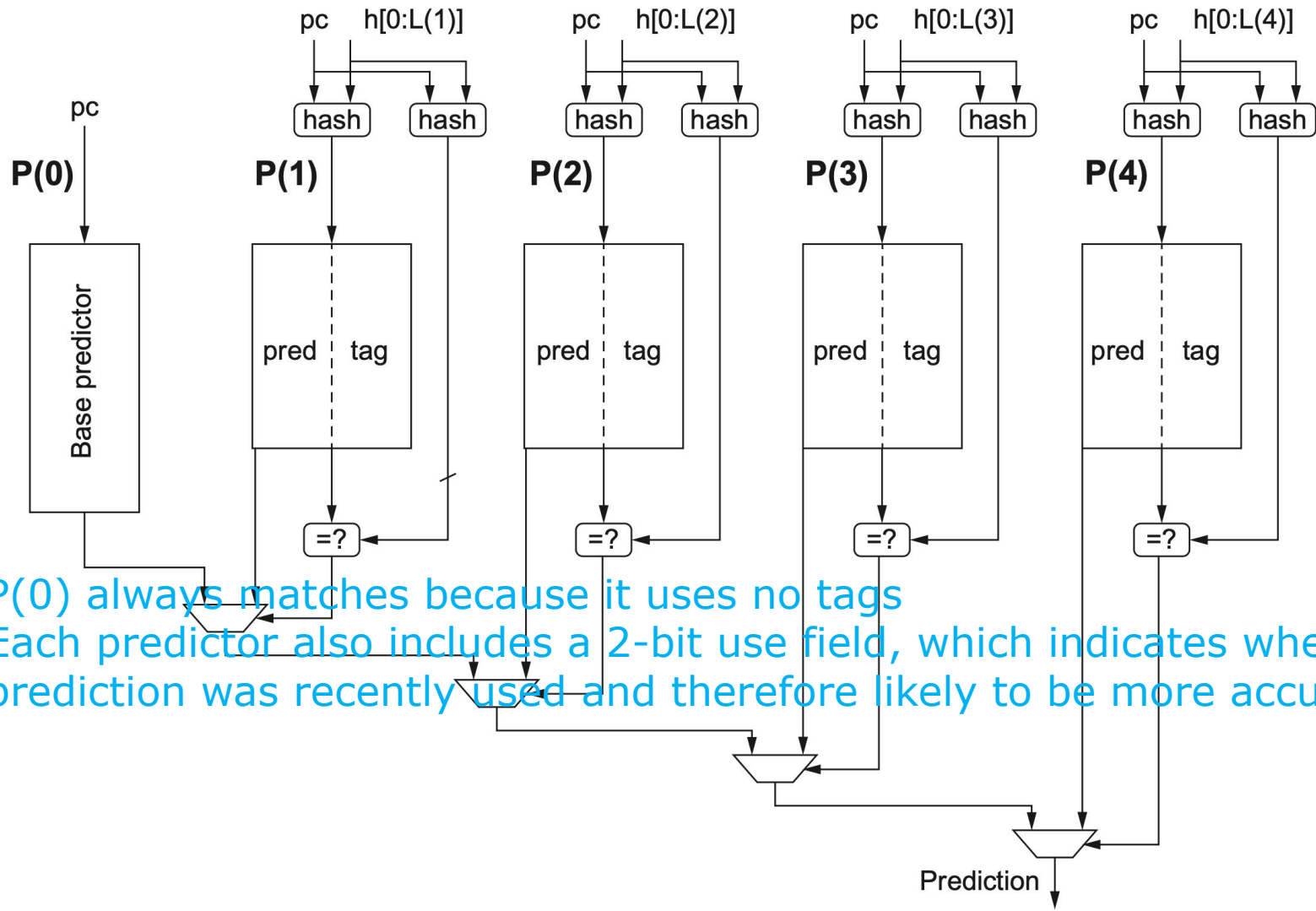
TAGE

$P(i)$ is accessed using hash of PC & history of the most recent i branches



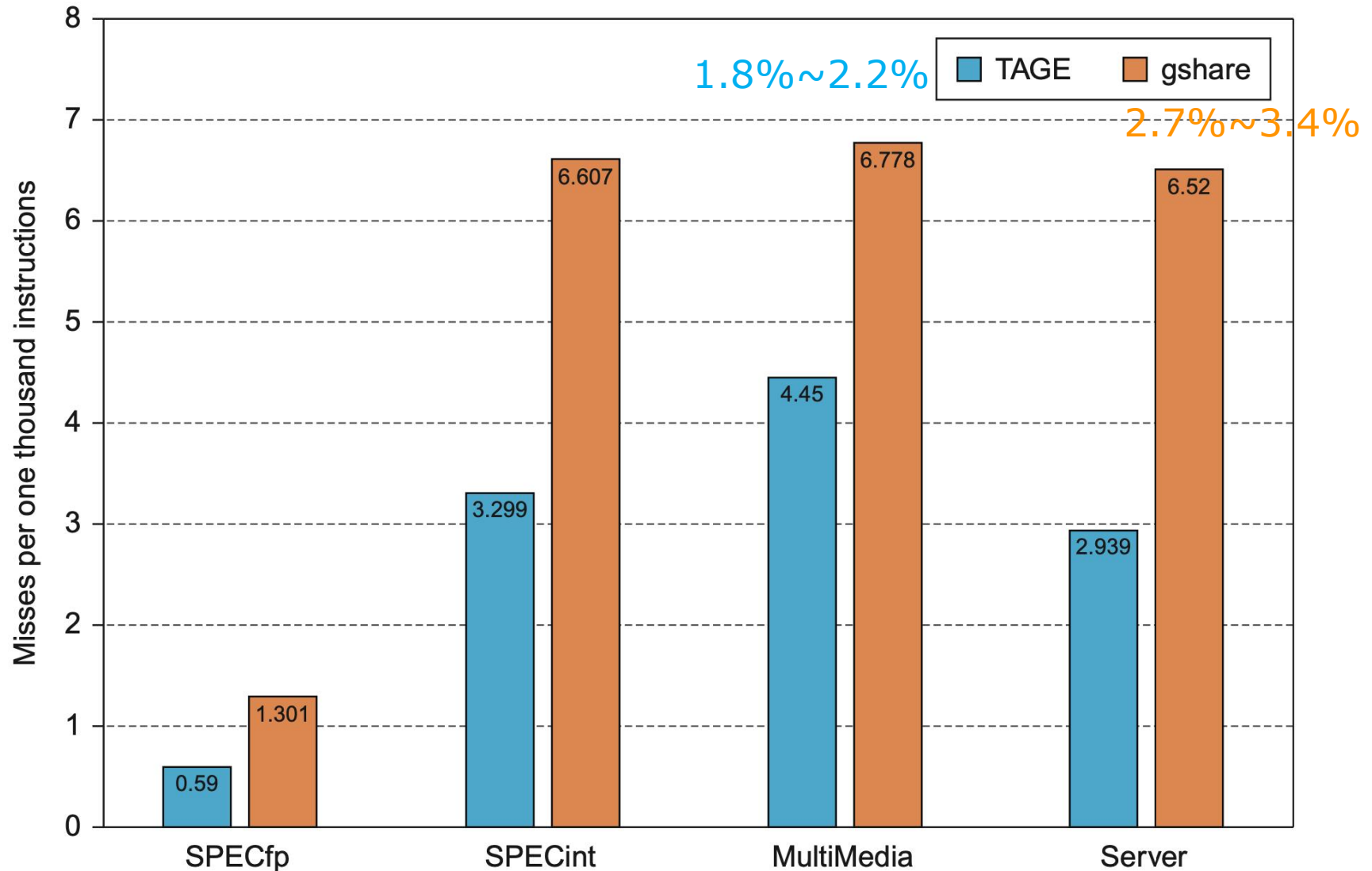
TAGE

$P(i)$ is accessed using hash of PC & history of the most recent i branches



TAGE Performance

branch frequency: 20%~25%



handle control hazard so far

how to handle data hazard?

data hazard?

data hazard?

fdiv.d f0, f2, f4

fadd.d f10, f0, f8

fsub.d f12, f8, f14

data hazard?

fdiv.d f0, f2, f4

fadd.d f10, f0, f8

fsub.d f12, f8, f14

data hazard?

fdiv.d f0, f2, f4

fadd.d f10, f0, f8

fsub.d f12, f8, f14



fsub.d is stalled until fdiv.d produces f0
and fadd.d resumes;

data hazard?

fdiv.d f0, f2, f4

fadd.d f10, f0, f8

fsub.d f12, f8, f14

fsub.d is, however, not data-dependent
on either fdiv.d or fadd.d

how to handle data hazard?

fdiv.d f0, f2, f4

fadd.d f10, f0, f8

fsub.d f12, f8, f14

fsub.d is, however, not data-dependent
on either fdiv.d or fadd.d

break the rule

```
fdiv.d  f0, f2, f4  
fadd.d  f10, f0, f8  
fsub.d  f12, f8, f14
```



fsub.d is executed while fadd.d waits for
fdiv.d to produce f0

dynamic scheduling

hardware reorders the instruction execution
to reduce the stalls while maintaining
data flow and exception behavior

Dynamic Scheduling

- Enable out-of-order execution (and thus out-of-order completion)
- Split ID into two stages:
 - Issue**: decode instructions, check for structural hazards
 - Read operands**: wait until no data hazards, then read operands

IF Issue Read-Operands EX MEM WB

Dynamic Scheduling

- Issue from IF: in order
- Read operands to EX: out of order when there are sufficient resources and no data dependences
- **Scheduling policy**: upon stalling instr, issue other instructions if they do not depend on any active or stalled instruction

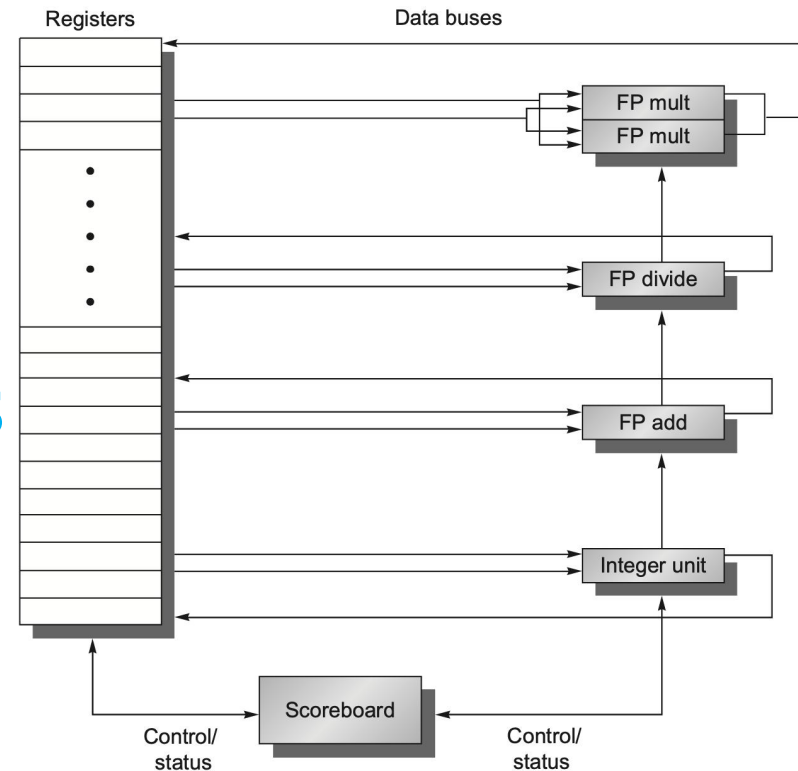
Dynamic Scheduling

- Issue from IF: in order
- Read operands to EX: out of order when there are sufficient resources and no data dependences
- **Scheduling policy**: upon stalling instr, issue other instructions if they do not depend on any active or stalled instruction
- **How to schedule**: use a centralized hazard detection and resolution unit

Scoreboarding

- Scoreboard controls the instruction progression from one step to the next by communicating with functional units

- Three parts:
 - Instruction status
 - Functional unit status
 - Register result status



Instruction Status

- Indicate which of the four steps the instruction is in
- **Four steps:** issue, read operands, execution, write back
- **Omit memory access** because the out-of-order focuses more on FP operations

Functional Unit Status

- Indicate the state of the functional unit
- Nine fields per functional unit (FU)

Busy—Indicates whether the unit is busy or not.

Op—Operation to perform in the unit (e.g., add or subtract).

Fi—Destination register.

Fj, Fk—Source-register numbers.

Qj, Qk—Functional units producing source registers Fj, Fk.

Rj, Rk—Flags indicating when Fj, Fk are ready and not yet read. Set to No after operands are read.

Register Result Status

- Indicate which functional unit will write each register, if an active instruction has the register as its destination
- Set to blank whenever no pending instructions will write that register

Scoreboard Example

Instruction		Instruction status			
		Issue	Read operands	Execution complete	Write result
L.D	F6, 34(R2)	✓	✓	✓	✓
L.D	F2, 45(R3)	✓	✓	✓	
MUL.D	F0, F2, F4	✓			
SUB.D	F8, F6, F2	✓			
DIV.D	F10, F0, F6	✓			
ADD.D	F6, F8, F2	stalled due to structural hazard with SUB.D			

all instructions
in scoreboard

2nd L.D is ready to write result

Scoreboard Example

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6, 34(R2)	✓	✓	✓	✓
L.D	F2, 45(R3)	✓	✓	✓	
MUL.D	F0, F2, F4	✓		wait for F2	
SUB.D	F8, F6, F2	✓		wait for F2	
DIV.D	F10, F0, F6	✓		wait for F0	
ADD.D	F6, F8, F2	stalled due to structural hazard with SUB.D			

2nd L.D is ready to write result

Scoreboard Example

		Instruction status			
		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	✓	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓	
MUL.D	F0,F2,F4	✓			
SUB.D	F8,F6,F2	✓			
DIV.D	F10,F0,F6	✓			
ADD.D	F6,F8,F2				

Functional unit status									
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Scoreboard Example

Instruction status				
Instruction		Issue	Read operands	Execution complete Write result
L.D	F6,34(R2)	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓
MUL.D	F0,F2,F4	✓	Busy—Indicates whether the unit is busy or not. Op—Operation to perform in the unit (e.g., add or subtract). Fi—Destination register. Fj, Fk—Source-register numbers. Qj, Qk—Functional units producing source registers Fj, Fk. Rj, Rk—Flags indicating when Fj, Fk are ready and not yet read. Set to No after operands are read.	
SUB.D	F8,F6,F2	✓		
DIV.D	F10,F0,F6	✓		
ADD.D	F6,F8,F2			

Functional unit status									
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Scoreboard Example

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	✓	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓	
MUL.D	F0,F2,F4	✓	Busy—Indicates whether the unit is busy or not. Op—Operation to perform in the unit (e.g., add or subtract). Fi—Destination register. Fj, Fk—Source-register numbers. Qj, Qk—Functional units producing source registers Fj, Fk. Rj, Rk—Flags indicating when Fj, Fk are ready and not yet read. Set to No after operands are read.		
SUB.D	F8,F6,F2	✓			
DIV.D	F10,F0,F6	✓			
ADD.D	F6,F8,F2				

		Functional unit status							
		dest	s1	s2	s1-src	s2-src			
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Scoreboard Example

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6, 34(R2)	✓	✓	✓	✓
L.D	F2, 45(R3)	✓	✓	✓	
MUL.D	F0, F2, F4	✓			
SUB.D	F8, F6, F2	✓			
DIV.D	F10, F0, F6	✓			
ADD.D	F6, F8, F2				

Busy—Indicates whether the unit is busy or not.

Op—Operation to perform in the unit (e.g., add or subtract).

Fi—Destination register.

Fj, Fk—Source-register numbers.

Qj, Qk—Functional units producing source registers Fj, Fk.

Rj, Rk—Flags indicating when Fj, Fk are ready and not yet read. Set to No after operands are read.

		Functional unit status							
		dest	s1	s2	s1-src	s2-src			
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

F2 from 2nd L.D

Scoreboard Example

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	✓	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓	
MUL.D	F0,F2,F4	✓			
SUB.D	F8,F6,F2	✓			
DIV.D	F10,F0,F6	✓			
ADD.D	F6,F8,F2				

Busy—Indicates whether the unit is busy or not.
 Op—Operation to perform in the unit (e.g., add or subtract).
 Fi—Destination register.
 Fj, Fk—Source-register numbers.
 Qj, Qk—Functional units producing source registers Fj, Fk.
 Rj, Rk—Flags indicating when Fj, Fk are ready and not yet read. Set to No after operands are read.

		Functional unit status							
		dest	s1	s2	s1-src	s2-src			
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

F2 from 2nd L.D

Scoreboard Example

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	✓	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓	
MUL.D	F0,F2,F4	✓			
SUB.D	F8,F6,F2	✓			
DIV.D	F10,F0,F6	✓			
ADD.D	F6,F8,F2				

Busy—Indicates whether the unit is busy or not.
 Op—Operation to perform in the unit (e.g., add or subtract).
 Fi—Destination register.
 Fj, Fk—Source-register numbers.
 Qj, Qk—Functional units producing source registers Fj, Fk.
 Rj, Rk—Flags indicating when Fj, Fk are ready and not yet read. Set to No after operands are read.

		Functional unit status							
		dest	s1	s2	s1-src	s2-src			
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2	Integer		Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

F0 from MUL.D

Scoreboard Example

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	✓	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓	
MUL.D	F0,F2,F4	✓			
SUB.D	F8,F6,F2	✓			
DIV.D	F10,F0,F6	✓			
ADD.D	F6,F8,F2				

Busy—Indicates whether the unit is busy or not.
 Op—Operation to perform in the unit (e.g., add or subtract).
 Fi—Destination register.
 Fj, Fk—Source-register numbers.
 Qj, Qk—Functional units producing source registers Fj, Fk.
 Rj, Rk—Flags indicating when Fj, Fk are ready and not yet read. Set to No after operands are read.

		Functional unit status							
		dest	s1	s2	s1-src	s2-src			
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F6	F2	F4	Integer		No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2	Integer		Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

these No's for ops Not Ready?

F0 from MUL.D

Scoreboard Example

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6, 34 (R2)	✓	✓	✓	✓
L.D	F2, 45 (R3)	✓	✓	✓	
MUL.D	F0, F2, F4	✓	<i>Register result status</i> —Indicates which functional unit will write each register, if an active instruction has the register as its destination. This field is set to blank whenever there are no pending instructions that will write that register.		
SUB.D	F8, F6, F2	✓			
DIV.D	F10, F0, F6	✓			
ADD.D	F6, F8, F2				

Functional unit status									
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2					No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status									
	F0	F2	F4	F6	F8	F10	F12	...	F30
FU	Mult1	Integer			Add	Divide			

Scoreboard Example

Instruction		Instruction status			
		Issue	Read operands	Execution complete	Write result
L.D	F6, 34 (R2)	✓	✓	✓	✓
L.D	F2, 45 (R3)	✓	✓	✓	
MUL.D	F0, F2, F4	✓	<i>Register result status</i> —Indicates which functional unit will write each register, if an active instruction has the register as its destination. This field is set to blank whenever there are no pending instructions that will write that register.		
SUB.D	F8, F6, F2	✓			
DIV.D	F10, F0, F6	✓			
ADD.D	F6, F8, F2				

check corresponding functional units

Functional unit status									
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	F2				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

check destination registers

FU		Register result status								
		F0	F2	F4	F6	F8	F10	F12	...	F30
		Mult1	Integer			Add	Divide			

Scoreboard Example

MUL.D is ready to write result

Instruction		Instruction status			
		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	✓	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓	✓
MUL.D	F0,F2,F4	✓	✓	✓	
SUB.D	F8,F6,F2	✓	✓	✓	✓
DIV.D	F10,F0,F6	✓			
ADD.D	F6,F8,F2	✓	✓	✓	

Functional unit status								
Name	Busy	Op	dest Fi	s1 Fj	s2 Fk	s1-src Qj	s2-src Qk	Rj Rk
Integer	No							
Mult1	Yes	Mult	F0	F2	F4			No No
Mult2	No							
Add	Yes	Add	F6	F8	F2			No No
Divide	Yes	Div	F10	F0	F6	Mult1		No Yes

Register result status									
	F0	F2	F4	F6	F8	F10	F12	...	F30
FU	Mult 1			Add		Divide			

Scoreboard Example

DIV.D is ready to write result

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	✓	✓	✓	✓
L.D	F2,45(R3)	✓	✓	✓	✓
MUL.D	F0,F2,F4	✓	✓	✓	✓
SUB.D	F8,F6,F2	✓	✓	✓	✓
DIV.D	F10,F0,F6	✓	✓	✓	
ADD.D	F6,F8,F2	✓	✓	✓	✓

Functional unit status									
Name	Busy	Op	dest	s1	s2	s1-src	s2-src	Rj	Rk
			Fi	Fj	Fk	Qj	Qk		
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6			No	Yes

F6 already read?

Register result status									
	F0	F2	F4	F6	F8	F10	F12	...	F30
FU	Mult 1			Add			Divide		

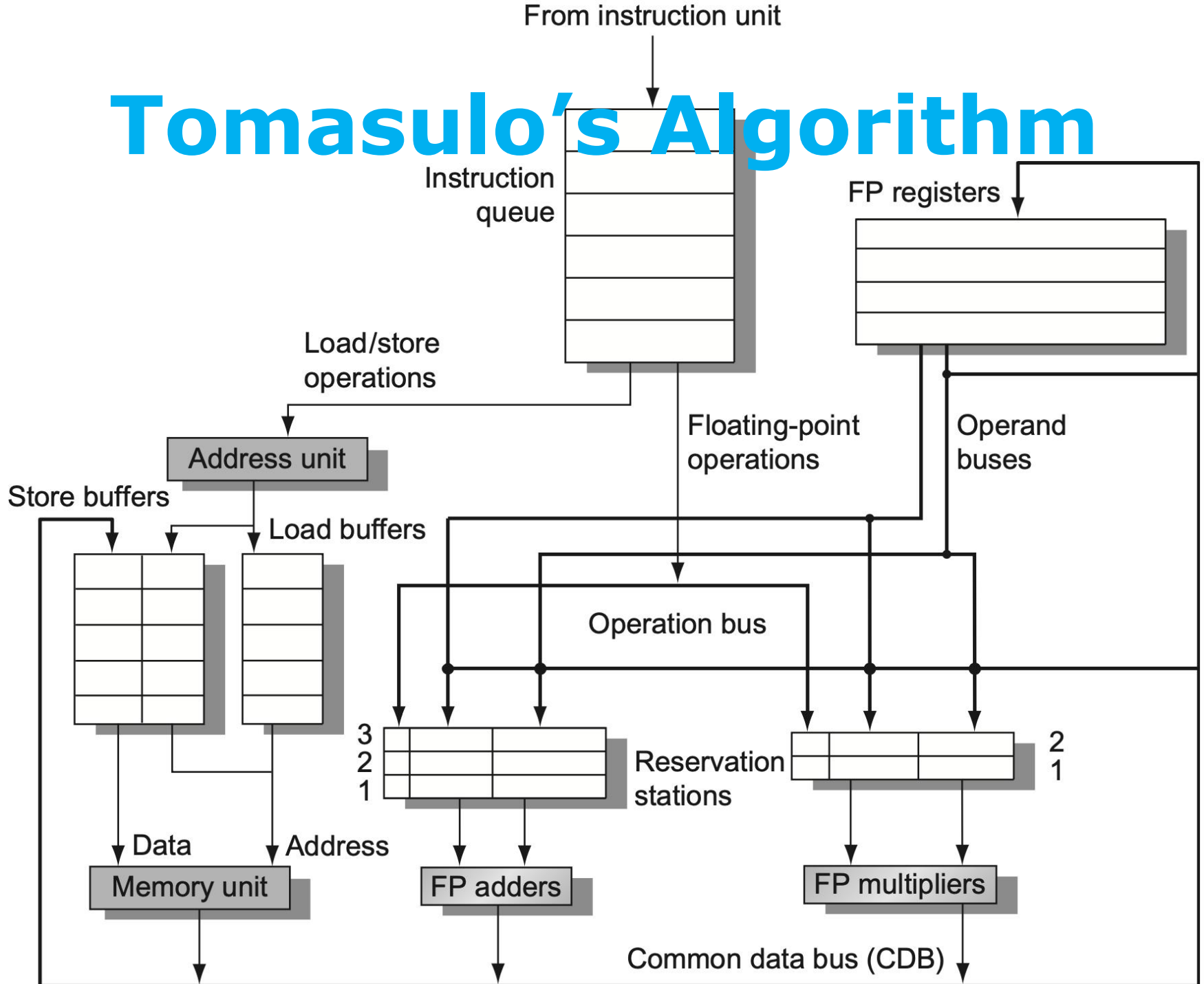


scoreboard not for exam?!

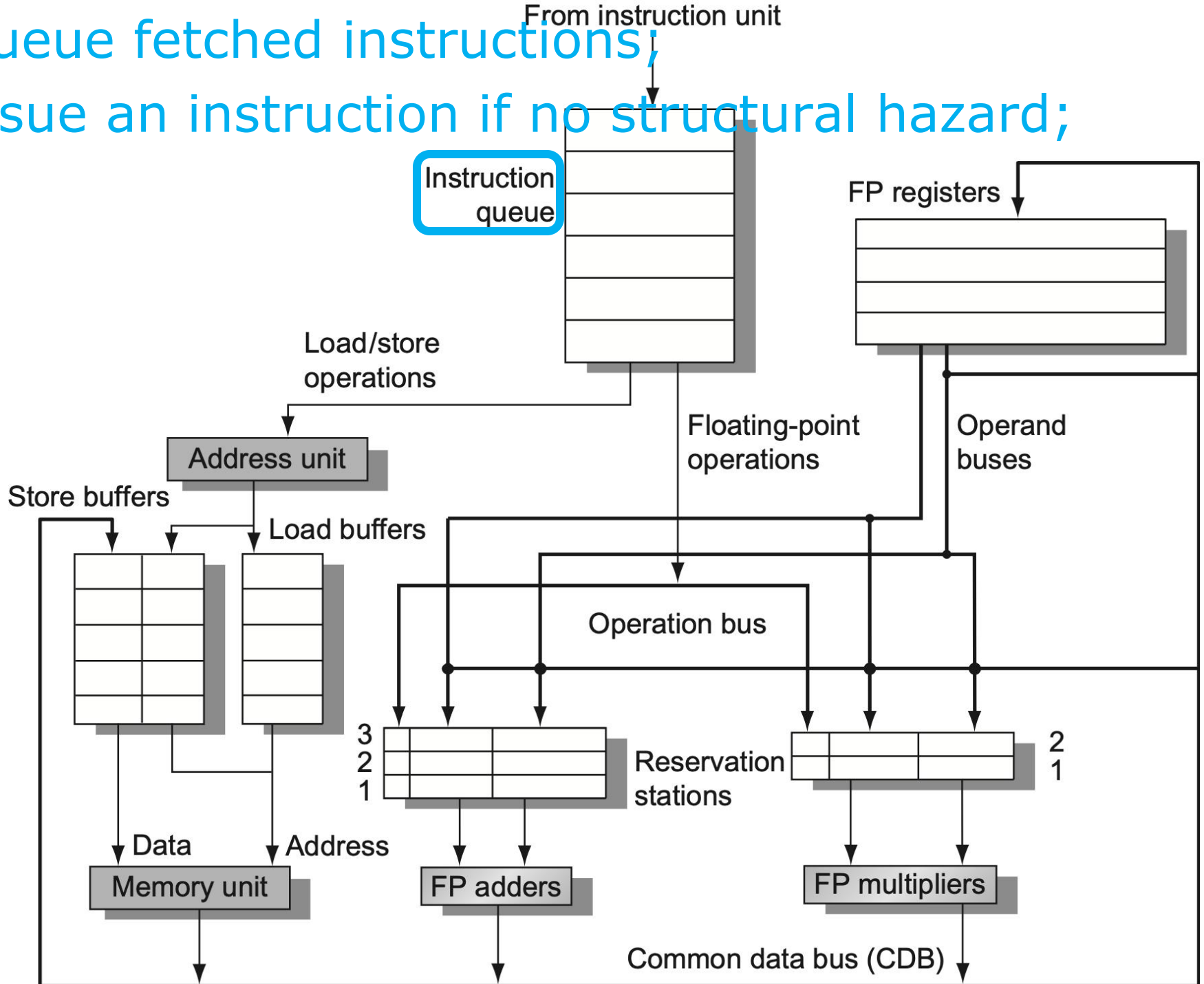


scoreboard not for exam?!
yet helpful for understanding the
more sophisticated Tomasulo's Alg

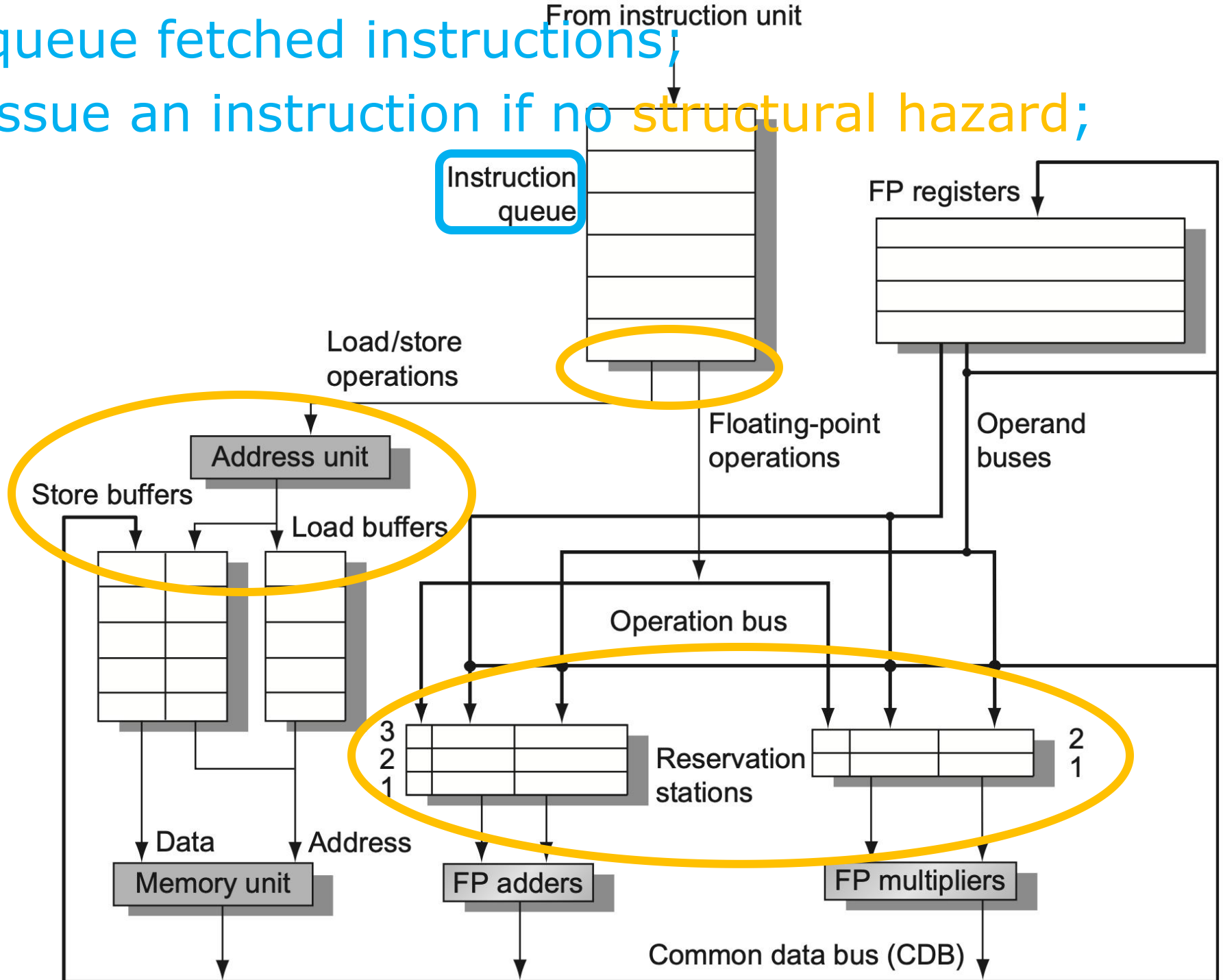
Tomasulo's Algorithm



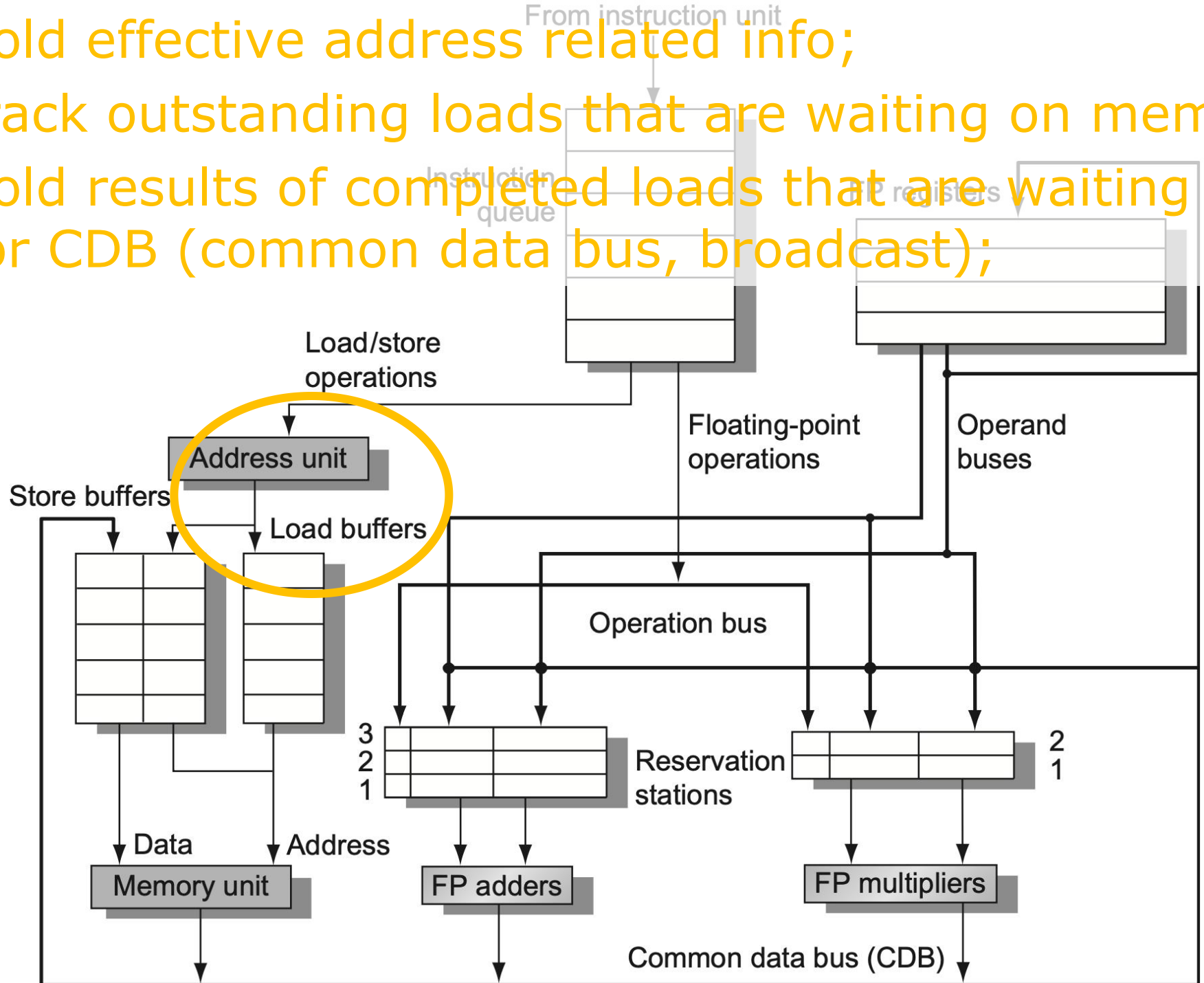
queue fetched instructions;
issue an instruction if no structural hazard;



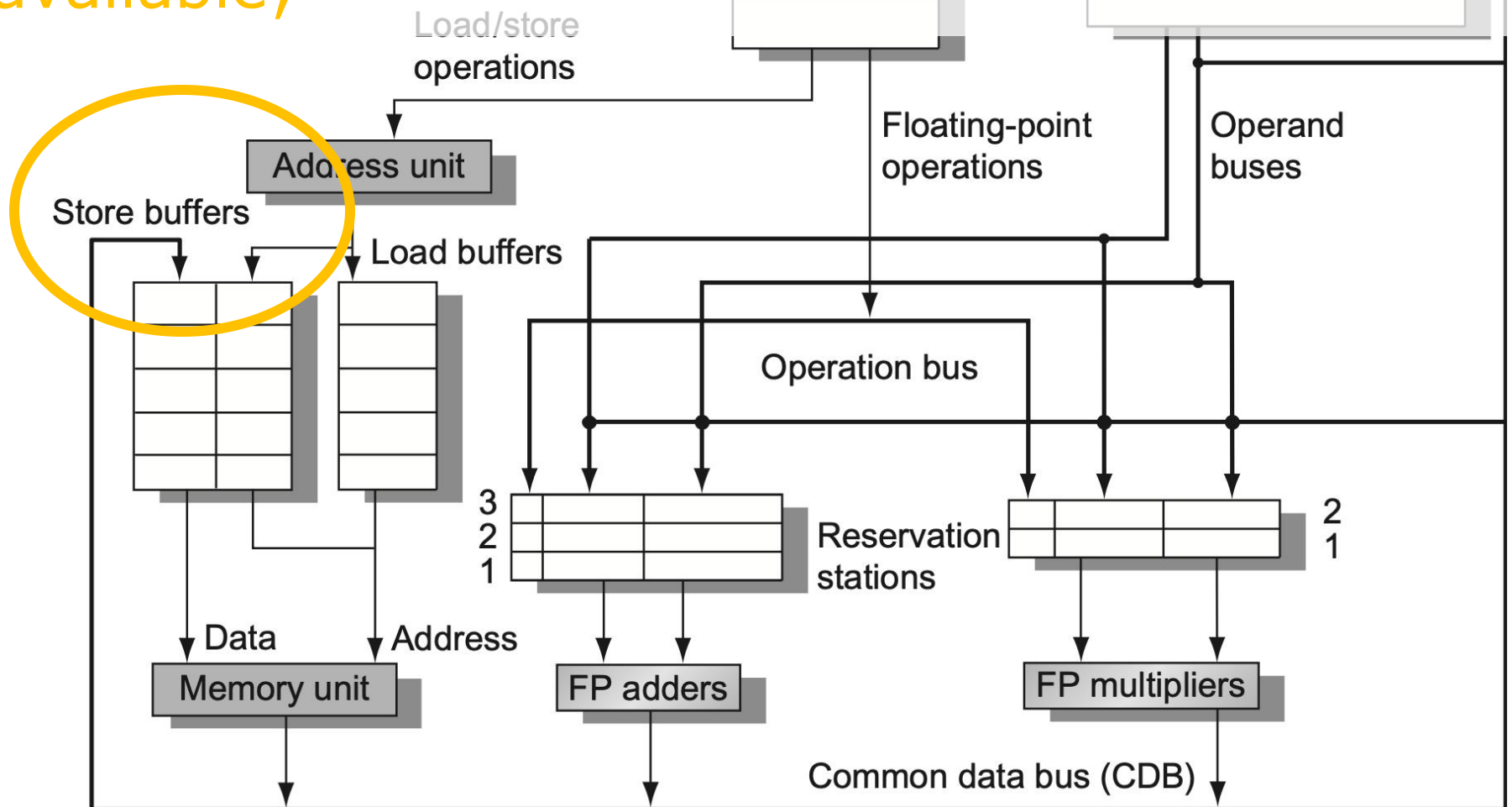
queue fetched instructions;
issue an instruction if no structural hazard;

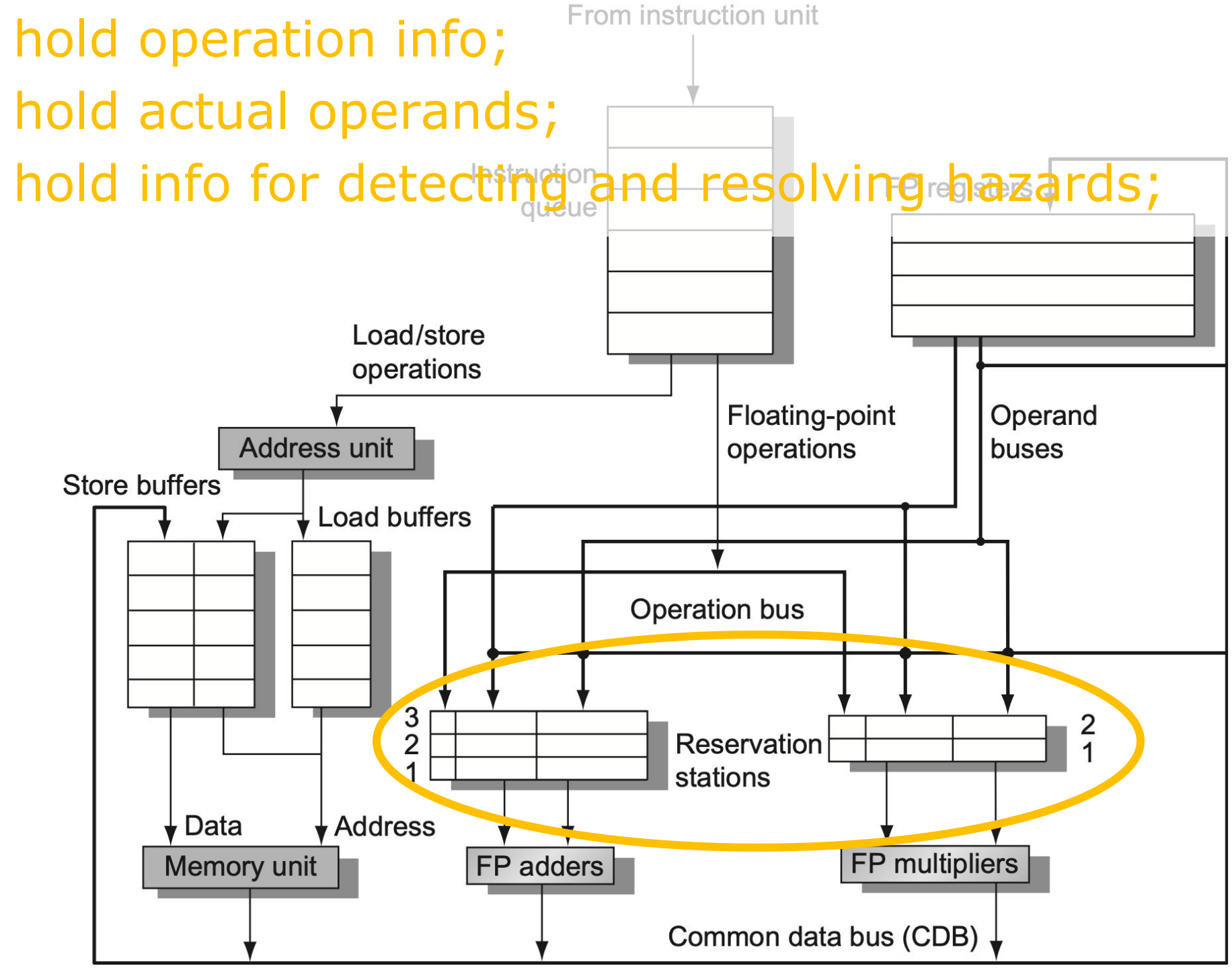


hold effective address related info;
track outstanding loads that are waiting on mem;
hold results of completed loads that are waiting
for CDB (common data bus, broadcast);

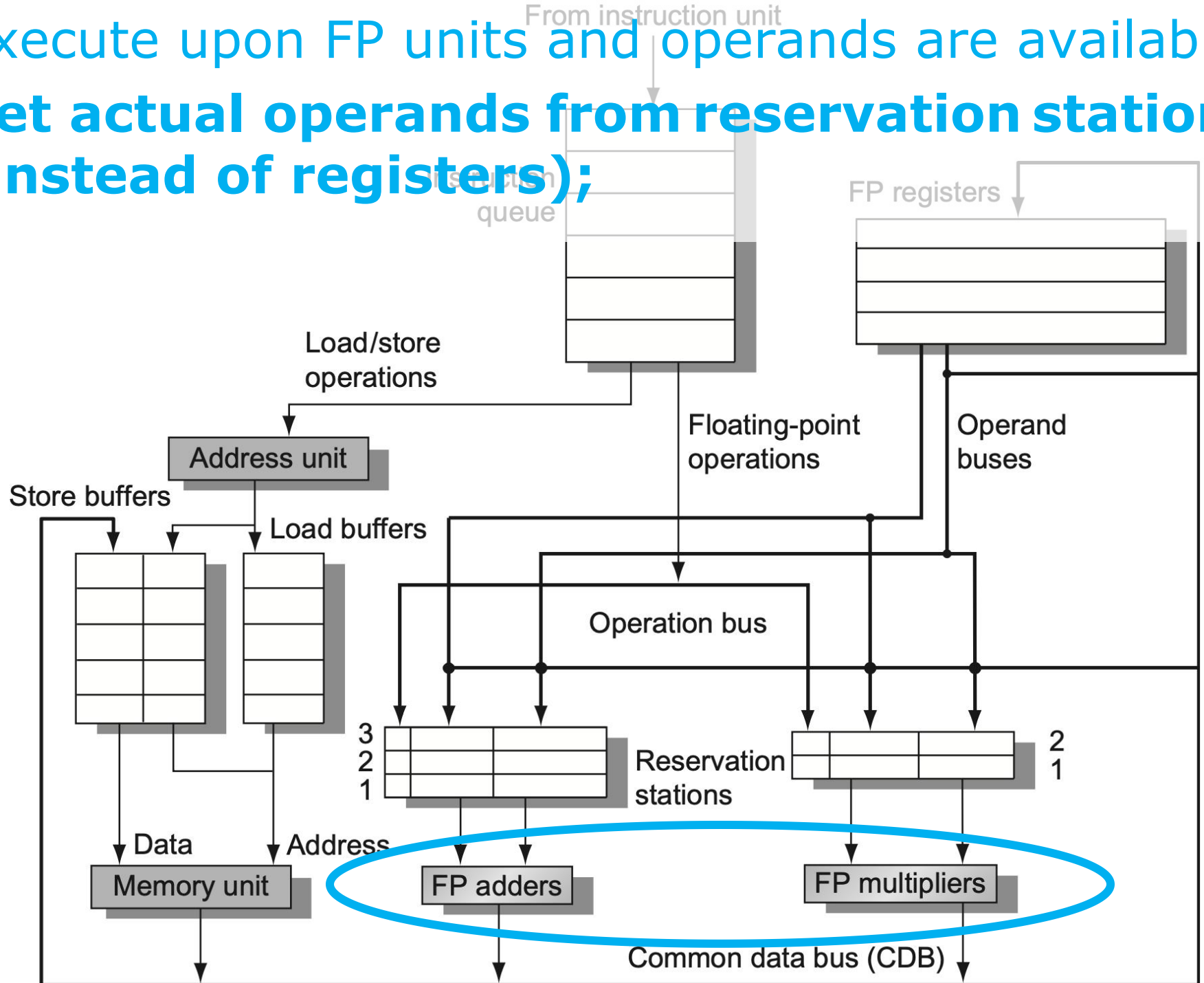


hold effective address related info;
hold dest mem addr of outstanding stores that
are waiting for data value to store;
hold address and value to store until memory is
available;

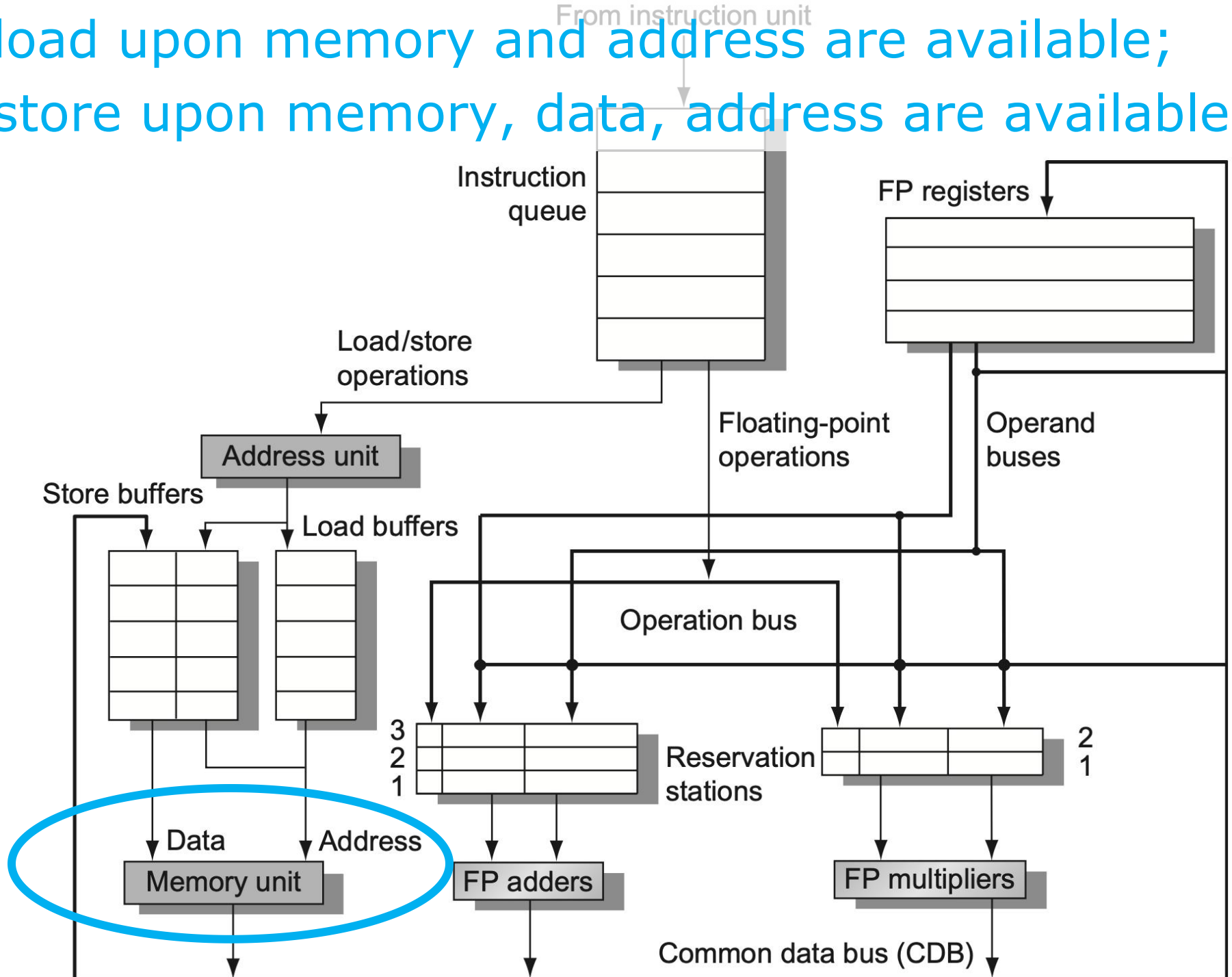




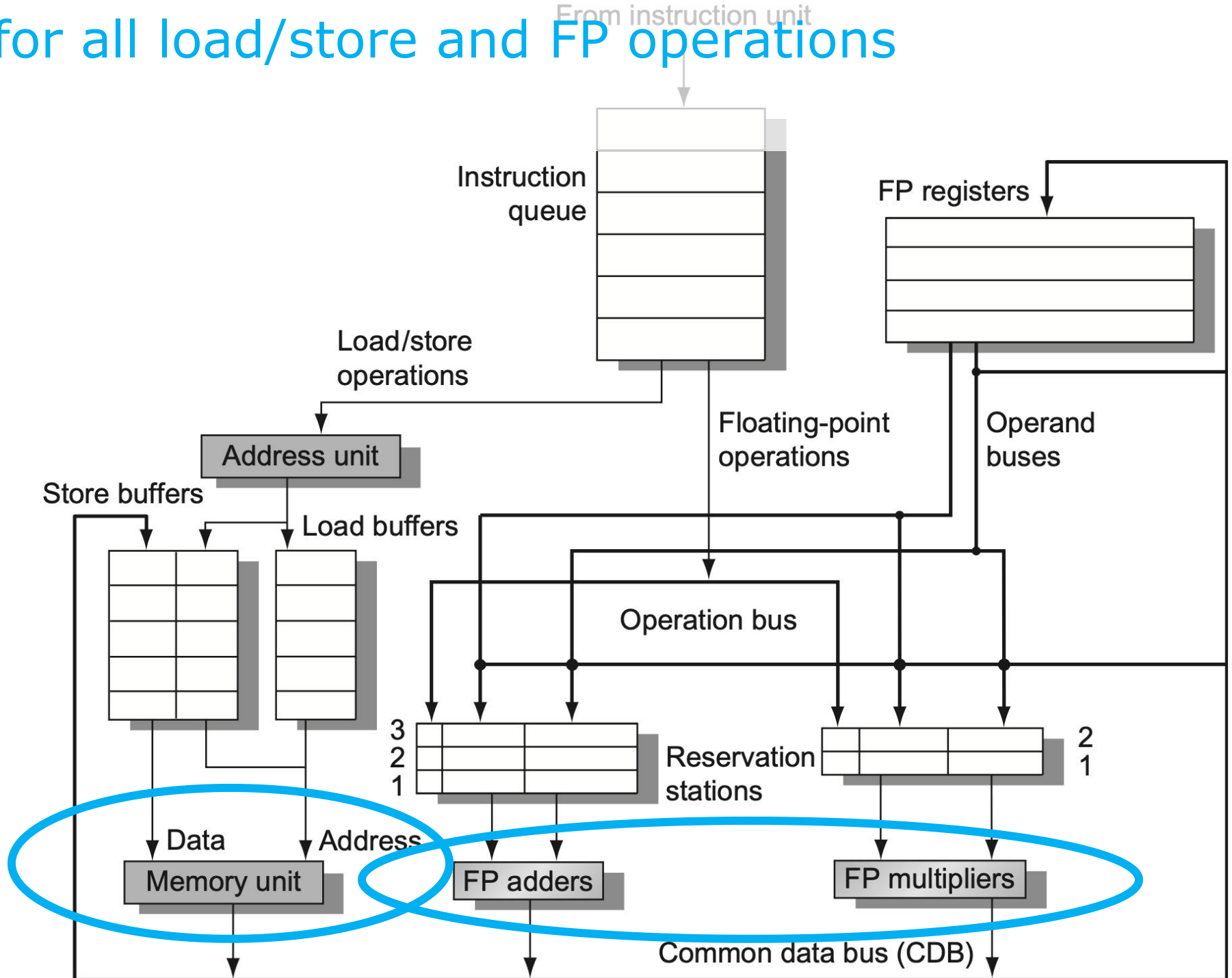
execute upon FP units and operands are available;
**get actual operands from reservation stations
(instead of registers);**



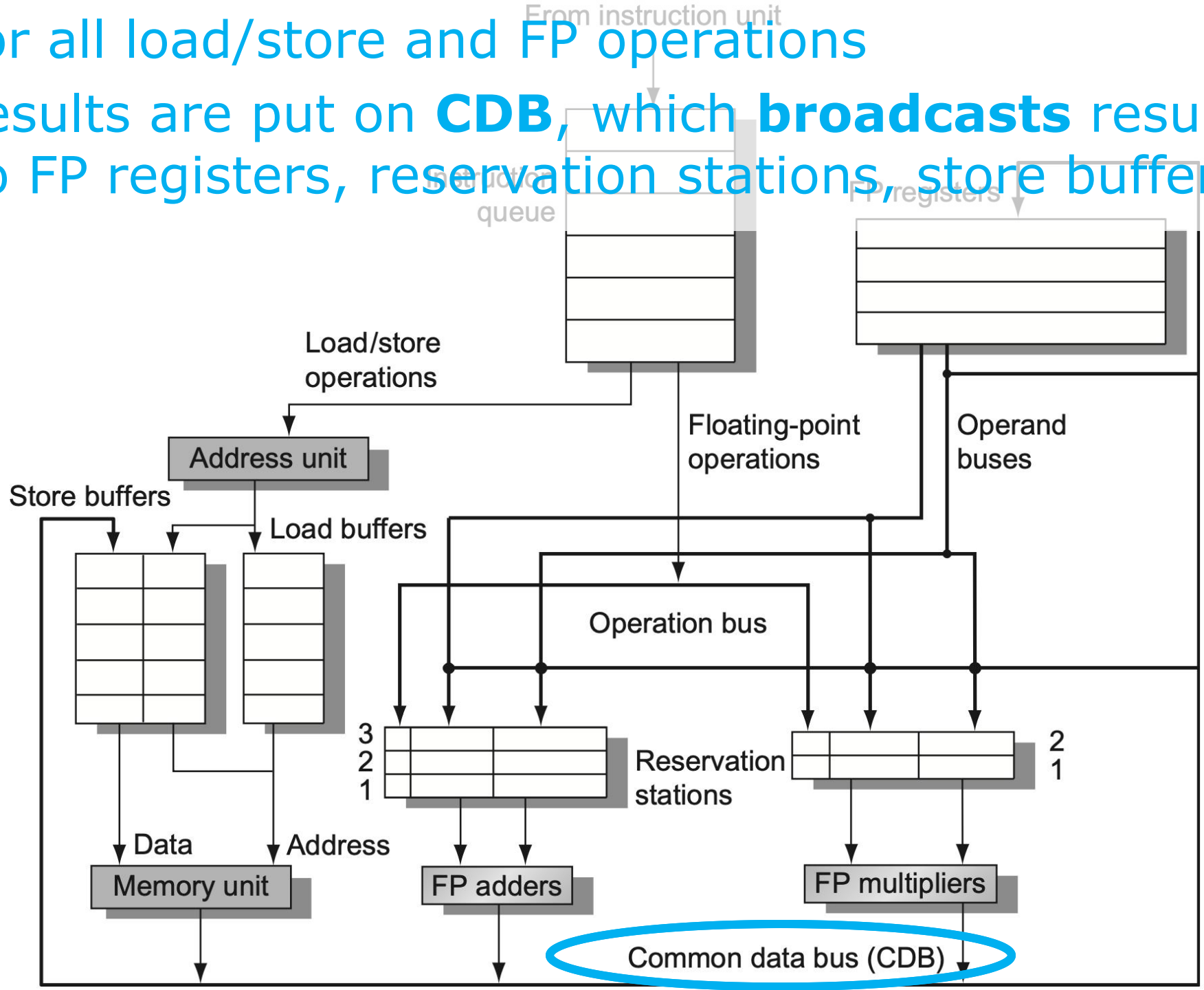
load upon memory and address are available;
store upon memory, data, address are available;



for all load/store and FP operations



for all load/store and FP operations
results are put on **CDB**, which **broadcasts** results
to FP registers, reservation stations, store buffers



Tomasulo's Algorithm

Two major advantages over scoreboard:

- Distribution of hazard detection logic across load/store buffers and reservation stations
- Elimination of stalls for WAW and WAR hazards through Register Renaming

Register Renaming

- Eliminate WAW and WAR hazards
- Rename related destination registers, including those with a pending read or write for an earlier instruction
- Enable out-of-order write that does not affect any instructions depending on an earlier value of an operand

Register Renaming

- Example code sequence

fdiv.d f0, f2, f4

fadd.d f6, f0, f8

fsd f6, 0(x1)

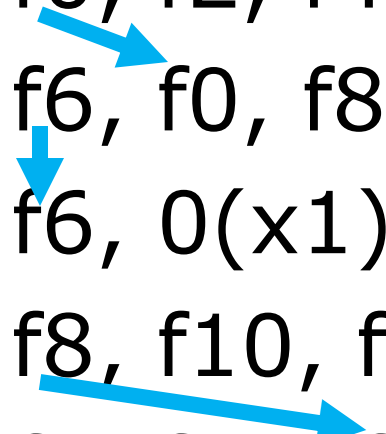
fsub.d f8, f10, f14

fmul.d f6, f10, f8

RAW Hazard

- Example code sequence

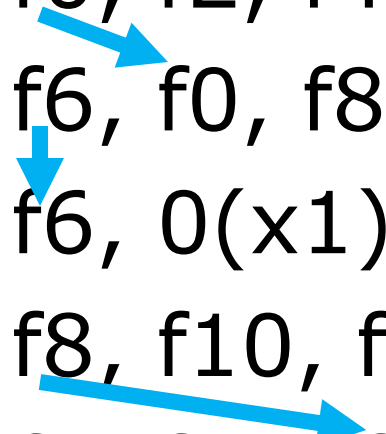
fdiv.d	f0, f2, f4
fadd.d	f6, f0, f8
fsd	f6, 0(x1)
fsub.d	f8, f10, f14
fmul.d	f6, f10, f8



RAW Hazard

- Example code sequence

```
fdiv.d    f0, f2, f4
fadd.d    f6, f0, f8
fsd       f6, 0(x1)
fsub.d    f8, f10, f14
fmul.d    f6, f10, f8
```



- Execute an instruction only when its operands are available

WAR Hazard

- Example code sequence

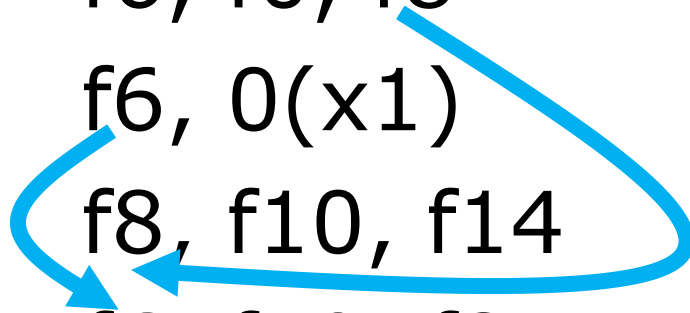
fdiv.d f0, f2, f4

fadd.d f6, f0, f8

fsd f6, 0(x1)

fsub.d f8, f10, f14

fmul.d f6, f10, f8



WAW Hazard

- Example code sequence

fdiv.d f0, f2, f4

fadd.d f6, f0, f8

fsd f6, 0(x1)

fsub.d f8, f10, f14

fmul.d f6, f10, f8



WAR/WAW Hazard

- Example code sequence

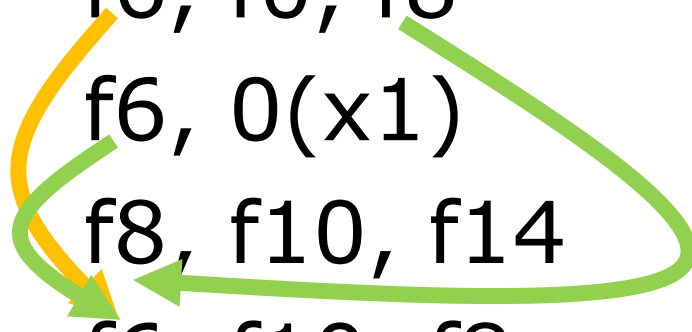
fdiv.d f0, f2, f4

fadd.d f6, f0, f8

fsd f6, 0(x1)

fsub.d f8, f10, f14

fmul.d f6, f10, f8



how to rename registers?

how to rename registers?

an extended set of physical registers is used to hold both the **architecturally visible registers** (x0,...x31 and f0,...f31) as well as temporary values;

how to rename registers?

if a given physical register does not appear as a source and it is not designated as arch, alternatively, another instruction that writes the same architectural reg commits, it may be reclaimed and reallocated;

WAR/WAW Hazard

- Example code sequence

fdiv.d f0, f2, f4

fadd.d f6, f0, f8

fsd f6, 0(x1)

fsub.d f8, f10, f14

fmul.d f6, f10, f8

guarantee
program order

- WAR: rename latter/destination
- WAW: rename former

WAR/WAW Hazard

- Example code sequence

fdiv.d f0, f2, f4

fadd.d f6, f0, f8

fsd f6, 0(x1)

fsub.d f8, f10, f14

fmul.d f6, f10, f8

guarantee
program order

- WAR: rename latter/destination
- WAW: rename former
- after rename dest, rename later source

WAR/WAW Hazard

- Example code sequence

fdiv.d f0, f2, f4

fadd.d f6, f0, f8

fsd f6, 0(x1)

fsub.d T, f10, f14

fmul.d f6, f10, T

guarantee
program order

- WAR: rename latter/destination
- WAW: rename former
- after rename dest, rename later source

WAR/WAW Hazard

- Example code sequence

fdiv.d f0, f2, f4

fadd.d S, f0, f8

fsd S, 0(x1)

fsub.d T, f10, f14

fmul.d f6, f10, T

guarantee
program order

- WAR: rename latter/destination
- WAW: rename former
- after rename dest, rename later source

WAR/WAW Hazard

- Example code sequence

fdiv.d f0, f2, f4

fadd.d S, f0, f8

fsd S, 0(x1)

fsub.d T, f10, f14

fmul.d f6, f10, T

guarantee
program order

- WAR: rename latter/destination
- WAW: rename former
- after rename dest, rename later source

Register Renaming

- Example: six-instruction code sequence
- Architectural registers x1, x2, and x3 are initially held by physical registers p1, p2, and p3, respectively

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	add x1,x2,x3	p32	add p32,p2,p3	x1-> p32
2	sub x1,x1,x2	p33	sub p33,p32,p2	x1->p33
3	add x2,x1,x2	p34	add p34,p33,p2	x2->p34
4	sub x1,x3,x2	p35	sub p35,p3,p34	x1->p35
5	add x1,x1,x2	p36	add p36,p35,p34	x1->p36
6	sub x1,x3,x1	p37	sub p37,p3,p36	x1->p37

Register Renaming

- Example: six-instruction code sequence
- Architectural registers x1, x2, and x3 are initially held by physical registers p1, p2, and p3, respectively

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	add x1, x2, x3	p32	add p32, p2, p3	x1-> p32
2	sub x1, x1, x2	p33	sub p33, p32, p2	x1->p33
3	add x2, x1, x2	p34	add p34, p33, p2	x2->p34
4	sub x1, x3, x2	p35	sub p35, p3, p34	x1->p35
5	add x1, x1, x2	p36	add p36, p35, p34	x1->p36
6	sub x1, x3, x1	p37	sub p37, p3, p36	x1->p37

Register Renaming

- Example: six-instruction code sequence
- Architectural registers x1, x2, and x3 are initially held by physical registers p1, p2, and p3, respectively

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	add x1, x2, x3	p32	add p32, p2, p3	x1-> p32
2	sub x1, x1, x2	p33	sub p33, p32, p2	x1->p33
3	add x2, x1, x2	p34	add p34, p33, p2	x2->p34
4	sub x1, x3, x2	p35	sub p35, p3, p34	x1->p35
5	add x1, x1, x2	p36	add p36, p35, p34	x1->p36
6	sub x1, x3, x1	p37	sub p37, p3, p36	x1->p37

Register Renaming

- Example: six-instruction code sequence
- Architectural registers x1, x2, and x3 are initially held by physical registers p1, p2, and p3, respectively

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	add x1, x2, x3	p32	add p32, p2, p3	x1-> p32
2	sub x1, x1, x2	p33	sub p33, p32, p2	x1->p33
3	add x2, x1, x2	p34	add p34, p33, p2	x2->p34
4	sub x1, x3, x2	p35	sub p35, p3, p34	x1->p35
5	add x1, x1, x2	p36	add p36, p35, p34	x1->p36
6	sub x1, x3, x1	p37	sub p37, p3, p36	x1->p37

Register Renaming

- Example: six-instruction code sequence
- Architectural registers x1, x2, and x3 are initially held by physical registers p1, p2, and p3, respectively

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	add x1, x2, x3	p32	add p32, p2, p3	x1-> p32
2	sub x1, x1, x2	p33	sub p33, p32, p2	x1->p33
3	add x2, x1, x2	p34	add p34, p33, p2	x2->p34
4	sub x1, x3, x2	p35	sub p35, p3, p34	x1->p35
5	add x1, x1, x2	p36	add p36, p35, p34	x1->p36
6	sub x1, x3, x1	p37	sub p37, p3, p36	x1->p37

Register Renaming

- More examples:

add x1, x2, x3

sub x4, x1, x5

xor x6, x7, x8

mul x5, x8, x9

add x4, x8, x9

Register Renaming

- More examples:

add x1, x2, x3

sub x4, x1, x5

xor x6, x7, x8

mul x5, x8, x9

add x4, x8, x9

add p1, x2, x3

sub p2, p1, x5

xor p3, x7, x8

mul p4, x8, x9

add p5, x8, x9

Register Renaming

- More examples:

add x1, x2, x3

sub x4, x1, x5

xor x6, x7, x8

mul x5, x8, x9

add x4, x8, x9

add p1, x2, x3

sub p2, p1, x5

xor p3, x7, x8

mul p4, x8, x9

add p5, x8, x9

follow default allocation

Register Renaming

- More examples:

add x1, x2, x3

sub x4, x1, x5

xor x6, x7, x8

mul x5, x8, x9

add x4, x8, x9

add p1, p12, p13

sub p2, p1, p15

xor p3, p17, p18

mul p4, p18, p19

add p5, p18, p19

follow default allocation

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

x1	x2	x3	x4	x5	x6
p1	p2	p3	p4	p5	p6

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

mul p7, p2, p2

x1	x2	x3	x4	x5	x6
p1	p7	p3	p4	p5	p6

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

mul p7, p2, p2

add p8, p1, p7

x1	x2	x3	x4	x5	x6
p8	p7	p3	p4	p5	p6

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

mul p7, p2, p2

add p8, p1, p7

mul p9, p4, p4

x1	x2	x3	x4	x5	x6
p8	p7→p9	p3	p4	p5	p6

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

mul p7, p2, p2

add p8, p1, p7

mul p9, p4, p4

add p10, p3, p9

x1	x2	x3	x4	x5	x6
p8	p9	p10	p4	p5	p6

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

mul p7, p2, p2

add p8, p1, p7

mul p9, p4, p4

add p10, p3, p9

mul p11, p6, p6

x1	x2	x3	x4	x5	x6
p8	p9→p11	p10	p4	p5	p6

Register Renaming

- More examples:

mul x2, x2, x2

add x1, x1, x2

mul x2, x4, x4

add x3, x3, x2

mul x2, x6, x6

add x5, x5, x2

mul p7, p2, p2

add p8, p1, p7

mul p9, p4, p4

add p10, p3, p9

mul p11, p6, p6

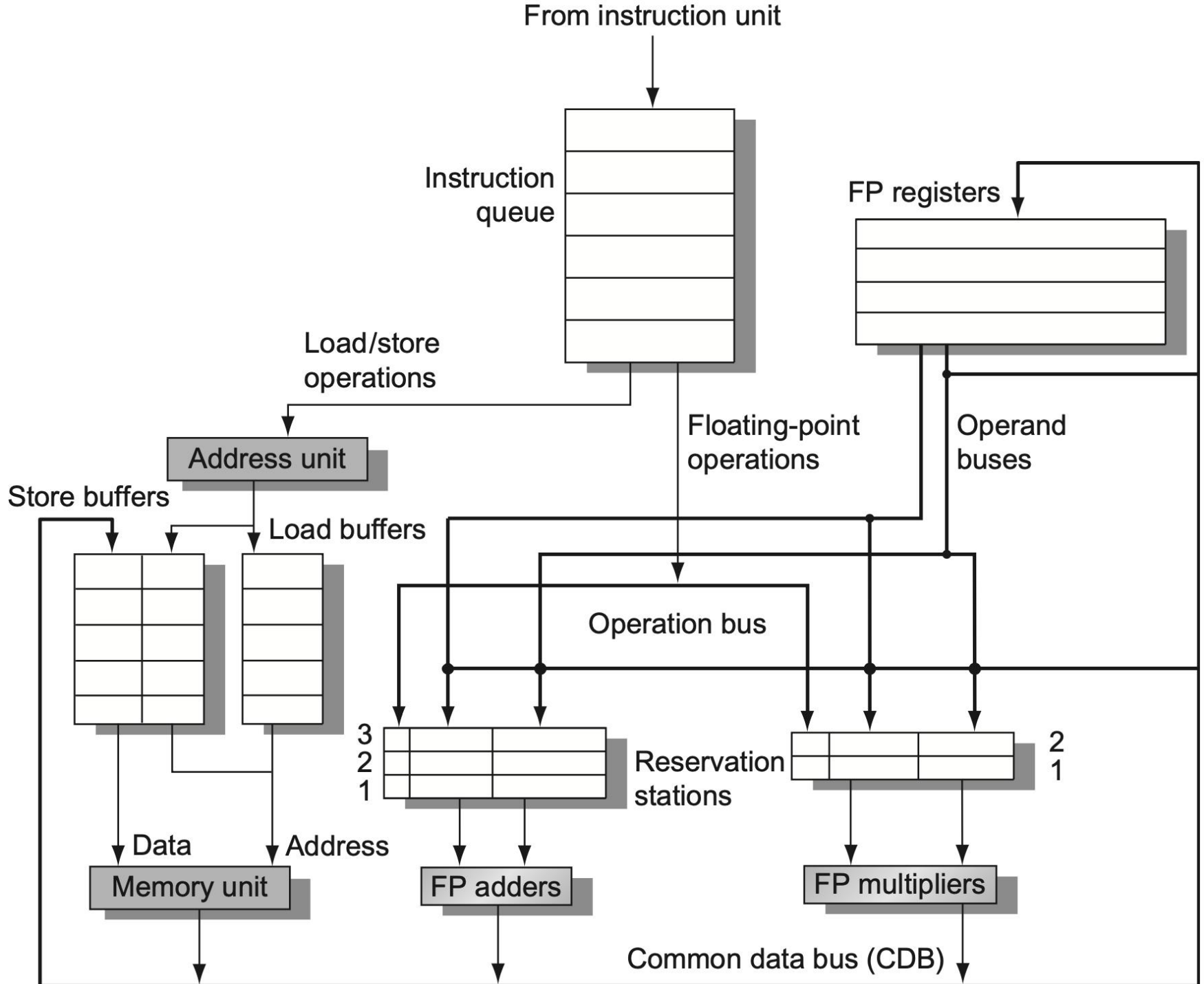
add p12, p5, p11

x1	x2	x3	x4	x5	x6
p8	p11	p10	p4	p12	p6

in-order vs out-of-order!

for in-order execution, all instructions after a stalled instruction cannot proceed

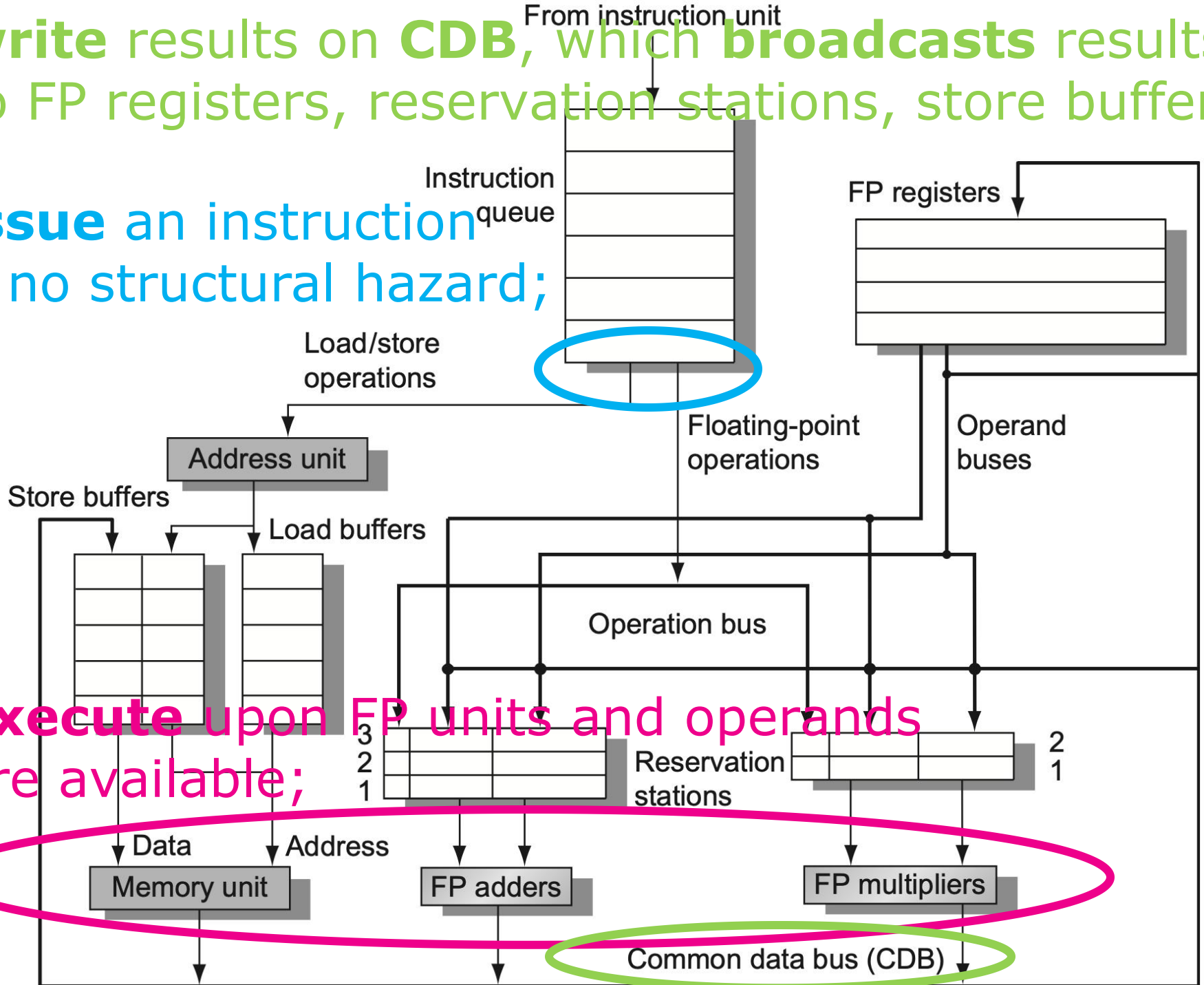
how to tomasulo?



write results on **CDB**, which **broadcasts** results to FP registers, reservation stations, store buffers

issue an instruction if no structural hazard;

execute upon FP units and operands are available;



Tomasulo's Algorithm

- Issue
- Execute
- Write result

Tomasulo's Algorithm

- Issue/Dispatch

get next instr from instr queue head
upon empty reservation-station/buffer;
rename registers if necessary;

- Execute

- Write result

Tomasulo's Algorithm

- Issue

- Execute

when all operands are available,
execute the operation at functional unit;
no instr is allowed to execute until a
branch that precedes the instruction in
program order has completed;

- Write result

Tomasulo's Algorithm

- Issue
- Execute
- Write result
 - when the result is available, write it on CDB, which broadcasts it to registers, reservation stations, and store buffers

Reservation Station Status

- Seven fields:

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Reservation Station Status

- Seven fields:

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

for load/store buffer

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Register File Status

- One field
Qi: number of the reservation station that contains the operation whose result should be stored into this register;
set to blank or 0 if no currently active instruction is computing a result destined for this register → the value is simply the register contents

Load/Store Buffer Status

- One field
A: hold the result of the effective address once the first step of execution has been completed;
as for load/store, effective address is computed in the execution stage

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
f1d f6,32(x2)	✓	✓	✓
f1d f2,44(x3)	✓	✓	
fmul.d f0,f2,f4	✓		
fsub.d f8,f2,f6	✓		
fdiv.d f1 f0,f6	✓		
fadd.d f6 f8,f2	✓		

1st fld has completed and written
2nd fld has executed

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
f1d f6,32(x2)	✓	✓	f6 ready ✓
f1d f2,44(x3)	✓	✓	f2 not ready
fmul.d f0,f2,f4	✓		
fsub.d f8,f2,f6	✓		
fdiv.d f1 f0,f6	✓		
fadd.d f6 f8,f2	✓		

1st fld has completed and written
2nd fld has executed

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6,32(x2)	✓	✓	f6 ready ✓
fld f2,44(x3)	✓	✓	f2 not ready
fmul.d f0,f2,f4	✓		
fsub.d f8,f2,f6	✓		
fdiv.d f1 ,f0,f6	✓		
fadd.d t ,f8,f2	✓		

1st fld has completed and written
2nd fld has executed
fmul/fsub/fadd cannot execute

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6, 32(x2)	✓	✓	f6 ready ✓
fld f2, 44(x3)	✓	✓	f2 not ready
fmul.d f0, f2 , f4	✓		f0 not ready
fsub.d f8, f2 , f6	✓		
fdiv.d f1 , f0, f6	✓		
fadd.d f0, f8 , f2	✓		

1st fld has completed and written
2nd fld has executed
fmul/fsub/fadd cannot execute

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6,32(x2)	✓	✓	f6 ready ✓
fld f2,44(x3)	✓	✓	f2 not ready
fmul.d f0, f2 ,f4	✓		f0 not ready
fsub.d f8, f2 ,f6	✓		
fdiv.d f1 f0 ,f6	✓		
fadd.d f6 f8, f2	✓		

1st fld has completed and written

2nd fld has executed

fmul/fsub/fadd cannot execute

fdiv cannot execute

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
f1d f6,32(x2)	✓	✓	f6 ready ✓
f1d f2,44(x3)	✓	✓	f2 not ready
fmul.d f0,f2,f4	✓		f0 not ready
fsub.d f8,f2, f6	✓		
fdiv.d f1 f0, f6	✓		
fadd.d f0 f8,f2	✓		

f6 WAR: fsub.d/fdiv.d → fadd.d
why co-issued?

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
f1d f6,32(x2)	✓	✓	f6 ready ✓
f1d f2,44(x3)	✓	✓	f2 not ready
fmul.d f0,f2,f4	✓		f0 not ready
fsub.d f8,f2,f6	✓		
fdiv.d f0,f6	✓		
fadd.d f0,f8,f2	✓		

f6 WAR: fsub.d/fdiv.d → fadd.d
why co-issued?

fmul/fsub/fadd cannot execute
fdiv cannot execute

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6,32(x2)	✓	✓	f6 ready ✓
fld f2,44(x3)	✓	✓	f2 not ready
fmul.d f0,f2,f4	✓		f0 not ready
fsub.d f8,f2,f6	✓		
fdiv.d f0,f6	✓		
fadd.d f0,f8,f2	✓		

f6 WAR: fsub.d/fdiv.d → fadd.d
why co-issued: register renaming
fsub/fdiv get f6 from fld buffer
fadd latter write wins

Tomasulo Example

Instruction		Instruction status		
		Issue	Execute	Write result
f1d	f6,32(x2)	✓	✓	✓
f1d	f2,44(x3)	✓	✓	
fmul.d	f0,f2,f4	✓		
fsub.d	f8,f2,f6	✓		
fdiv.d	f0,f0,f6	✓		
fadd.d	f6,f8,f2	✓		

Reservation stations							
Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

Tomasulo Example

Instruction		Instruction status		
		Issue	Execute	Write result
fld	f6,32(x2)	✓	✓	✓
fld	f2,44(x3)	✓	✓	
fmul.d	f0,f2,f4	✓	Op—The operation to perform on source operands S1 and S2. Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary. Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field. A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here. Busy—Indicates that this reservation station and its accompanying functional unit are occupied.	
fsub.d	f8,f2,f6	✓		
fdiv.d	f1 f0,f6	✓		
fadd.d	6 f8,f2	✓		

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6,32(x2)	✓	✓	✓
fld f2,44(x3)	✓	✓	
fmul.d f0,f2,f4	✓		
fsub.d f8,f2,f6	✓		
fdiv.d f1,f0,f6	✓		
fadd.d f6,f8,f2	✓		

Op—The operation to perform on source operands S1 and S2.
 Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
 Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
 A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
 Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No	fld has completed and written					
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6,32(x2)	✓	✓	✓
fld f2,44(x3)	✓	✓	
fmul.d f0,f2,f4	✓	Op—The operation to perform on source operands S1 and S2. Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary. Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field. A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here. Busy—Indicates that this reservation station and its accompanying functional unit are occupied.	
fsub.d f8,f2,f6	✓		
fdiv.d f1,f0,f6	✓		
fadd.d f6,f8,f2	✓		

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load	fld has computed effective address				44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6,32(x2)	✓	✓	✓
fld f2,44(x3)	✓	✓	
fmul.d f0,f2,f4	✓		
fsub.d f8,f2,f6	✓		
fdiv.d f1 f0,f6	✓		
fadd.d f6 f8,f2	✓		

Op—The operation to perform on source operands S1 and S2.
 Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
 Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
 A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
 Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

directly from f4

wait for f2 from fld

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
f1d f6,32(x2)	✓	✓	✓
f1d f2,44(x3)	✓	✓	
fmul.d f0,f2,f4	✓		
fsub.d f8,f2,f6	✓		
fdiv.d f1 f0,f6	✓		
fadd.d f6 f8,f2	✓		

Op—The operation to perform on source operands S1 and S2.
Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB			Mem[32 + Regs[x2]]	Load2	
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL			Regs[f4]	Load2	
Mult2	Yes	DIV			Mem[32 + Regs[x2]]	Mult1	

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
f1d f6,32(x2)	✓	✓	✓
f1d f2,44(x3)	✓	✓	
fmul.d f0,f2,f4	✓	Op—The operation to perform on source operands S1 and S2. Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.	
fsub.d f8,f2,f6	✓	Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.	
fdiv.d f1 f0,f6	✓	A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.	
fadd.d f6 f8,f2	✓	Busy—Indicates that this reservation station and its accompanying functional unit are occupied.	

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Mem[32 + Regs[x2]]	Load1	Load2	
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

directly from Load1

wait for f0 ← fmul

Tomasulo Example

Instruction	Instruction status		
	Issue	Execute	Write result
fld f6,32(x2)	✓	✓	✓
fld f2,44(x3)	✓	✓	
fmul.d f0,f2,f4	✓		
fsub.d f8,f2,f6	✓		
fdiv.d f1,f0,f6	✓		
fadd.d f6,f8,f2	✓		

Op—The operation to perform on source operands S1 and S2.
 Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
 Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
 A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
 Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

wait for f8 from fsub wait for f2 from fld

Tomasulo Example

Instruction		Instruction status		
		Issue	Execute	Write result
f1d	f6,32(x2)	✓	✓	✓
f1d	f2,44(x3)	✓	✓	
fmul.d	f0,f2,f4	✓		
fsub.d	f8,f2,f6	✓		
fdiv.d	f1f0,f6	✓		
fadd.d	f6f8,f2	✓		

Qi—The number of the reservation station that contains the operation whose result should be stored into this register. If the value of *Qi* is blank (or 0), no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

Register status									
Field	f0	f2	f4	f6	f8	f10	f12	...	f30
Qi	Mult1	Load2		Add2	Add1	Mult2			

Tomasulo Example

Instruction		Instruction status		
		Issue	Execute	Write result
f1d	f6, 32(x2)	✓	✓	✓
f1d	f2, 44(x3)	✓	✓	
fmul.d	f0, f2, f4	✓		
fsub.d	f8, f2, f6	✓		
fdiv.d	f0, f6	✓		
fadd.d	f0, f8, f2	✓		

check destination registers

Q_i —The number of the reservation station that contains the operation whose result should be stored into this register. If the value of Q_i is blank (or 0), no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

Register status									
Field	f0	f2	f4	f6	f8	f10	f12	...	f30
Q_i	Mult1	Load2		Add2	Add1	Mult2			

Tomasulo Example

		Instruction status			
Instruction		Issue	Execute	Write result	
f1d	f6,32(x2)	✓	✓	✓	
f1d	f2,44(x3)	✓	✓		
fmul.d	f0,f2,f4	✓			
fsub.d	f8,f2,f6	✓			
fdiv.d	f,f0,f6	✓			
fadd.d	f,f8,f2	✓			

check destination registers

Qi—The number of the reservation station that contains the operation whose result should be stored into this register. If the value of Qi is blank (or 0), no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

check corresponding instructions and reservation stations and buffers

		Register status						
Field		f0	f2	f4	f6	f8	f10	f12 ... f30
Qi		Mult1	Load2		Add2	Add1	Mult2	

Tomasulo Example

fmul.d is ready to write result

Instruction		Instruction status		
		Issue	Execute	Write result
f1d	f6,32(x2)	✓	✓	✓
f1d	f2,44(x3)	✓	✓	✓
fmul.d	f0,f2,f4	✓	✓	
fsub.d	f8,f2,f6	✓	✓	✓
fdiv.d	f1f0,f6	✓		
fadd.d	f0f8,f2	✓	✓	✓

Reservation stations						
Name	Busy	Op	Vj	Vk	Qj	Qk
Load1	No					
Load2	No					
Add1	No					
Add2	No					
Add3	No					
Mult1	Yes	MUL	Mem[44 + Regs[x3]]	Regs[f4]		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1	

Register status									
Field	f0	f2	f4	f6	f8	f10	f12	...	f30
Qi	Mult1					Mult2			

show me the code

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi $\neq$ 0) {RS[r].Qj $\leftarrow$ RegisterStat[rs].Qi} else {RS[r].Vj $\leftarrow$ Regs[rs]; RS[r].Qj $\leftarrow$ 0}; if (RegisterStat[rt].Qi $\neq$ 0) {RS[r].Qk $\leftarrow$ RegisterStat[rt].Qi else {RS[r].Vk $\leftarrow$ Regs[rt]; RS[r].Qk $\leftarrow$ 0}; RS[r].Busy $\leftarrow$ yes; RegisterStat[rd].Qi $\leftarrow$ r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi $\neq$ 0) {RS[r].Qj $\leftarrow$ RegisterStat[rs].Qi} else {RS[r].Vj $\leftarrow$ Regs[rs]; RS[r].Qj $\leftarrow$ 0}; RS[r].A $\leftarrow$ imm; RS[r].Busy $\leftarrow$ yes;
Load only		RegisterStat[rt].Qi $\leftarrow$ r;
Store only		if (RegisterStat[rt].Qi $\neq$ 0) {RS[r].Qk $\leftarrow$ RegisterStat[rt].Qi} else {RS[r].Vk $\leftarrow$ Regs[rt]; RS[r].Qk $\leftarrow$ 0};
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A $\leftarrow$ RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	$\forall x$ (if (RegisterStat[x].Qi = r) {Regs[x] $\leftarrow$ result; RegisterStat[x].Qi $\leftarrow$ 0}); $\forall x$ (if (RS[x].Qj = r) {RS[x].Vj $\leftarrow$ result; RS[x].Qj $\leftarrow$ 0}); $\forall x$ (if (RS[x].Qk = r) {RS[x].Vk $\leftarrow$ result; RS[x].Qk $\leftarrow$ 0}); RS[r].Busy $\leftarrow$ no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] $\leftarrow$ RS[r].Vk; RS[r].Busy $\leftarrow$ no;

Tomasulo's Algorithm

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi $\neq$ 0) {RS[r].Qj $\leftarrow$ RegisterStat[rs].Qi} else {RS[r].Vj $\leftarrow$ Regs[rs]; RS[r].Qj $\leftarrow$ 0}; if (RegisterStat[rt].Qi $\neq$ 0) {RS[r].Qk $\leftarrow$ RegisterStat[rt].Qi else {RS[r].Vk $\leftarrow$ Regs[rt]; RS[r].Qk $\leftarrow$ 0}; RS[r].Busy $\leftarrow$ yes; RegisterStat[rd].Qi $\leftarrow$ r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi $\neq$ 0) {RS[r].Qj $\leftarrow$ RegisterStat[rs].Qi} else {RS[r].Vj $\leftarrow$ Regs[rs]; RS[r].Qj $\leftarrow$ 0}; RS[r].A $\leftarrow$ imm; RS[r].Busy $\leftarrow$ yes;
Load only		RegisterStat[rt].Qi $\leftarrow$ r;
Store only		if (RegisterStat[rt].Qi $\neq$ 0) {RS[r].Qk $\leftarrow$ RegisterStat[rt].Qi} else {RS[r].Vk $\leftarrow$ Regs[rt]; RS[r].Qk $\leftarrow$ 0};
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A $\leftarrow$ RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	$\forall x$ (if (RegisterStat[x].Qi = r) {Regs[x] $\leftarrow$ result; RegisterStat[x].Qi $\leftarrow$ 0}); $\forall x$ (if (RS[x].Qj = r) {RS[x].Vj $\leftarrow$ result; RS[x].Qj $\leftarrow$ 0}); $\forall x$ (if (RS[x].Qk = r) {RS[x].Vk $\leftarrow$ result; RS[x].Qk $\leftarrow$ 0}); RS[r].Busy $\leftarrow$ no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] $\leftarrow$ RS[r].Vk; RS[r].Busy $\leftarrow$ no;

**Tomasulo's
Algorithm
step-by-step**

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	rd: destination rs, rt: source RS: reservation station <pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].i ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Load only		RegisterStat[rt].Qi ← r;
Store only		<pre> if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result;RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result;RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

Tomasulo's

Algorithm

step-by-step

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].i ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Load only		RegisterStat[rt].Qi ← r;
Store only		<pre> if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

otherwise, structural hazard should wait in instruction queue

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	<pre> if (RegisterStat[rs].Qi ≠ 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qi ← 0}; if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi ≠ 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Load only		RegisterStat[rt].Qi ← r ;
Store only		<pre> if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x (if (RegisterStat[x].Qi = r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x (if (RS[x].Qj = r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x (if (RS[x].Qk = r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

Qi != 0 indicates that rs waits for some other result, which is also waited by r

Qi—The number of the reservation station that contains the operation whose result should be stored into this register. If the value of Qi is blank (or 0), no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qi ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].i ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qi ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Load only		RegisterStat[rt].Qi ← r ;
Store only		<pre> if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

**Qi = 0 indicates that rs value is available,
get the value from rs and set $r.Qj$ to 0**

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qi ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Store only		<pre> if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

same policy for the other operand rt

hold imm for computing effective address

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].i ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Load only		RegisterStat[rt].Qi ← r ;
Store only		<pre> if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Load only		RegisterStat[rt].Qi ← r;
Store only		<pre> if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi $\neq$ 0) {RS[r].Qj $\leftarrow$ RegisterStat[rs].Qi} else {RS[r].Vj $\leftarrow$ Regs[rs]; RS[r].Qj $\leftarrow$ 0}; if (RegisterStat[rt].Qi $\neq$ 0) {RS[r].Qk $\leftarrow$ RegisterStat[rt].Qi} else {RS[r].Vk $\leftarrow$ Regs[rt]; RS[r].Qk $\leftarrow$ 0}; RS[r].Busy $\leftarrow$ yes; RegisterStat[rd].Qi $\leftarrow$ r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi $\neq$ 0) {RS[r].Qj $\leftarrow$ RegisterStat[rs].Qi} else {RS[r].Vj $\leftarrow$ Regs[rs]; RS[r].Qj $\leftarrow$ 0}; RS[r].A $\leftarrow$ imm; RS[r].Busy $\leftarrow$ yes;
Load only		RegisterStat[rt].Qi $\leftarrow$ r;
Store only		if (RegisterStat[rt].Qi $\neq$ 0) {RS[r].Qk $\leftarrow$ RegisterStat[rt].Qi} else {RS[r].Vk $\leftarrow$ Regs[rt]; RS[r].Qk $\leftarrow$ 0};
FP: compute upon both operands are available		
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A $\leftarrow$ RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	$\forall x$ (if (RegisterStat[x].Qi= r) {Regs[x] $\leftarrow$ result; RegisterStat[x].Qi $\leftarrow$ 0}); $\forall x$ (if (RS[x].Qj= r) {RS[x].Vj $\leftarrow$ result; RS[x].Qj $\leftarrow$ 0}); $\forall x$ (if (RS[x].Qk= r) {RS[x].Vk $\leftarrow$ result; RS[x].Qk $\leftarrow$ 0}); RS[r].Busy $\leftarrow$ no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] $\leftarrow$ RS[r].Vk; RS[r].Busy $\leftarrow$ no;

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;
Load only		RegisterStat[rt].Qi ← r;
Store only		if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};
load/store: compute effective address		
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;
Load only		RegisterStat[rt].Qi ← r;
Store only		if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};
load: read data from memory		
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;
Load only		RegisterStat[rt].Qi ← r;
Store only		if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A] write result to rd
Write result FP operation or load	Execution complete at r & CDB available result available	 ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	<pre> if (RegisterStat[rs].Qi ≠ 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r; </pre>
Load or store	Buffer r empty	<pre> if (RegisterStat[rs].Qi ≠ 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Load only		RegisterStat[rt].Qi ← r ;
Store only		<pre> if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	<pre> ∀x (if (RegisterStat[x].Qi = r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x (if (RS[x].Qj = r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x (if (RS[x].Qk = r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execution complete at r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

write result to
dependent source operands

result available

Op—The operation to perform on source operands S1 and S2.
Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the effective address.
A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;
Load only		RegisterStat[rt].Qi ← r;
Store only		if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;
Load only		RegisterStat[rt].Qi ← r;
Store only		if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

how to tomasulo with static?

Tomasulo & Loop Unrolling

- Example code sequence

```
Loop:  fld      f0, 0(x1)
       fmul.d   f4, f0, f2
       fsd      f4, 0(x1)
       addi     x1, x1, -8
       bne      x1, x2, Loop
```

Tomasulo & Loop Unrolling

two iterations & no complete instr

Instruction		Instruction status			
		From iteration	Issue	Execute	Write result
f1d	f0,0(x1)	1	✓	✓	
fmul.d	f4,f0,f2	1	✓		
fsd	f4,0(x1)	1	✓		
f1d	f0,0(x1)	2	✓	✓	
fmul.d	f4,f0,f2	2	✓		
fsd	f4,0(x1)	2	✓		

	Reservation stations						
Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	Yes	Load					Regs[x1] + 0
Load2	Yes	Load					Regs[x1] − 8
Add1	No						
Add2	No						
Add3	No						
Mult1	Yes	MUL		Regs[f2]	Load1		
Mult2	Yes	MUL		Regs[f2]	Load2		
Store1	Yes	Store	Regs[x1]			Mult1	
Store2	Yes	Store	Regs[x1] − 8			Mult2	

Register status									
Field	f0	f2	f4	f6	f8	f10	f12	...	f30
Qi	Load2		Mult2						

how to tomasulo with static?

remember branch prediction?

Branch Prediction

- **Example**

20-stage pipeline

1 out of 5 instructions is a branch

5-instruction block ends with a branch

- How long to fetch 500 instructions given various prediction accuracy?

100%: 1 instr per CC $\rightarrow$ 500 CC

99%: $500 + 20 \times 1 = 520$ CC

90%: $500 + 20 \times 10 = 700$ CC

how to tomasulo with static?

remember branch prediction:

while resolving branch, fetch&issue instr

if correct prediction, continue execution

if misprediction, re-fetch instructions

how to tomasulo with static?

if correct prediction, continue execution

how to exec before resolved?

how to exec before resolved?

speculate on branch outcome

execute program as if guesses are correct

how to exec before resolved?

fetch, issue, and execute instructions as if
branch predictions are always correct;
commit results till prediction is determined;
undo effects by misprediction;

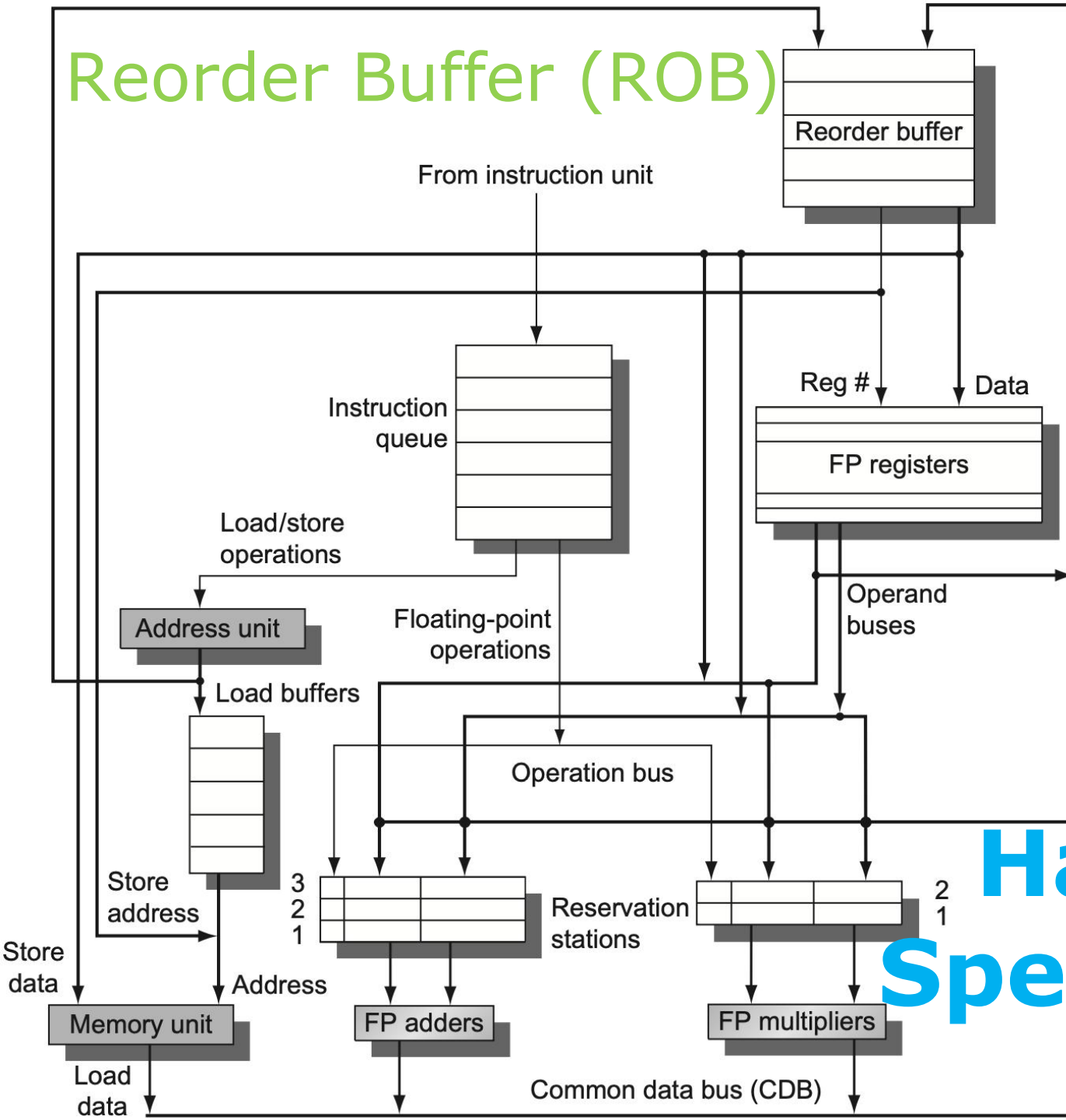
Hardware Speculation

- out-of-order execution & in-order commit
- Instruction Commit
allow an instruction to update register file or memory until the instruction is no longer speculative
- Reorder Buffer (ROB)
additional set of hardware buffers to hold the results of instructions that have finished execution but have not committed

Hardware Speculation

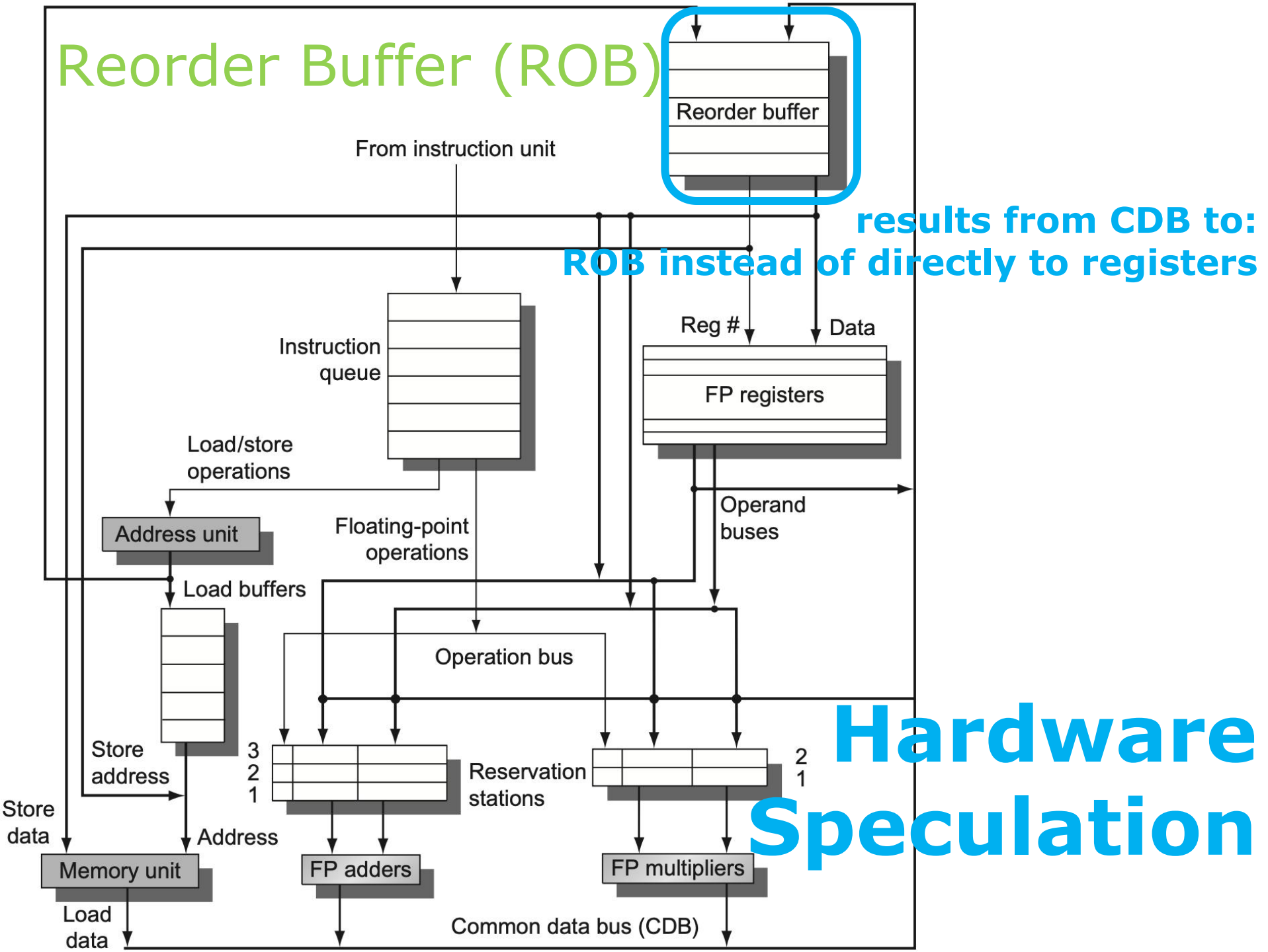
- out-of-order execution & in-order commit
- Instruction Commit
allow an instruction to update register file or memory until the instruction is no longer speculative
- Reorder Buffer (ROB)
similar to stations/buffers in Tomasulo,
act as a source of operands for instr;
results updated to registers/memory
until instruction commits;

Reorder Buffer (ROB)

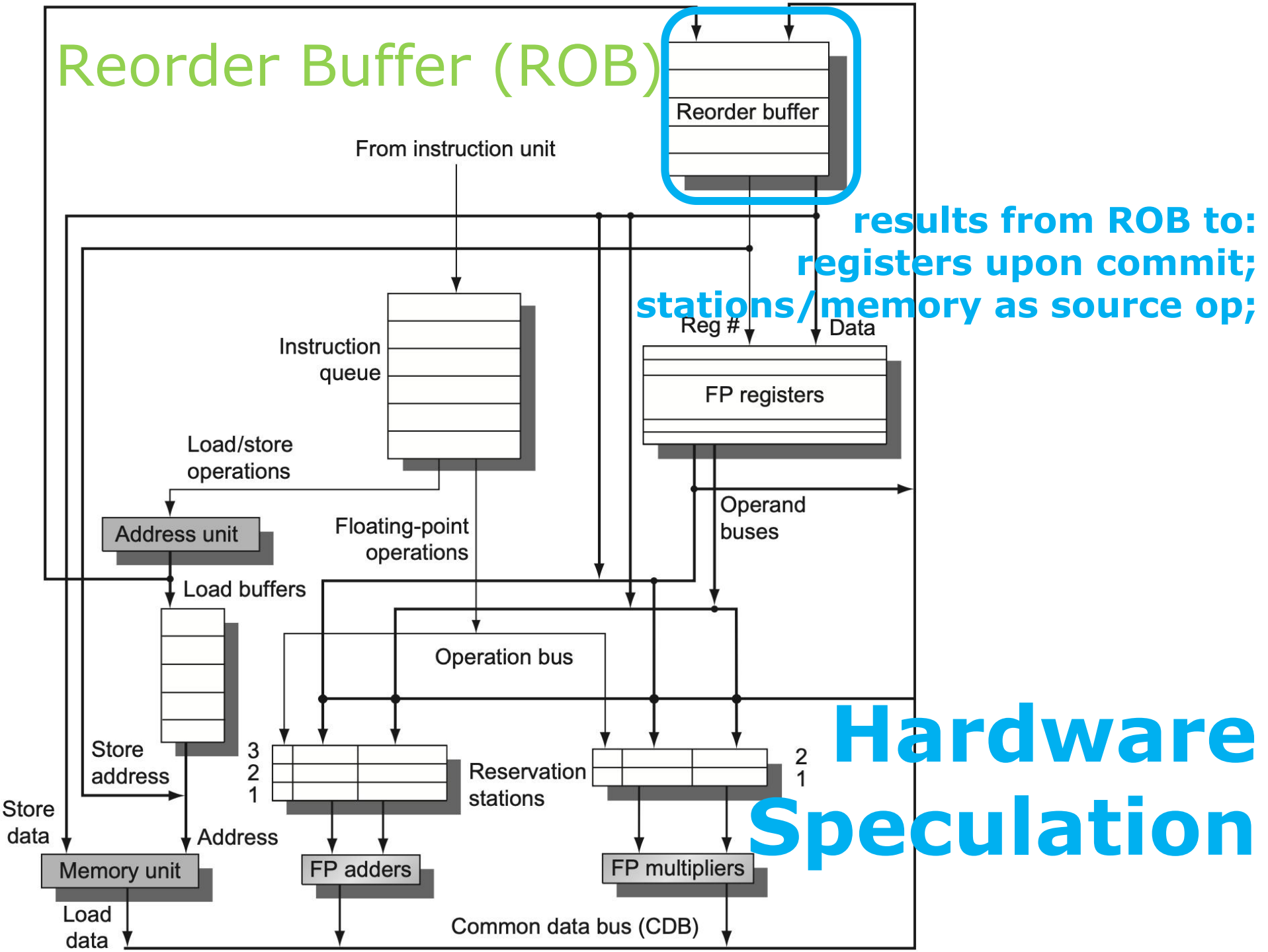


Hardware
Speculation

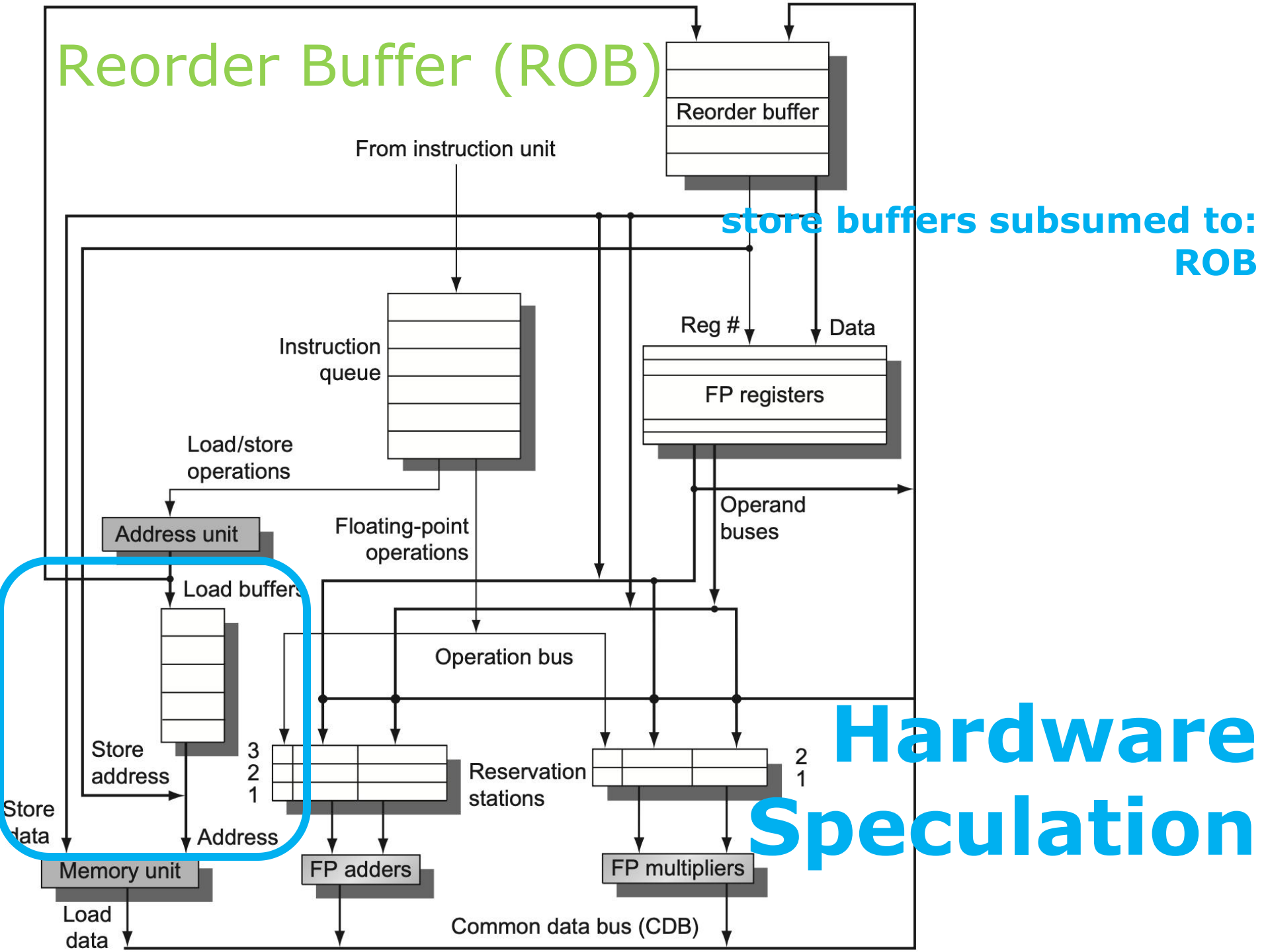
Reorder Buffer (ROB)



Reorder Buffer (ROB)



Reorder Buffer (ROB)



Hardware Speculation

- Reorder Buffer (ROB) entry fields:
- Instruction type
- Destination
- Value
- Ready

Hardware Speculation

- Reorder Buffer (ROB) entry fields:
- **Instruction type**
branch, store, register op (ALU/load)
- **Destination**
register # for loads and ALU operations
memory address for stores
- **Value**
hold insr result until instruction commits
- **Ready:** execution complete, value ready

Hardware Speculation

- Issue
- Execute
- Write result
- Commit

Hardware Speculation

- Issue

get next instr from instr queue head
upon empty reservation-station buffer
& empty ROB entry;
send operands to station if available in
either registers or ROB;
correlate station with ROB entry #;

- Execute
- Write result
- Commit

Hardware Speculation

- Issue

- Execute

when both operands are available,
execute the operation at functional unit;
stores only need base register for
effective address calculation;

- Write result
- Commit

Hardware Speculation

- Issue
- Execute
- Write result
 - when the result is available, write it on CDB (with ROB entry #), which broadcasts it to ROB and stations
- Commit

Hardware Speculation

- Issue
 - Execute
 - Write result
 - when the result is available, write it on CDB (with ROB entry #), which broadcasts it to ROB and stations
 - Commit
- not directly to registers/memory

Hardware Speculation

- Issue
- Execute
- Write result
- **Commit: Completion/Graduation**
branch with incorrect prediction
store
normal commit

Hardware Speculation

- Issue
- Execute
- Write result
- **Commit: Completion/Graduation**
branch with incorrect prediction
store / normal commit: instruction
reaches ROB head and its result is
available;
write result to dest register / memory;
remove instruction from ROB;

Hardware Speculation

- Issue
- Execute
- Write result
- Commit: Completion/Graduation
branch with incorrect prediction: flush
ROB & restart at the correct successor o
the branch; store
normal commit

Speculation Example

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f ,f0,f6	Execute	f	
6	Yes	fadd.d	f ,f8,f2	Write result	f	#4 + #2

both fld have committed
fmul.d is ready to commit

Instruction type

branch, store, register op (ALU/load)

Destination

register # for loads and ALU operations
memory address for stores

Value

hold insr result until instruction commits

Ready: execution complete, value ready

Speculation Example

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f ,f0,f6	Execute	f	still in execution due to long latency
6	Yes	fadd.d	f ,f8,f2	Write result	f	

both fld have committed
fmul.d is ready to commit

Instruction type

branch, store, register op (ALU/load)

Destination

register # for loads and ALU operations
memory address for stores

Value

hold insr result until instruction commits

Ready: execution complete, value ready

Speculation Exa

Op—The operation to perform on source operands S1 and S2.

Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.

Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.

A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.

Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

Reorder buffer

Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f1 f0,f6	Execute	f	
6	Yes	fadd.d	6 f8,f2	Write result	f1	#4 + #2

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	fmul.d	Mem[44 + Regs[x3]]	Regs[f4]			#3	
Mult2	Yes	fdiv.d		Mem[32 + Regs[x2]]	#3		#5	

could be blank as Mult1 is not busy?

Speculation Example

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f1 f0,f6	Execute	f	
6	Yes	fadd.d	f6 f8,f2	Write result	f1	#4 + #2

Reservation stations								
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	fmul.d	Mem[44 + Regs[x3]]	Regs[f4]			#3	
Mult2	Yes	fdiv.d		Mem[32 + Regs[x2]]	#3		#5	

which ROB entry's destination is this register

FP register status										
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	...	Yes	Yes

Speculation Example

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f1 f0,f6	Execute	f1	
6	Yes	fadd.d	f6 f8,f2	Write result	f6	#4 + #2

Reservation stations									
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A	
Load1	No								
Load2	No								
Add1	No								
Add2	No								
Add3	No								
Mult1	No	fmul.d	Mem[44 + Regs[x3]]	Regs[f4]			#3		
Mult2	Yes	fdiv.d		Mem[32 + Regs[x2]]	#3		#5		

FP register status										
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	...	Yes	Yes

which ROB entry's destination is this register

Speculation Example

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f ,f0,f6	Execute	f	
6	Yes	fadd.d	f ,f8,f2	Write result	f	#4 + #2

Reservation stations									
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A	
Load1	No								
Load2	No								
Add1	No								
Add2	No								
Add3	No								
Mult1	No	fmul.d	Mem[44 + Regs[x3]]	Regs[f4]				#3	
Mult2	Yes	fdiv.d		Mem[32 + Regs[x2]]	#3			#5	

FP register status										
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	...	Yes	Yes

which ROB entry's destination is this register

Speculation Example

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f ,f0,f6	Execute	f	
6	Yes	fadd.d	f1 ,f8,f2	Write result	f1	#4 + #2

Reservation stations									
Name	Busy	Op	Vj	Vk		Qj	Qk	Dest	A
Load1	No								
Load2	No								
Add1	No								
Add2	No								
Add3	No								
Mult1	No	fmul.d	Mem[44 + Regs[x3]]	Regs[f4]				#3	
Mult2	Yes	fdiv.d		Mem[32 + Regs[x2]]		#3		#5	

uncommitted instructions' destination registers

Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	...	Yes	Yes

Speculation Example

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 − #1
5	Yes	fdiv.d	f1 ,f0,f6	Execute	f1	
6	Yes	fadd.d	f0 ,f8,f2	Write result	f0	#4 + #2

Reservation stations									
Name	Busy	Op	Vj	Vk		Qj	Qk	Dest	A
Load1	No								
Load2	No								
Add1	No								
Add2	No								
Add3	No								
Mult1	No	fmul.d	Mem[44 + Regs[x3]]	Regs[f4]				#3	
Mult2	Yes	fdiv.d		Mem[32 + Regs[x2]]				#1	

FP register status										
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	...	Yes	Yes

wait for results to be written from ROB to destination registers upon commit

Speculation & Loop Unrolling

two iterations&fld/fmul committed

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f0,0(x1)	Commit	f0	Mem[0 + Regs[x1]]
2	No	fmul.d	f4,f0,f2	Commit	f4	#1 × Regs[f2]
3	Yes	fsd	f4,0(x1)	Write result	0 + Regs[x1]	#2
4	Yes	addi	x1,x1,-8	Write result	x1	Regs[x1] - 8
5	Yes	bne	x1,x2,Loop	Write result		
6	Yes	fld	f0,0(x1)	Write result	f0	Mem[#4]
7	Yes	fmul.d	f4,f0,f2	Write result	f4	#6 × Regs[f2]
8	Yes	fsd	f4,0(x1)	Write result	0 + #4	#7
9	Yes	addi	x1,x1,-8	Write result	x1	#4 - 8
10	Yes	bne	x1,x2,Loop	Write result		

FP register status									
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8
Reorder #	6				7				
Busy	Yes	No	No	No	Yes	No	No	...	No

show me the code, again

Status	Wait until	Action or bookkeeping
Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Hardware Speculation

Status	Wait until	Action or bookkeeping
Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Hardware Speculation step-by-step

Issue all instructions	Reservation station (r) and ROB (b) both available	if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;;} RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no;
		if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;;}
FP operations and stores		RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd;
FP operations		RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt;
Loads		RS[r].A ← imm;
Stores		
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Issue all instructions

FP operations and stores

FP operations

Loads

Stores

Reservation station (r) and ROB (b) both available

```
if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/
{h ← RegisterStat[rs].Reorder;
if (ROB[h].Ready)/* Instr completed already */
{RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;}
else {RS[r].Qj ← h;} /* wait for instruction */
} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;};
RS[r].Busy ← yes; RS[r].Dest ← b;
ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no;

if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/
{h ← RegisterStat[rt].Reorder;
if (ROB[h].Ready)/* Instr completed already */
{RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;}
else {RS[r].Qk ← h;} /* wait for instruction */
} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;};

RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes;
ROB[b].Dest ← rd;

RS[r].A ← imm; RegisterStat[rt].Reorder ← b;
RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt;

RS[r].A ← imm;
```

Store	Execution done at r and (RS[r].Qj == 0) and store is complete	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result to store	Execution done at r and store is complete	b ← RS[r].Dest; RS[r].Busy ← no; ∀x (if (RS[x].Qj == b) {RS[x].Vj ← RS[r].Vj; RS[x].Qj ← 0}); ∀x (if (RS[x].Qk == b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

otherwise, structural hazard should wait in instruction queue

Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>

if source operand rs is not available,

Write result all but store	Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	<pre> ROB[h].Value ← RS[r].Vk; </pre>
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>

if source operand rs is not available

monitor rs' ROB entry h's status till its result is available; pay attention to the assignment of h here, not necessarily the ROB head as defined by default;

Store	Execution done at r and	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;}} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;} ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>

if source operand rs is not available

Write result, monitor rs' ROB entry h's status till its result is available if the result is available,

send it from ROB to the reservation station;

Commit	Execution done at r and ROB[h].ready == yes Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> ROB[h].Value ← RS[r].Vk; d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>
--------	---	--

Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>

if source operand rs is available,
send it from register to the reservation station;

Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;}} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;} ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vi ← Regs[rs]; RS[r].Oi ← 0;}: RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>
occupy the reservation station, tag it with the corresponding ROB entry b, that is, the result of this reservation station will be written to b;		
Store	Execution done at r and (RS[r].Qk == 0)	<pre> ROB[h].Value ← RS[r].Vk; </pre>
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>
		Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
FP operations		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Loads		<pre> RS[r].A ← imm; </pre>
Stores		

**write instruction type to ROB entry b,
set destination register of ROB entry b as instruction's dest rd,
the result of this ROB entry b will be finally written to rd;**

Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre> <pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre> <pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre> <pre> RS[r].A ← imm; </pre>
FP operations		
Loads		
Stores		

similar policy to get the other operand rt ready;

Store	Write result all but store Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	<pre> ROB[h].Value ← RS[r].Vk; </pre>
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>

link ROB entry b to the corresponding register rd;
the result of this ROB entry b will be finally written to rd;

Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>
link ROB entry b to the corresponding register rd; the result of this ROB entry b will be finally written to rd; for load, the other operand comes from imm;		
Store	Execution done at r and (RS[r].Qk == 0)	<pre> ROB[h].Value ← RS[r].Vk; </pre>
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>
		Issue

Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre> <pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre> <pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre> RS[r].A ← imm;
FP operations		
Loads		
Stores		

for store, the other operand comes from imm;
store's destination is memory instead of destination register;

Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;

FP: compute upon both operands are available

FP operations		RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rt;
Loads		RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt;
Stores		RS[r].A ← imm;
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Execute

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;

load/store: compute effective address

FP operations		RegisterStat[rd].Reorder ← h; RegisterStat[rd].Busy ← yes; if (ROB[h].Ready) {
Loads		RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt;
Stores		RS[r].A ← imm;
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Execute

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
---------------	-------------------------------------	---

Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
-------------	--	-------------------------------

Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
-------------	---	------------------------

Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
-------	---	--------------------------------------

load/store: compute effective address
avoid RAW hazard, complete all stores before load;
may not have A-address conflict among them, for simplicity
without comparing address fields of them;

Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Execute

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;

load/store: compute effective address
value to store need not arrive until just before store commit;
value can be placed directly into the store's ROB entry by the
sourcing instruction;
so compute store's effective address when it is at ROB head,
and about to commit;

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; Vx(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); Vx(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Execute

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]

Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
-------	---	--------------------------------------

load/store: compute effective address
avoid RAW hazard, if there are earlier stores with same addr, store first, then load;

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Execute

Write result all but store	Execution done at r and CDB available	$b \leftarrow RS[r].Dest; RS[r].Busy \leftarrow no;$ $\forall x (if (RS[x].Qj == b) \{ RS[x].Vj \leftarrow result; RS[x].Qj \leftarrow 0 \});$ $\forall x (if (RS[x].Qk == b) \{ RS[x].Vk \leftarrow result; RS[x].Qk \leftarrow 0 \});$ $ROB[b].Value \leftarrow result; ROB[b].Ready \leftarrow yes;$
Store	Execution done at r and $(RS[r].Qk == 0)$	$ROB[h].Value \leftarrow RS[r].Vk;$
data to store is available, writted from station to ROB; according to prior requirement, store is ROB head, so ROB[h];		
FP operations		$b \leftarrow RS[r].Dest; RS[r].Busy \leftarrow no;$ $\forall x (if (RS[x].Qj == b) \{ RS[x].Vj \leftarrow result; RS[x].Qj \leftarrow 0 \});$ $\forall x (if (RS[x].Qk == b) \{ RS[x].Vk \leftarrow result; RS[x].Qk \leftarrow 0 \});$ $ROB[b].Value \leftarrow result; ROB[b].Ready \leftarrow yes;$
Loads		$RS[r].A \leftarrow imm;$
Stores		$RS[r].A \leftarrow imm;$
Execute FP op	$(RS[r].Qj == 0)$ and $(RS[r].Qk == 0)$	Compute results—operands are in Vj and Vk
Load step 1	$(RS[r].Qj == 0)$ and there are no stores earlier in the queue	$RS[r].A \leftarrow RS[r].Vj + RS[r].A;$
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from $Mem[RS[r].A]$
Store	$(RS[r].Qj == 0)$ and store at queue head	$ROB[h].Address \leftarrow RS[r].Vj + RS[r].A;$
Write result all but store	Execution done at r and CDB available	$b \leftarrow RS[r].Dest; RS[r].Busy \leftarrow no;$ $\forall x (if (RS[x].Qj == b) \{ RS[x].Vj \leftarrow result; RS[x].Qj \leftarrow 0 \});$ $\forall x (if (RS[x].Qk == b) \{ RS[x].Vk \leftarrow result; RS[x].Qk \leftarrow 0 \});$ $ROB[b].Value \leftarrow result; ROB[b].Ready \leftarrow yes;$
Store	Execution done at r and $(RS[r].Qk == 0)$	$ROB[h].Value \leftarrow RS[r].Vk;$
Commit	Instruction is at the head of the ROB (entry h) and $ROB[h].ready == yes$	$d \leftarrow ROB[h].Dest; /* register dest, if exists */$ $if (ROB[h].Instruction == Branch)$ $\{ if (branch is mispredicted)$ $\{ clear ROB[h], RegisterStat; fetch branch dest; \}; \}$ $else if (ROB[h].Instruction == Store)$ $\{ Mem[ROB[h].Destination] \leftarrow ROB[h].Value; \}$ $else /* put the result in the register destination */$ $\{ Regs[d] \leftarrow ROB[h].Value; \};$ $ROB[h].Busy \leftarrow no; /* free up ROB entry */$ $/* free up dest register if no one else writing it */$ $if (RegisterStat[d].Reorder == h) \{ RegisterStat[d].Busy \leftarrow no; \};$
		Write Result

Instruction is at the
head of the ROB (entry h)
and ROB[h].ready ==
yes

```

d ← ROB[h].Dest; /* register dest, if exists */
if (ROB[h].Instruction==Branch)
    {if (branch is mispredicted)
        {clear ROB[h], RegisterStat; fetch branch dest;}}
else if (ROB[h].Instruction==Store)
    {Mem[ROB[h].Destination] ← ROB[h].Value;}
else /* put the result in the register destination */
    {Regs[d] ← ROB[h].Value;};
ROB[h].Busy ← no; /* free up ROB entry */
/* free up dest register if no one else writing it */
if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

```

**commit an instruction only when it reaches ROB head, and
its result is available;**

Operations

Loads

Stores

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;}} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Commit

Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes

```
d ← ROB[h].Dest; /* register dest, if exists */
if (ROB[h].Instruction==Branch)
    {if (branch is mispredicted)
        {clear ROB[h], RegisterStat; fetch branch dest;}}
else if (ROB[h].Instruction==Store)
    {Mem[ROB[h].Destination] ← ROB[h].Value;}
else /* put the result in the register destination */
    {Regs[d] ← ROB[h].Value;};
ROB[h].Busy ← no; /* free up ROB entry */
/* free up dest register if no one else writing it */
if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};
```

commit an instruction only when it reaches ROB head, and its result is available;
if misprediction, clear ROB (according to the textbook?)
(at least clear mispredicted branch&its dependent instructions)

Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;}} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Commit

Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>
--------	---	---

commit an instruction only when it reaches ROB head, and its result is available;

store: write result to memory

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;}} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>
--------	---	--

commit an instruction only when it reaches ROB head, and its result is available;

normal commit: write result to destination register

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;}} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Commit

Commit

Instruction is at the
head of the ROB (entry h)
and ROB[h].ready ==
yes

```
d ← ROB[h].Dest; /* register dest, if exists */  
if (ROB[h].Instruction==Branch)  
    {if (branch is mispredicted)  
        {clear ROB[h], RegisterStat; fetch branch dest;}}  
else if (ROB[h].Instruction==Store)  
    {Mem[ROB[h].Destination] ← ROB[h].Value;}  
else /* put the result in the register destination */  
    {Regs[d] ← ROB[h].Value;};  
ROB[h].Busy ← no; /* free up ROB entry */  
/* free up dest register if no one else writing it */  
if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};
```

**commit an instruction only when it reaches ROB head, and
its result is available;
after commit, release ROB entry and the corresponding station.**

Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;}} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Commit

Status	Wait until	Action or bookkeeping
Issue all instructions		<pre> if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	ROB[h].Address ← RS[r].Vj + RS[r].A;
Write result all but store	Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;}; </pre>

Hardware Speculation Commit

Hardware Speculation

- **Example:**

free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	?		
mul	x1, x5, x6				
add	x3, x7, x8				
mul	x1, x1, x3				
sub	x4, x1, x5				
add	x1, x4, x2				

Hardware Speculation

- **Example:**

free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6				
add	x3, x7, x8				
mul	x1, x1, x3				
sub	x4, x1, x5				
add	x1, x4, x2				

Hardware Speculation

- **Example:** assume single-issue pipeline
free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8				
mul	x1, x1, x3				
sub	x4, x1, x5				
add	x1, x4, x2				

Hardware Speculation

- **Example:**

free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8	3	4	5	45
mul	x1, x1, x3				
sub	x4, x1, x5				
add	x1, x4, x2				

Hardware Speculation

- **Example:** need mul reservation station
free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8	3	4	5	45
mul	x1, x1, x3	14			
sub	x4, x1, x5				
add	x1, x4, x2				

Hardware Speculation

- **Example:**

free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8	3	4	5	45
mul	x1, x1, x3	14	15	25	46
sub	x4, x1, x5				
add	x1, x4, x2				

Hardware Speculation

- **Example:** need x1 as source operand
free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8	3	4	5	45
mul	x1, x1, x3	14	15	25	46
sub	x4, x1, x5	15	26		
add	x1, x4, x2				

Hardware Speculation

- **Example:**

free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8	3	4	5	45
mul	x1, x1, x3	14	15	25	46
sub	x4, x1, x5	15	26	27	47
add	x1, x4, x2				

Hardware Speculation

- **Example:** need x2 as source operand
free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8	3	4	5	45
mul	x1, x1, x3	14	15	25	46
sub	x4, x1, x5	15	26	27	47
add	x1, x4, x2	16	43		

Hardware Speculation

- **Example:**

free reservation station on result
broadcast (not instruction dispatch);
issue, capture, dispatch in same cycle;
1-cycle add, 10-cycle mul, 40-cycle div;

inst	operand	issue	exe	write	commit
div	x2, x3, x4	1	2	42	43
mul	x1, x5, x6	2	3	13	44
add	x3, x7, x8	3	4	5	45
mul	x1, x1, x3	14	15	25	46
sub	x4, x1, x5	15	26	27	47
add	x1, x4, x2	16	43	44	48

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty <i>otherwise, structural hazard</i>	if (RegisterStat[rs].Qi ≠ 0) <i>reg file status</i> {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; <i>indicate that source operand is available in Vj</i> if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Qi ← r; <i>result to write into reg rd</i>
Load or store	Buffer r empty <i>otherwise, structural hazard</i>	if (RegisterStat[rs].Qi ≠ 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; <i>load/store needs only one source op for computing addr</i>
Load only	<i>A for storing addr-meminp</i>	RegisterStat[rt].Qi ← r; <i>load target</i>
Store only	<i>then for store addr result</i>	if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; <i>both operands are available</i>
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk <i>address result</i>
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A; <i>imm op for computing effective address is available</i>
Load step 2	Load step 1 complete	Read from Mem[RS[r].A] <i>wait result from r</i>
Write result FP operation or load	Execution complete at r & CDB available	$\forall x(\text{if}(\text{RegisterStat}[x].\text{Qi} = r) \{ \text{Regs}[x] \leftarrow \text{result}; \text{RegisterStat}[x].\text{Qi} \leftarrow 0 \});$ $\forall x(\text{if}(\text{RS}[x].\text{Qj} = r) \{ \text{RS}[x].\text{Vj} \leftarrow \text{result}; \text{RS}[x].\text{Qj} \leftarrow 0 \});$ $\forall x(\text{if}(\text{RS}[x].\text{Qk} = r) \{ \text{RS}[x].\text{Vk} \leftarrow \text{result}; \text{RS}[x].\text{Qk} \leftarrow 0 \});$ <i>RAW dependence, write result to dependent source operands</i>
Store	Execution complete at r & RS[r].Qk = 0 <i>data available</i>	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

how i understood tomasulo

Status	Wait until	Action or bookkeeping
Issue all instructions		<pre> if (RegisterStat[rs].Busy) /* in-flight instr. writes rs */ { h ← RegisterStat[rs].Reorder; ? ROB head? if (ROB[h].Ready) /* Instr completed already */ { RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0; } else { RS[r].Qj ← h; } /* wait for instruction */ } else { RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0; }; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rt].Busy) /* in-flight instr writes rt */ { h ← RegisterStat[rt].Reorder; if (ROB[h].Ready) /* Instr completed already */ { RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0; } else { RS[r].Qk ← h; } /* wait for instruction */ } else { RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0; }; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	<pre> RS[r].A ← RS[r].Vj + RS[r].A; </pre>
Load step 2	Load step 1 done and all stores earlier in ROB have different address	<pre> Read from Mem[RS[r].A] </pre>
Store	(RS[r].Qj == 0) and store at queue head	<pre> ROB[h].Address ← RS[r].Vj + RS[r].A; </pre>
Write result all but store	Execution done at r and CDB available	<pre> b ← RS[r].Dest; RS[r].Busy ← no; ∀x (if (RS[x].Qj == b) { RS[x].Vj ← result; RS[x].Qj ← 0; }); ∀x (if (RS[x].Qk == b) { RS[x].Vk ← result; RS[x].Qk ← 0; }); ROB[b].Value ← result; ROB[b].Ready ← yes; </pre>
Store	Execution done at r and (RS[r].Qk == 0)	<pre> ROB[h].Value ← RS[r].Vk; </pre>
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	<pre> d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction == Branch) { if (branch is mispredicted) { clear ROB[h]; RegisterStat; fetch branch dest; } else if (ROB[h].Instruction == Store) { Mem[ROB[h].Destination] ← ROB[h].Value; } else /* put the result in the register */ { Regs[d] ← ROB[h].Value; }; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder == h) { RegisterStat[d].Busy ← no; }; </pre>

and speculation

Review

- Branch prediction
- Register renaming
- Scoreboarding
- Tomasulo
- Speculation